Software Requirements Specification

for

MaintainGov: Automated Review-to-Maintenance- Backlog Pipeline for Local Government Super App

Version 1.0 approved

Prepared by Kelompok 5

Gerard Viriyandra (187241047)
Demas Axel Marfiansyah (187241043)
Muhammad Hanif (187241062)
Valisha Vine Ravelina (187241012)

Universitas Airlangga

8th October 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

1. Introduction 1
    1.1 Purpose 1
    1.2 Document Conventions 1
    1.3 Intended Audience and Reading Suggestions 3
    1.4 Product Scope 4
    1.5 References 5
2. Overall Description 7
    2.1 Product Perspective 7
    2.2 Product Functions 7
    2.3 User Classes and Characteristics 10
    2.4 Operating Environment 10
    2.5 Design and Implementation Constraints 11
    2.6 User Documentation 12
    2.7 Assumptions and Dependencies 13
3. External Interface Requirements 14
    3.1 User Interfaces 14
    3.2 Hardware Interfaces 25
    3.3 Software Interfaces 27
    3.4 Communications Interfaces 29
4. System Features 33
    4.1 System Feature Overview 33
    4.2 System Feature 1 : Login 34
    4.3 System Feature 2 : Menganalisis Feedback (UC01) 37
    4.4 System Feature 3 : Memvalidasi Hasil Feedback (UC02) 40
    4.5 System Feature 4 : Mengedit Maintenance Backlog (UC03) 43
    4.6 System Feature 5 : Mengerjakan Maintenance Issue (UC04) 45
    4.7 System Feature 6 : Mengajukan Pull Request (UC05) 48
    4.8 System Feature 7 : Meninjau Pull Request (UC06) 50
    4.9 System Feature 8 : Logout 53
5. Other Nonfunctional Requirements 55
    5.1 Performance Requirements 55
    5.2 Safety Requirements 56

5.3 Security Requirements 57
5.4 Software Quality Attributes 57
5.5 Business Rules 59
6. Other Requirements 61

Revision History

Name	Date	Reason For Changes	Version
Initial Version	8th October 2025	Initial Software Requirement Specification for MaintainGov	1.0

1. Introduction

1.1 Purpose

Spesifikasi Kebutuhan Perangkat Lunak atau *Software Requirements Specification* (SRS) ini disusun untuk MaintainGov: Automated Review-to-Maintenance-Backlog Pipeline for Local Government Super App versi 1.0. MaintainGov merupakan aplikasi berbasis *web* yang dirancang untuk membantu Tim Maintenance dan Tim Developer mengubah *feedback* pengguna menjadi aktivitas *software maintenance* yang terstruktur, mulai dari analisis *feedback*, pembentukan *Maintenance Case*, pengelolaan *Maintenance Backlog*, pengerjaan *Maintenance Issue*, hingga *Pull Request* dan *Code Review*.

SRS ini mendefinisikan kebutuhan fungsional dan nonfungsional, kebutuhan antarmuka, data, keamanan, integrasi, aturan bisnis, serta batasan sistem yang menjadi acuan selama proses perancangan, pengembangan, pengujian, dan validasi MaintainGov. Cakupan tersebut juga mencakup fungsi pendukung berupa autentikasi, pengelolaan sesi, kontrol akses berbasis peran, sementara enam proses utama tetap mengacu pada UC01–UC06 yang telah ditetapkan pada perancangan sebelumnya.

Spesifikasi berfokus pada perilaku dan kondisi yang harus dipenuhi oleh sistem, bukan pada klaim keberhasilan implementasi. Detail teknis dicantumkan apabila telah ditetapkan sebagai kebutuhan atau batasan desain, sedangkan metode analisis otomatis seperti klasifikasi kategori, analisis sentimen, penentuan *severity*, ekstraksi issue, pengelompokan issue, dan perhitungan prioritas tetap ditentukan berdasarkan keputusan dan evaluasi teknis pada tahap implementasi.

1.2 Document Conventions

Dokumen ini ditulis dalam Bahasa Indonesia dengan mempertahankan istilah teknis berbahasa Inggris apabila istilah tersebut telah umum digunakan dalam pengembangan perangkat lunak atau merupakan bagian dari terminologi MaintainGov. Penggunaan istilah, identifier, status, tabel, gambar, serta requirement dilakukan secara konsisten agar kebutuhan sistem dapat dipahami dan ditelusuri tanpa menimbulkan interpretasi yang berbeda. Konvensi yang digunakan dalam dokumen ini adalah sebagai berikut:

a. Istilah “sistem” mengacu pada keseluruhan aplikasi MaintainGov beserta komponen perangkat lunak yang mendukung fungsinya.

b. Istilah “Tim Maintenance” mengacu pada pengguna yang menangani pengolahan *feedback*, validasi *Maintenance Case*, dan pengelolaan *Maintenance Backlog*.

c. Istilah “Tim Developer” mengacu pada pengguna yang menangani *Maintenance Issue*, perubahan *source code*, *Pull Request*, dan *Code Review*.

d. Istilah *Maintenance Case*, *Maintenance Backlog*, *Maintenance Issue*, *Pull Request*, *Code Review*, *severity*, *priority*, *repository*, dan istilah teknis lain digunakan secara konsisten di seluruh dokumen.

e. Status seperti Pending, Valid, Tidak Valid, Ready for Development, Sedang Dikerjakan, Ready for Pull Request, Open, Changes Requested, Approved, Ready for Merge, Merged, dan Selesai digunakan sesuai dengan konteks proses yang didefinisikan pada requirement terkait.

f. Kata “harus” digunakan untuk menyatakan kebutuhan yang bersifat wajib dan dapat diverifikasi melalui pengujian, inspeksi, analisis, atau demonstrasi.

g. Kata “tidak boleh” digunakan untuk menyatakan batasan atau perilaku yang dilarang oleh sistem.

h. Kata “dapat” digunakan untuk menyatakan kemampuan sistem atau tindakan yang tersedia bagi aktor tertentu.

i. Kebutuhan fungsional menggunakan identifier FR atau *Functional Requirement*, kebutuhan nonfungsional menggunakan NFR atau *Nonfunctional Requirement*, dan aturan bisnis menggunakan BR atau *Business Rule*.

j. Kebutuhan yang berasal dari enam use case utama menggunakan pola FR-UCxx-yy. Sebagai contoh, FR-UC01-01 menunjukkan kebutuhan fungsional pertama dari UC01 Menganalisis Feedback.

k. Kebutuhan pendukung menggunakan identifier sesuai domain, yaitu FR-AUTH-xx untuk Authentication dan Session Management, FR-DSH-xx, serta FR-GH-xx untuk integrasi GitHub yang berlaku lintas proses.

l. Kebutuhan nonfungsional menggunakan identifier berdasarkan atribut kualitas, antara lain NFR-PERF-xx untuk *Performance*, NFR-SEC-xx untuk *Security*, NFR-REL-xx untuk *Reliability*, NFR-USA-xx untuk *Usability*, dan NFR-MNT-xx untuk *Maintainability*.

m. Identifier UC01–UC06 dan US01–US06 tetap mengikuti Use Case dan User Story pada perancangan MaintainGov sebelumnya sehingga hubungan antara SRS, User Story, Activity Diagram, Sequence Diagram, serta skenario pengujian dapat dipertahankan.

n. Setiap requirement memiliki identifier unik. Identifier yang telah digunakan tidak dialihkan kepada requirement lain apabila requirement tersebut kemudian diubah atau dihapus.

o. Setiap *system feature* diberikan tingkat prioritas High, Medium, atau Low berdasarkan kepentingannya terhadap fungsi utama MaintainGov.

p. Seluruh tabel, gambar, dan diagram diberi nomor berdasarkan bab dan dilengkapi judul yang menggambarkan isi artefak.

q. Terminologi pada SRS, rancangan antarmuka, User Story Scenario, dan implementasi harus diselaraskan. Perbedaan terminologi atau alamat halaman yang ditemukan pada artefak sebelumnya tidak diubah secara implisit, melainkan diselaraskan pada tahap desain dan implementasi dengan tetap mempertahankan tujuan requirement yang sama.

Pemberian nomor requirement yang unik serta penetapan prioritas digunakan untuk mendukung keterlacakan dan pengelolaan requirement.

1.3 Intended Audience and Reading Suggestions

Dokumen SRS MaintainGov ditujukan kepada pihak-pihak yang terlibat langsung dalam penggunaan, pengembangan, dan pengujian sistem. Tim Maintenance dan Tim Developer menjadi pembaca utama karena kedua pihak tersebut merupakan aktor yang berinteraksi langsung dengan fungsi MaintainGov, sedangkan tim pengembang dan penguji menggunakan dokumen ini sebagai dasar implementasi dan verifikasi. Pihak-pihak yang menjadi pembaca utama dokumen ini adalah sebagai berikut:

a. **Tim Maintenance**
Tim Maintenance menggunakan dokumen ini untuk memahami dan memvalidasi kebutuhan yang berkaitan dengan pengolahan *feedback*, pemeriksaan *Maintenance Case*, serta pengelolaan *Maintenance Backlog*. Bagian yang terutama relevan adalah Overall Description, User Interfaces, serta System Features yang berkaitan dengan UC01–UC03.

b. **Tim Developer**
Tim Developer menggunakan dokumen ini untuk memahami kebutuhan pengerjaan *Maintenance Issue*, integrasi repository, pengajuan *Pull Request*, serta *Code Review*. Bagian yang terutama relevan adalah User Interfaces, Software Interfaces, serta System Features UC04–UC06.

c. **Tim Pengembang MaintainGov**
Tim Pengembang MaintainGov mencakup *system analyst*, perancang UI dan UX, *frontend developer*, *backend developer*, serta pengembang komponen analisis. Dokumen ini menjadi acuan dalam menerjemahkan kebutuhan menjadi rancangan antarmuka, model data, logika bisnis, API, integrasi eksternal, dan implementasi teknis. Tim ini terutama menggunakan Overall Description, External Interface Requirements, System Features, Other Nonfunctional Requirements, serta Other Requirements.

d. **Tim Penguji**
Tim Penguji menggunakan SRS sebagai dasar penyusunan *test case*, pengujian integrasi, dan *acceptance testing*. Functional Requirement, Nonfunctional Requirement, User Story Scenario berbasis Behat dan Gherkin, serta Requirements Traceability Matrix digunakan untuk memastikan setiap kebutuhan memiliki hasil yang dapat diverifikasi.

Untuk memperoleh pemahaman yang utuh mengenai MaintainGov, pembaca dapat mengikuti struktur pembahasan secara berurutan dari gambaran umum sistem, kebutuhan antarmuka, hingga kebutuhan fungsional dan nonfungsional. Tim Maintenance dan Tim Developer dapat memusatkan perhatian pada fitur yang berkaitan dengan peran masing-masing, sedangkan Tim Pengembang dan Tim Penguji menggunakan keseluruhan spesifikasi sebagai acuan implementasi, pengujian, dan verifikasi kebutuhan sistem.

1.4 Product Scope

MaintainGov merupakan aplikasi internal berbasis *web* yang dirancang untuk menghubungkan *feedback* pengguna dengan proses *software maintenance* secara terstruktur. Sistem tidak berhenti pada tahap analisis *feedback*, tetapi meneruskan hasil analisis menjadi *Maintenance Case*, *Maintenance Backlog*, pengerjaan *Maintenance Issue*, *Pull Request*, hingga *Code Review*. Alur tersebut memungkinkan setiap keputusan maintenance dan perubahan perangkat lunak ditelusuri kembali ke *feedback* yang menjadi sumber awalnya. Ruang lingkup MaintainGov versi 1.0 meliputi:

a. ***Authentication and Access Control***
Sistem menyediakan fungsi Login dan Logout, pengelolaan sesi, serta pembatasan akses berdasarkan peran. Tim Maintenance dan Tim Developer hanya dapat mengakses fungsi yang sesuai dengan kewenangannya. Akun pengguna disediakan secara internal sehingga versi 1.0 tidak mencakup registrasi mandiri.

b. ***Feedback Analysis***
Tim Maintenance dapat mengunggah file CSV berisi *feedback* pengguna. Sistem memvalidasi format dan struktur data sebelum menjalankan preprocessing, klasifikasi kategori, analisis sentimen, penentuan *severity*, ekstraksi issue, pengelompokan issue serupa, dan perhitungan prioritas. Hasil proses tersebut digunakan untuk membentuk *Maintenance Case* yang dapat ditinjau lebih lanjut.

c. ***Maintenance Case Validation***
Maintenance Case yang dihasilkan dari proses analisis memiliki status awal Pending. Tim Maintenance dapat memeriksa informasi case beserta *feedback* yang mendasarinya sebelum menentukan apakah case dinyatakan Valid atau Tidak Valid. Case yang dinyatakan valid diteruskan ke *Maintenance Backlog*, sedangkan case yang tidak valid tidak dilanjutkan ke tahap pengerjaan.

d. ***Maintenance Backlog Management***
Maintenance Case yang valid membentuk *Maintenance Backlog* dengan status awal Ready for Development. Tim Maintenance dapat melihat dan memperbarui informasi backlog sebelum pekerjaan teknis dimulai. Backlog menjadi acuan utama bagi Tim Developer dalam menentukan pekerjaan maintenance yang perlu ditindaklanjuti.

e. ***Maintenance Issue Development***
Tim Developer dapat melihat *Maintenance Issue* yang tersedia dan mengambil issue yang masih berstatus Ready for Development. Setelah issue diambil, sistem mencatat Developer yang bertanggung jawab dan mengubah status pekerjaan menjadi Sedang Dikerjakan. Proses pengerjaan mencakup analisis masalah, perubahan *source code*, pengujian, serta persiapan perubahan hingga issue berstatus Ready for Pull Request.

f. ***GitHub and Pull Request Integration***
MaintainGov terintegrasi dengan GitHub untuk mendukung pengelolaan repository, branch, dan *Pull Request*. Sistem menyimpan informasi yang diperlukan untuk menghubungkan perubahan kode dengan *Maintenance Issue* yang mendasarinya, termasuk branch sumber dan tujuan, nomor *Pull Request*, tautan *Pull Request*, serta status perubahan.

g. ***Code Review***
Tim Developer yang berperan sebagai reviewer dapat meninjau perubahan kode, informasi *Maintenance Issue* terkait, hasil pengujian, serta perbedaan *source code*. Reviewer dapat memberikan keputusan Approve atau Request Changes. Keputusan tersebut memengaruhi status *Pull Request* dan kelanjutan pengerjaan issue.

h. ***Traceability***
Sistem mempertahankan keterhubungan antara *feedback*, *Maintenance Case*, *Maintenance Backlog*, pengerjaan *Maintenance Issue*, *Pull Request*, dan *Code Review*. Keterhubungan tersebut memungkinkan proses maintenance ditelusuri dari sumber permasalahan hingga tindakan pengembangan yang dilakukan.

MaintainGov versi 1.0 menggunakan file CSV sebagai sumber utama *feedback* dan belum mencakup pengambilan *feedback* secara langsung dari Google Play Store maupun App Store. Sistem juga tidak melakukan pembuatan *source code*, keputusan *Code Review*, *merge*, atau *deployment* secara otomatis. Proses *merge* dan *deployment* dilakukan melalui lingkungan pengembangan eksternal, sedangkan MaintainGov dapat menyimpan atau menyinkronkan status yang diperlukan untuk mempertahankan keterlacakan proses maintenance.

1.5 References

Penyusunan *Software Requirements Specification* MaintainGov mengacu pada standar *requirements engineering*, contoh implementasi SRS, dokumentasi teknis yang relevan, serta

literatur mengenai pemanfaatan *app review* dalam proses *software maintenance*. Referensi yang digunakan adalah sebagai berikut:

Dąbrowski, J., Letier, E., Perini, A., & Susi, A. (2022). Analysing app reviews for software engineering: A systematic literature review. *Empirical Software Engineering, 27*, Article 43. https://doi.org/10.1007/s10664-021-10065-7

GitHub. (n.d.). *About authentication with a GitHub App*. GitHub Docs.

GitHub. (n.d.). *REST API endpoints for pull requests*. GitHub Docs.

Google Play. (n.d.). *Tangerang LIVE* [Mobile application]. Google Play.

International Organization for Standardization. (2018). *ISO/IEC/IEEE 29148:2018 systems and software engineering—Life cycle processes—Requirements engineering*. International Organization for Standardization.

Liu, T., Wang, C., Huang, K., Liang, P., Zhang, B., Daneva, M., & van Sinderen, M. (2023). RoseMatcher: Identifying the impact of user reviews on app updates. *Information and Software Technology, 161*, 107261. https://doi.org/10.1016/j.infsof.2023.107261

Malgaonkar, S., Licorish, S. A., & Savarimuthu, B. T. R. (2022). Prioritizing user concerns in app reviews: A study of requests for new features, enhancements and bug fixes. *Information and Software Technology, 144*, 106798. https://doi.org/10.1016/j.infsof.2021.106798

MEASURE Evaluation. (2019). *Implementing Nigeria's Health Facility Registry: Software Requirements Specifications*. MEASURE Evaluation, University of North Carolina at Chapel Hill. SRS Nigeria Master Facility Lis.

Pemerintah Kota Tangerang. (2025, December 19). *Pengguna aplikasi Tangerang LIVE bertambah 283.767 sepanjang 2025*. Pemerintah Kota Tangerang.

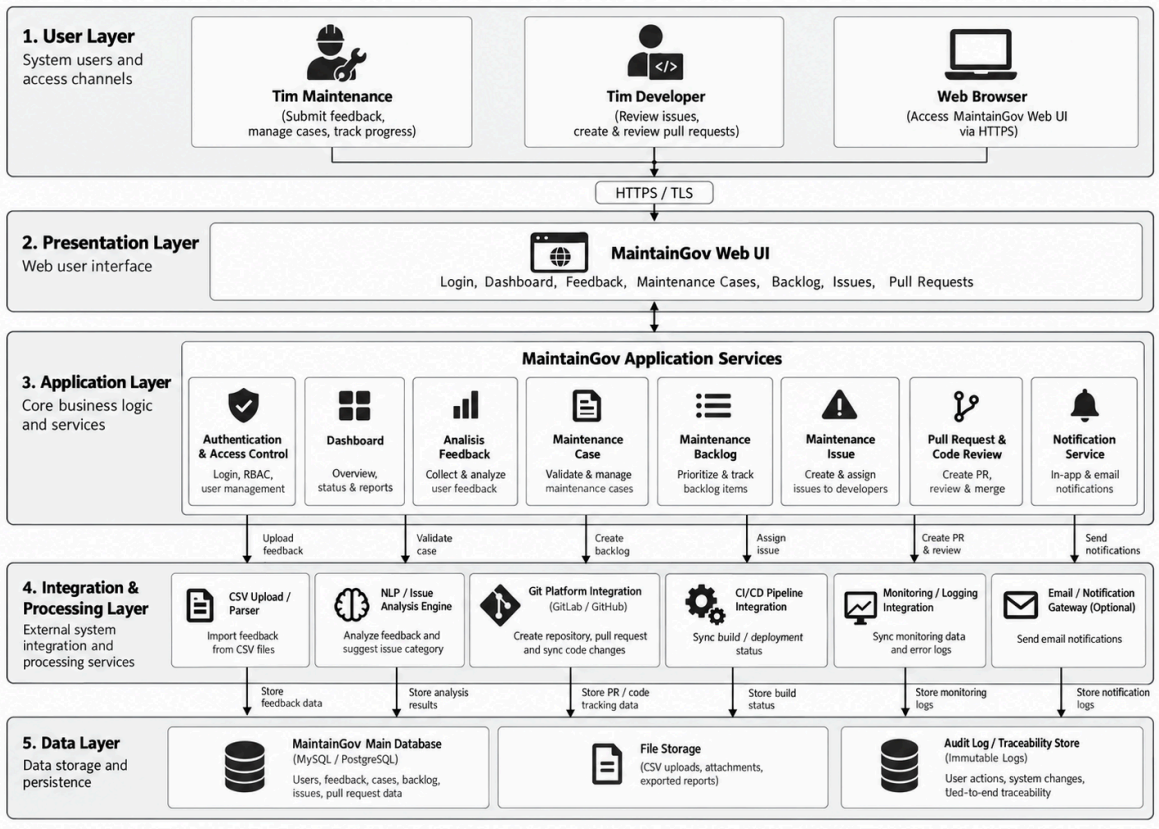
Wang, Y., Wang, J., Zhang, H., Ming, X., & Wang, Q. (2025). Better together: Automated app review analysis with deep multi-task learning. *Information and Software Technology, 177*, 107597. https://doi.org/10.1016/j.infsof.2024.107597.

2. Overall Description

2.1 Product Perspective

MaintainGov merupakan aplikasi internal berbasis *web* yang mendukung proses *software maintenance* berbasis *feedback* pengguna. Sistem mencakup proses analisis *feedback*, pembentukan dan validasi *Maintenance Case*, pengelolaan *Maintenance Backlog*, pengerjaan *Maintenance Issue*, pengajuan *Pull Request*, serta *Code Review*. Dengan cakupan tersebut, MaintainGov menghubungkan informasi dari pengguna aplikasi dengan tindakan maintenance yang dilakukan oleh Tim Maintenance dan Tim Developer.

MaintainGov beroperasi sebagai aplikasi mandiri dengan GitHub sebagai sistem eksternal untuk mendukung repository, branch, *Pull Request*, dan *Code Review*. Pada versi 1.0, *feedback* diterima melalui file CSV dan dianalisis untuk menghasilkan kategori, sentimen, *severity*, issue, kelompok issue serupa, serta prioritas. Hasil analisis kemudian membentuk *Maintenance Case* yang harus divalidasi oleh Tim Maintenance sebelum diteruskan menjadi pekerjaan pengembangan.



Gambar 2.1. *Architecture Diagram* MaintainGov

Arsitektur MaintainGov memisahkan antarmuka pengguna, logika aplikasi, layanan analisis, penyimpanan data, dan integrasi eksternal. Antarmuka menggunakan Next.js dan TypeScript,

backend menggunakan Laravel, basis data menggunakan MySQL, sedangkan layanan analisis menggunakan Python dan FastAPI. Integrasi dengan GitHub dilakukan melalui backend untuk menjaga keamanan kredensial dan konsistensi logika bisnis.

2.2 Product Functions

MaintainGov menyediakan serangkaian fungsi yang mendukung pengelolaan *feedback* pengguna hingga proses maintenance perangkat lunak. Fungsi utama sistem mengikuti enam proses pada Use Case MaintainGov, dengan tambahan autentikasi sebagai fungsi pendukung penggunaan sistem.

a. Authentication and Access Control

- Pengguna dapat melakukan *Login* menggunakan akun yang telah tersedia pada sistem.
- Sistem mengidentifikasi pengguna sebagai Tim Maintenance atau Tim Developer berdasarkan role yang dimiliki.
- Sistem membatasi halaman dan tindakan yang dapat digunakan sesuai dengan role pengguna.
- Pengguna dapat melakukan *Logout* untuk mengakhiri sesi aktif.
- Sistem menjaga agar fungsi yang membutuhkan autentikasi tidak dapat diakses melalui sesi yang tidak valid.

b. Feedback Analysis

- Tim Maintenance dapat mengunggah file CSV yang berisi *feedback* pengguna.
- Sistem memeriksa format file dan struktur data sebelum proses analisis dijalankan.
- *Feedback* yang memenuhi ketentuan disimpan dalam suatu *analysis batch* dan diproses melalui tahapan preprocessing, klasifikasi kategori, analisis sentimen, penentuan *severity*, serta ekstraksi issue.
- Sistem mengelompokkan *feedback* yang merepresentasikan masalah serupa agar tidak membentuk pekerjaan maintenance yang redundan.
- Hasil analisis digunakan untuk membentuk *Maintenance Case* berstatus Pending yang selanjutnya memerlukan validasi Tim Maintenance.

Fungsi ini tidak hanya menghasilkan hasil klasifikasi individual. Sejumlah *feedback* yang memiliki permasalahan serupa dikonsolidasikan menjadi unit masalah yang lebih terstruktur sehingga hasil analisis dapat digunakan pada proses maintenance berikutnya.

c. Maintenance Case Validation

- Tim Maintenance dapat melihat daftar *Maintenance Case* yang masih memerlukan validasi.
- Tim Maintenance dapat membuka detail case untuk memeriksa kategori, modul, fitur terdampak, *severity*, prioritas, ringkasan masalah, rekomendasi, serta *feedback* yang berkaitan.

- Tim Maintenance dapat menentukan case sebagai Valid apabila permasalahan layak diteruskan atau Tidak Valid apabila tidak diperlukan tindakan maintenance.
- Sistem hanya membentuk *Maintenance Backlog* dari *Maintenance Case* yang telah dinyatakan valid.
- Identitas pengguna dan waktu validasi disimpan agar keputusan terhadap case dapat ditelusuri.

Fungsi validasi mempertahankan keterlibatan manusia dalam pengambilan keputusan. Hasil analisis otomatis berperan sebagai dasar penyusunan case dan tidak secara langsung menghasilkan pekerjaan pengembangan.

d. Maintenance Backlog Management

- Sistem membentuk *Maintenance Backlog* dari *Maintenance Case* yang telah dinyatakan valid.
- Backlog yang baru dibentuk memiliki status Ready for Development.
- Tim Maintenance dapat melihat daftar dan detail backlog serta melakukan pencarian atau penyaringan terhadap data yang tersedia.
- Tim Maintenance dapat memperbarui informasi backlog selama item masih berada pada tahap yang memungkinkan perubahan.
- Sistem mempertahankan hubungan antara backlog dan *Maintenance Case* yang menjadi sumbernya.

Maintenance Backlog berfungsi sebagai titik peralihan dari proses analisis dan validasi menuju pekerjaan teknis. Informasi yang telah tervalidasi menjadi acuan bagi Tim Developer tanpa menghilangkan hubungan dengan *feedback* awal.

e. Maintenance Issue Development

- Tim Developer dapat melihat pekerjaan maintenance yang tersedia untuk dikerjakan.
- Developer dapat mengambil item yang masih berstatus Ready for Development.
- Sistem mencatat Developer yang mengambil pekerjaan dan memperbarui status menjadi Sedang Dikerjakan.
- Item yang telah diambil oleh Developer lain tidak dapat diklaim kembali oleh pengguna berbeda.
- Developer melakukan perubahan *source code* dan pengujian melalui lingkungan pengembangan yang digunakan.
- Setelah pekerjaan dan pengujian selesai, status issue dapat dilanjutkan menjadi Ready for Pull Request.

Istilah *Maintenance Issue* digunakan untuk merepresentasikan pekerjaan backlog pada konteks pengerjaan Developer. Fungsi ini tidak membentuk entitas data baru yang terpisah dari *Maintenance Backlog*, tetapi menunjukkan perubahan konteks dan status backlog ketika telah memasuki tahap pengembangan.

f. Pull Request Submission

- Tim Developer dapat mengajukan *Pull Request* untuk *Maintenance Issue* yang telah berstatus Ready for Pull Request.
- Developer dapat menentukan branch tujuan, judul, deskripsi perubahan, serta informasi pendukung yang diperlukan.
- Sistem mengirimkan permintaan pembuatan *Pull Request* melalui integrasi GitHub.
- Metadata seperti nomor, URL, branch sumber, branch tujuan, pengaju, dan status *Pull Request* disimpan pada MaintainGov.
- *Pull Request* yang berhasil dibuat memiliki status awal Open.
- Sistem menampilkan kesalahan apabila informasi yang dibutuhkan belum lengkap atau komunikasi dengan GitHub tidak berhasil.

Dengan fungsi ini, perubahan kode tetap dilakukan pada lingkungan pengembangan dan GitHub, sedangkan MaintainGov mempertahankan keterhubungan antara perubahan tersebut dan pekerjaan maintenance yang mendasarinya.

g. Pull Request Review

- Tim Developer dapat membuka *Pull Request* yang masih memerlukan peninjauan.
- Reviewer dapat memeriksa informasi perubahan, *Maintenance Issue* terkait, branch, hasil pengujian, serta perbedaan *source code*.
- Reviewer dapat memberikan keputusan Approve apabila perubahan telah memenuhi kebutuhan.
- Keputusan Approve mengubah *Pull Request* menjadi Approved dan memungkinkan proses dilanjutkan ke status Ready for Merge.
- Reviewer dapat memilih Request Changes beserta catatan apabila perubahan masih memerlukan perbaikan.
- Keputusan Request Changes mengubah status *Pull Request* menjadi Changes Requested dan mengembalikan pekerjaan terkait ke tahap pengerjaan.

Seluruh fungsi tersebut membentuk proses yang saling terhubung. MaintainGov mempertahankan relasi antara *feedback*, *Maintenance Case*, *Maintenance Backlog*, pekerjaan Developer, *Pull Request*, dan *Code Review* sehingga sumber suatu pekerjaan, keputusan yang diambil, serta perubahan yang dihasilkan dapat ditelusuri dalam satu alur maintenance.

2.3 User Classes and Characteristics

MaintainGov memiliki dua kelas pengguna utama sesuai dengan pembagian tanggung jawab pada proses maintenance. Akses terhadap fitur dibatasi menggunakan *role-based access control* sehingga setiap pengguna hanya dapat menjalankan fungsi yang sesuai dengan tanggung jawabnya.

a. **Tim Maintenance**

Tim Maintenance bertanggung jawab terhadap proses awal pengelolaan *feedback* hingga

pembentukan *Maintenance Backlog*. Pengguna pada kelas ini diasumsikan memahami konteks aplikasi, modul, fitur, serta dampak suatu permasalahan. Tim Maintenance menggunakan fitur *Feedback Analysis*, *Maintenance Case*, dan *Maintenance Backlog*.

b. **Tim Developer**

Tim Developer bertanggung jawab terhadap pelaksanaan pekerjaan teknis setelah kebutuhan maintenance dinyatakan valid. Pengguna pada kelas ini diasumsikan memahami pengembangan perangkat lunak, repository, branch, pengujian, *Pull Request*, dan *Code Review*. Tim Developer menggunakan fitur *Maintenance Issue* dan *Pull Request*.

2.4 Operating Environment

MaintainGov dijalankan sebagai aplikasi berbasis *web* yang dapat diakses melalui *browser* modern tanpa memerlukan instalasi perangkat lunak khusus pada perangkat pengguna. Lingkungan operasional sistem dirancang untuk mendukung pemisahan antara sisi klien, layanan backend, layanan analisis, basis data, dan integrasi eksternal. Rincian lingkungan operasional MaintainGov disajikan pada Tabel 2.1.

Tabel 2.1 *Operating Environment* MaintainGov

Komponen	Teknologi atau Lingkungan	Penggunaan
Perangkat Klien	Laptop atau PC	Mengakses MaintainGov
Sistem Operasi Klien	Windows, macOS, Linux	Menjalankan *web browser*
Web Browser	Chrome, Edge, Firefox, Safari	Menampilkan antarmuka sistem
Frontend	Next.js dan TypeScript	Antarmuka dan interaksi pengguna
Backend	Laravel	Logika bisnis, autentikasi, dan API
Analysis Service	Python dan FastAPI	Pemrosesan dan analisis *feedback*
Database	MySQL	Penyimpanan data sistem
Server	Linux dan Nginx	Menjalankan layanan aplikasi
Komunikasi	HTTPS, REST API, JSON	Pertukaran data antarkomponen
Repository	GitHub	Repository, *Pull Request*, dan *Code Review*

Lingkungan tersebut memungkinkan MaintainGov diakses dari berbagai sistem operasi selama perangkat mendukung *browser* modern dan koneksi jaringan yang memadai. Komponen server dijalankan secara terpisah sesuai tanggung jawab masing-masing agar pengelolaan sistem lebih terstruktur.

2.5 Design and Implementation Constraints

Pengembangan MaintainGov mengikuti beberapa batasan yang menentukan bagaimana fungsi sistem dapat diimplementasikan dan digunakan. Batasan ini terutama berkaitan dengan sumber *feedback*, mekanisme akses pengguna, proses validasi, integrasi GitHub, serta batas otomatisasi sistem.

a. **Feedback Input**
MaintainGov versi 1.0 menerima *feedback* melalui file CSV dan belum melakukan pengambilan data secara langsung dari Google Play Store maupun App Store.

b. **User Account**
Sistem tidak menyediakan registrasi mandiri. Akun Tim Maintenance dan Tim Developer disediakan secara internal sesuai dengan role masing-masing.

c. **Role-Based Access Control**
Akses terhadap fitur dibatasi berdasarkan role pengguna dan divalidasi pada sisi backend.

d. **Human Validation**
Maintenance Case harus divalidasi oleh Tim Maintenance sebelum dapat diteruskan menjadi *Maintenance Backlog*.

e. **Analysis Pipeline**
Sistem harus menghasilkan kategori, sentimen, *severity*, issue, pengelompokan issue, dan prioritas. Metode analisis dapat disesuaikan berdasarkan hasil evaluasi teknis.

f. **GitHub Integration**
Integrasi repository menggunakan GitHub. Credential dan token integrasi dikelola pada sisi server dan tidak diekspos kepada pengguna.

g. **Development Boundary**
MaintainGov tidak menghasilkan *source code*, melakukan *merge*, atau menjalankan *deployment* secara otomatis. Aktivitas tersebut dilakukan melalui lingkungan pengembangan eksternal.

h. **Workflow Integrity**
Perubahan status hanya dapat dilakukan apabila kondisi pada tahap sebelumnya telah

terpenuhi. Hubungan antarobjek maintenance harus tetap dipertahankan untuk mendukung *traceability*.

2.6 User Documentation

Dokumentasi MaintainGov disediakan untuk membantu pengguna memahami pengoperasian sistem sekaligus mendukung kebutuhan teknis selama pengembangan dan pemeliharaan. Dokumentasi dibedakan berdasarkan kebutuhan operasional dan teknis agar informasi yang disampaikan tetap relevan bagi masing-masing pengguna.

a. ***User Guide***
Berisi panduan penggunaan MaintainGov untuk Tim Maintenance dan Tim Developer, termasuk Login, analisis *feedback*, validasi *Maintenance Case*, pengelolaan *Maintenance Backlog*, pengerjaan *Maintenance Issue*, *Pull Request*, dan *Code Review*.

b. ***CSV Input Guide***
Berisi struktur file CSV, kolom yang dibutuhkan, tipe data, aturan validasi, serta contoh data yang dapat diproses oleh sistem.

c. ***System and API Documentation***
Berisi arsitektur aplikasi, konfigurasi komponen, struktur basis data, endpoint API, layanan analisis, integrasi GitHub, serta kebutuhan deployment dan pemeliharaan.

2.7 Assumptions and Dependencies

Pengembangan dan penggunaan MaintainGov didasarkan pada beberapa kondisi yang diasumsikan tersedia serta ketergantungan teknis yang mendukung operasional sistem. Asumsi berkaitan dengan kondisi pengguna dan lingkungan penggunaan, sedangkan *dependencies* berkaitan dengan komponen atau layanan yang diperlukan agar fungsi sistem dapat berjalan.

a. Assumptions

- Tim Maintenance dan Tim Developer telah memiliki akun dengan role yang sesuai sebelum mengakses sistem.
- File CSV yang diunggah memiliki struktur data yang dapat dipetakan ke format input MaintainGov.
- Tim Maintenance memahami konteks aplikasi, modul, dan fitur yang dikelola sehingga dapat memvalidasi *Maintenance Case*.
- Tim Developer memiliki hak akses terhadap repository yang berkaitan dengan pekerjaan maintenance.

- Repository yang digunakan telah dikonfigurasi dan terhubung dengan MaintainGov sebelum proses pengerjaan dimulai.
- Perubahan *source code* dan pengujian dilakukan melalui lingkungan pengembangan eksternal.
- Hasil analisis otomatis tetap dapat ditinjau oleh Tim Maintenance sebelum diteruskan ke proses pengembangan.

b. Dependencies

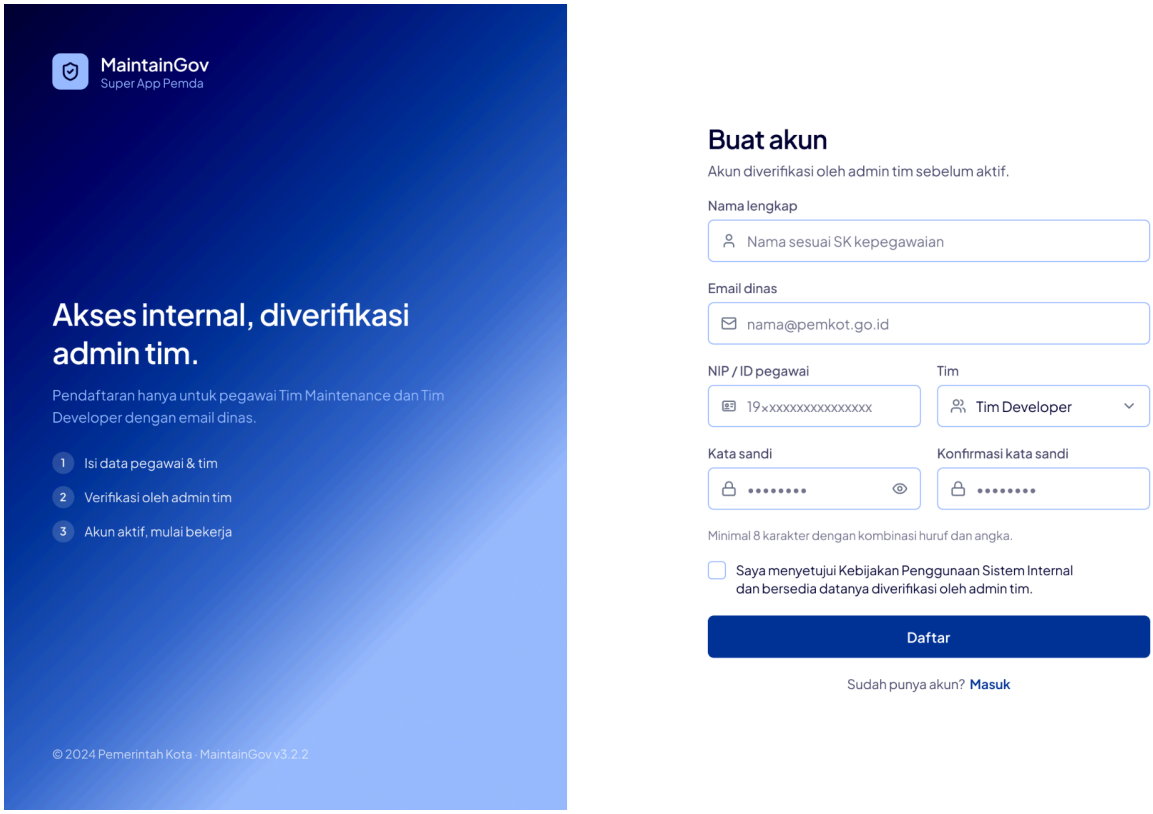
- **GitHub**, digunakan untuk pengelolaan repository, branch, *Pull Request*, dan *Code Review*.
- **Analysis Service**, digunakan untuk menjalankan proses analisis *feedback* dan menghasilkan informasi yang dibutuhkan untuk membentuk *Maintenance Case*.
- **MySQL**, digunakan sebagai penyimpanan utama data pengguna, hasil analisis, case, backlog, *Pull Request*, dan *Code Review*.
- **Network Connection**, digunakan untuk komunikasi antara pengguna, layanan aplikasi, layanan analisis, basis data, dan GitHub.
- **Application Frameworks**, meliputi Next.js, Laravel, Python, dan FastAPI sebagai lingkungan utama implementasi MaintainGov.

3. External Interface Requirements

3.1 User Interfaces

Antarmuka pengguna (*User Interface*) MaintainGov berbasis *web* dan dirancang untuk mendukung proses pemeliharaan perangkat lunak secara terstruktur, mulai dari analisis *feedback*, pembentukan *maintenance case*, pengelolaan *maintenance backlog*, pengerjaan *maintenance issue*, hingga pengajuan dan peninjauan *pull request*. Antarmuka ini menggunakan tata letak yang konsisten pada setiap halaman, yaitu *top bar* untuk identitas sistem dan informasi pengguna, *sidebar* untuk navigasi berdasarkan peran, serta area konten utama untuk menampilkan data, formulir, status, dan aksi lanjutan. Desain dibuat sederhana, informatif, dan berorientasi tugas agar pengguna dapat memahami posisi proses maintenance yang sedang berjalan tanpa harus berpindah konteks secara berlebihan. Seluruh elemen visual, seperti *badge*, warna status, tabel, kartu ringkasan, dan tombol aksi, disusun untuk memperjelas prioritas masalah, tahapan workflow, serta hubungan antarobjek dalam sistem.

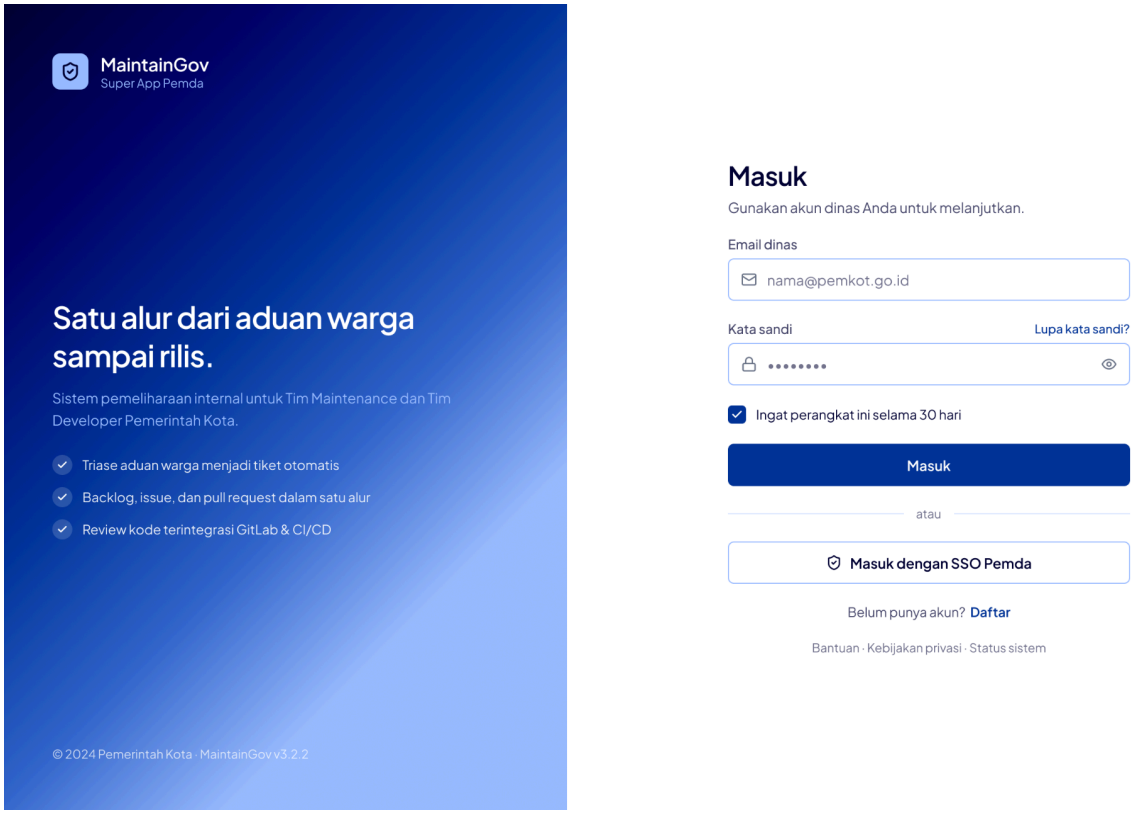
a. Halaman Register



Gambar 3.1 Halaman Register

Halaman Register menyediakan formulir pendaftaran akun internal bagi calon pengguna dari Tim Maintenance dan Tim Developer. Informasi yang harus diisi meliputi nama lengkap, email dinas, NIP atau ID pegawai, pilihan tim, kata sandi, dan konfirmasi kata sandi. Pengguna juga harus menyetujui kebijakan penggunaan sistem sebelum mengirimkan pendaftaran melalui tombol Daftar. Akun yang didaftarkan memerlukan verifikasi oleh admin tim sebelum diaktifkan, sedangkan tautan Masuk tersedia bagi pengguna yang telah memiliki akun.

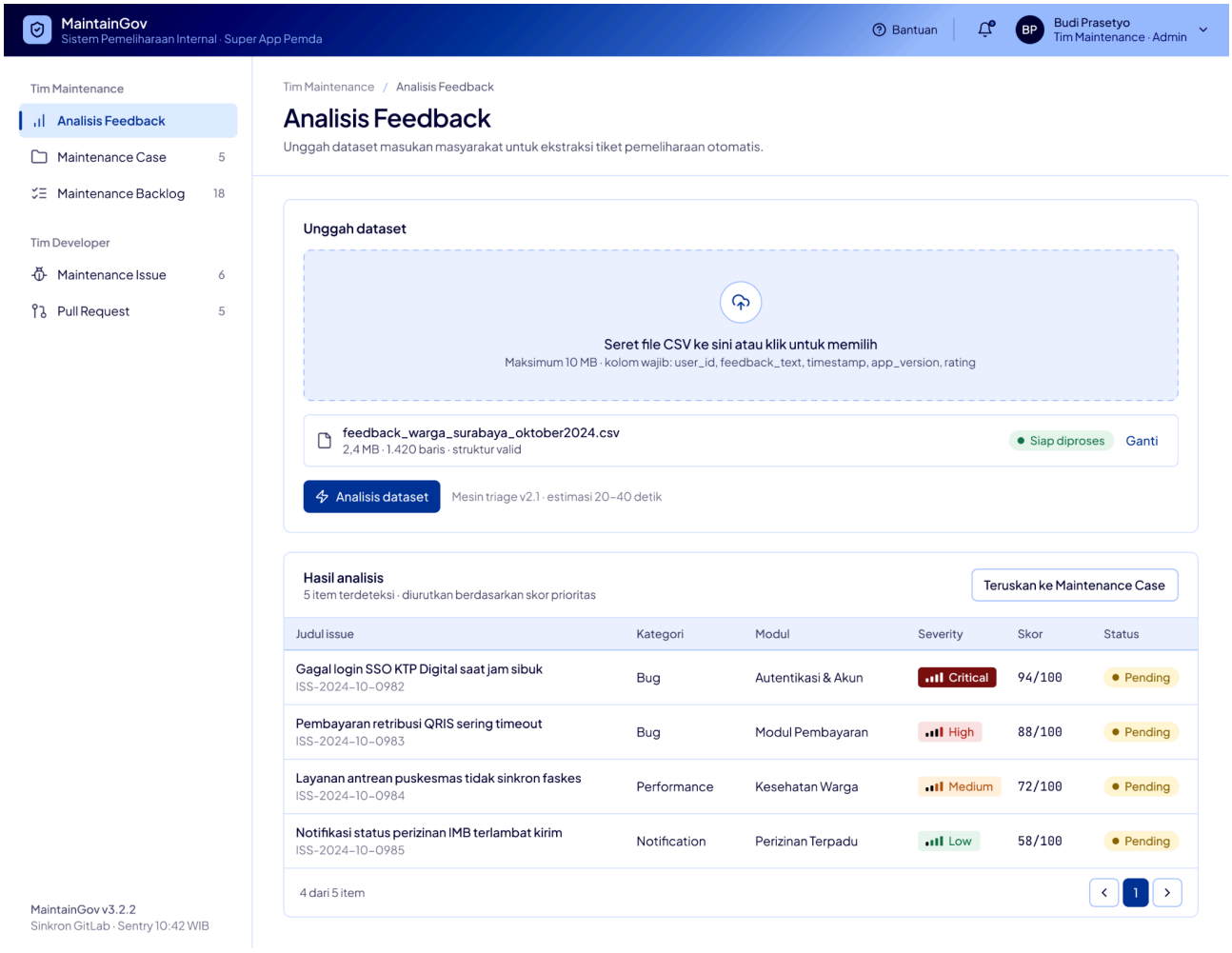
b. Halaman Login



Gambar 3.2 Halaman Login

Halaman Login digunakan untuk melakukan autentikasi sebelum pengguna memperoleh akses terhadap fitur MaintainGov. Antarmuka menampilkan kolom email dinas dan kata sandi, opsi Ingat perangkat ini selama 30 hari, serta tombol Masuk untuk mengirimkan kredensial. Sistem juga menyediakan alternatif autentikasi melalui SSO Pemda, tautan pemulihan kata sandi, dan navigasi menuju halaman Register. Setelah autentikasi berhasil, pengguna dapat mengakses fungsi sesuai perannya, sedangkan kredensial yang tidak valid menghasilkan pesan kesalahan tanpa memberikan akses ke sistem.

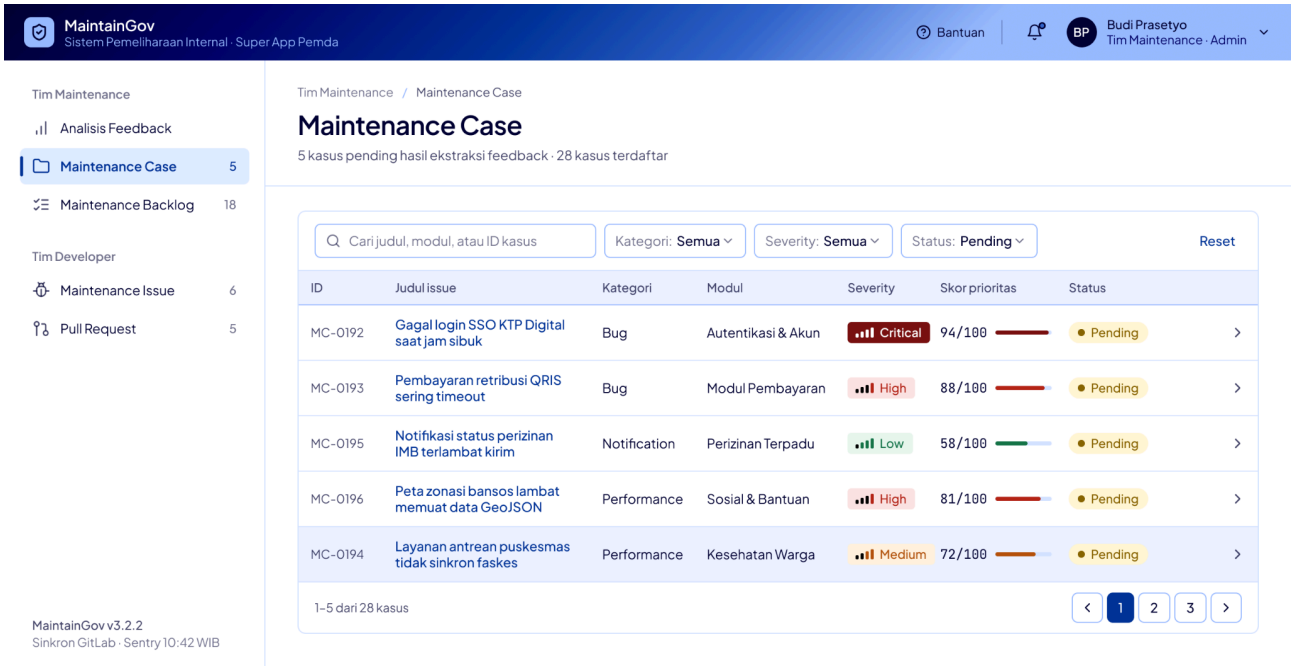
c. Halaman Analisis Feedback



Gambar 3.3 Halaman Analisis Feedback

Halaman Analisis Feedback digunakan Tim Maintenance untuk mengunggah file CSV sebagai sumber data dalam proses analisis ulasan pengguna. Antarmuka menyediakan area unggah dengan batas ukuran 10 MB dan ketentuan kolom user_id, feedback_text, timestamp, app_version, serta rating, disertai informasi nama file, jumlah baris, dan hasil validasi struktur data. Setelah pengguna menekan tombol Analisis dataset, sistem memproses feedback dan menampilkan hasil berupa judul issue, kategori, modul, severity, skor prioritas, serta status dalam tabel. Tombol Teruskan ke Maintenance Case memungkinkan hasil analisis dilanjutkan ke tahap pemeriksaan dan validasi oleh Tim Maintenance.

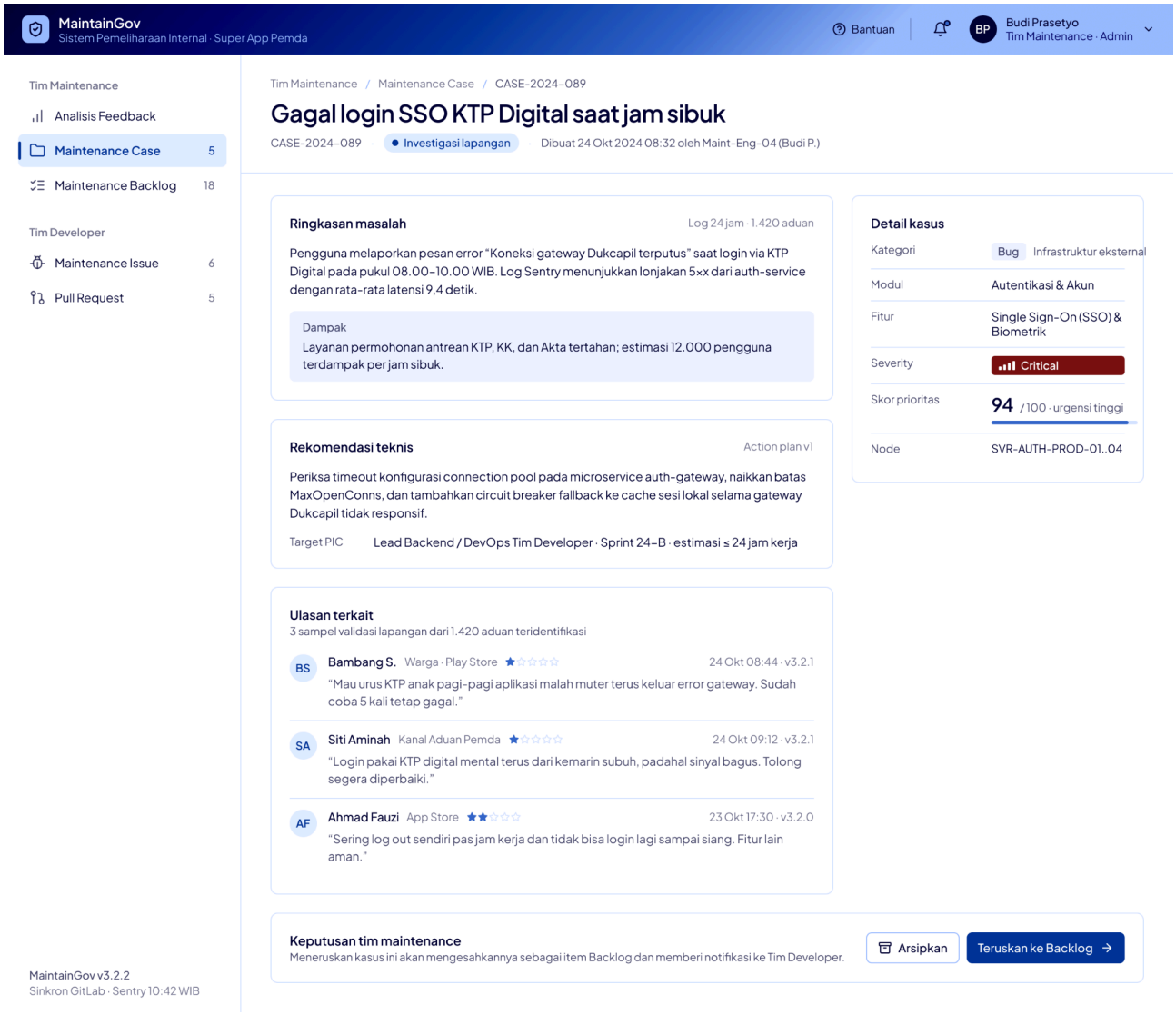
d. Halaman Maintenance Case



Gambar 3.4 Halaman Maintenance Case

Halaman Maintenance Case menampilkan daftar kasus yang terbentuk dari hasil analisis feedback dan memerlukan validasi lebih lanjut. Tabel utama menyajikan ID kasus, judul issue, kategori, modul, severity, skor prioritas, dan status sebagai dasar identifikasi permasalahan. Fasilitas pencarian, penyaringan berdasarkan kategori, severity, dan status, serta pagination membantu Tim Maintenance mengelola kasus dalam jumlah besar. Pengguna dapat memilih salah satu kasus untuk membuka halaman detail dan memeriksa informasi yang diperlukan sebelum menentukan tindak lanjut maintenance.

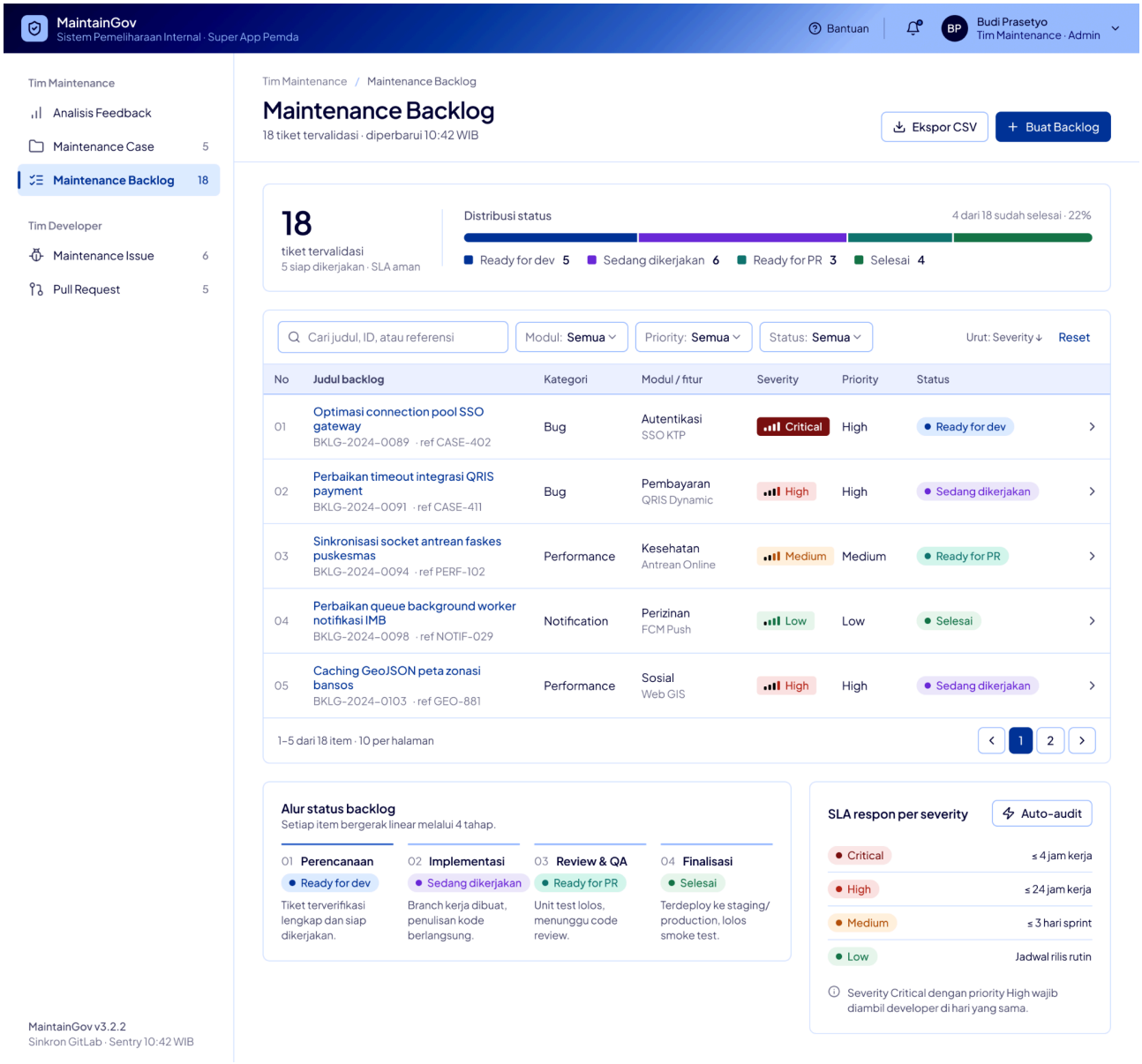
e. Halaman Detail Maintenance Case



Gambar 3.5 Halaman Detail Maintenance Case

Halaman Detail Maintenance Case menampilkan informasi terperinci mengenai permasalahan yang teridentifikasi dari sejumlah ulasan pengguna. Informasi utama mencakup ringkasan masalah, dampak terhadap layanan, rekomendasi teknis, kategori, modul, fitur terdampak, severity, serta skor prioritas. Sistem juga menampilkan ulasan terkait agar Tim Maintenance dapat memeriksa kesesuaian hasil analisis dengan permasalahan yang dilaporkan pengguna. Pada bagian keputusan tersedia tombol Arsipkan dan Teruskan ke Backlog untuk menentukan apakah kasus dihentikan atau diteruskan menjadi pekerjaan maintenance.

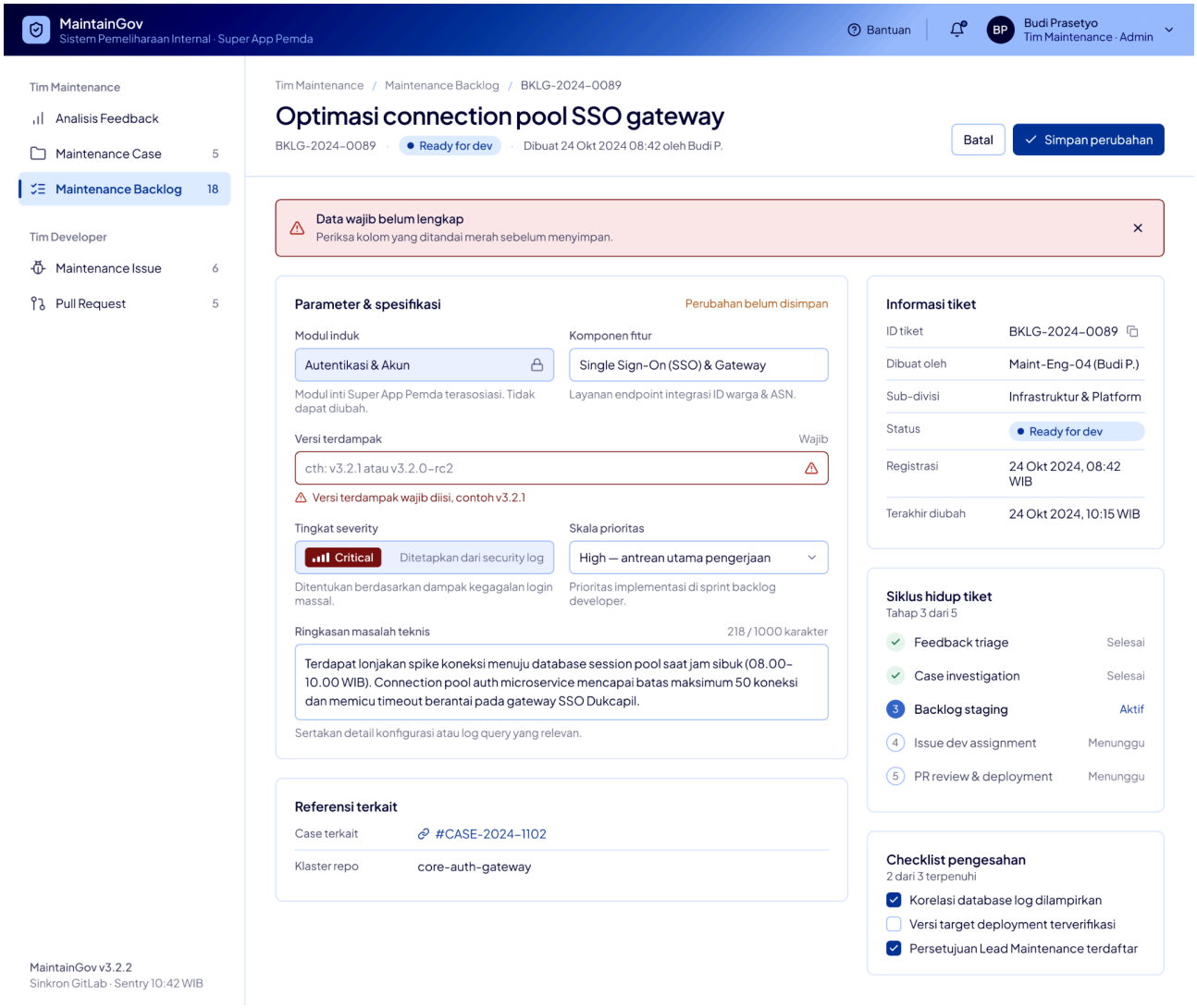
f. Halaman Maintenance Backlog



Gambar 3.6 Halaman Maintenance Backlog

Halaman Maintenance Backlog digunakan Tim Maintenance untuk memantau pekerjaan yang telah terbentuk dari proses validasi Maintenance Case. Bagian ringkasan menyajikan jumlah tiket tervalidasi, distribusi status pekerjaan, dan persentase penyelesaian, sedangkan tabel utama menampilkan judul backlog, kategori, modul, severity, prioritas, dan status. Pengguna dapat melakukan pencarian, penyaringan, pengurutan, dan ekspor data untuk mempermudah pemeriksaan backlog berdasarkan kebutuhan operasional. Informasi alur status dan target respons berdasarkan severity turut disediakan untuk menunjukkan posisi serta urgensi setiap pekerjaan dalam proses maintenance.

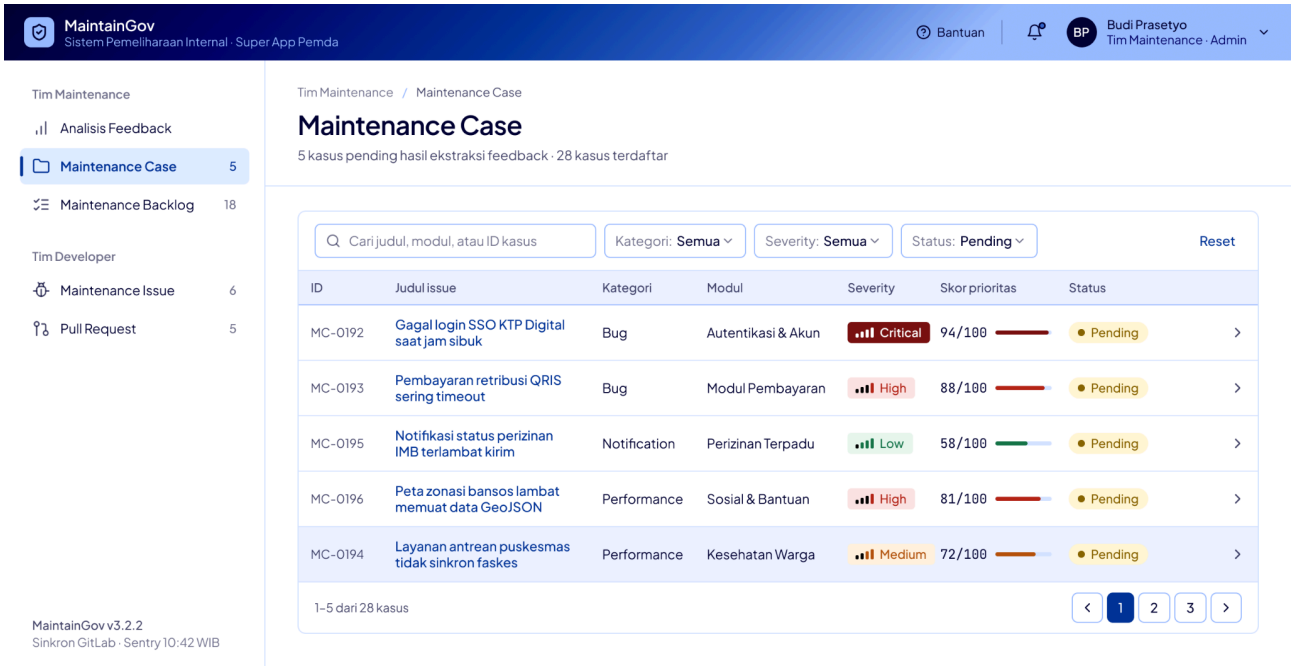
g. Halaman Detail Maintenance Backlog



Gambar 3.7 Halaman Detail Maintenance Backlog

Halaman Detail Maintenance Backlog menyediakan formulir pemeriksaan dan perubahan informasi pekerjaan sebelum memasuki tahap pengerjaan oleh Tim Developer. Parameter yang ditampilkan meliputi modul induk, komponen fitur, versi terdampak, severity, skala prioritas, dan ringkasan masalah teknis, serta dilengkapi referensi terhadap Maintenance Case terkait. Panel informasi tiket, siklus hidup pekerjaan, dan checklist pengesahan membantu Tim Maintenance memeriksa kelengkapan informasi backlog. Apabila terdapat data wajib yang belum lengkap, sistem menampilkan pesan validasi pada kolom terkait dan mempertahankan formulir sampai pengguna memperbaikinya sebelum menekan Simpan perubahan.

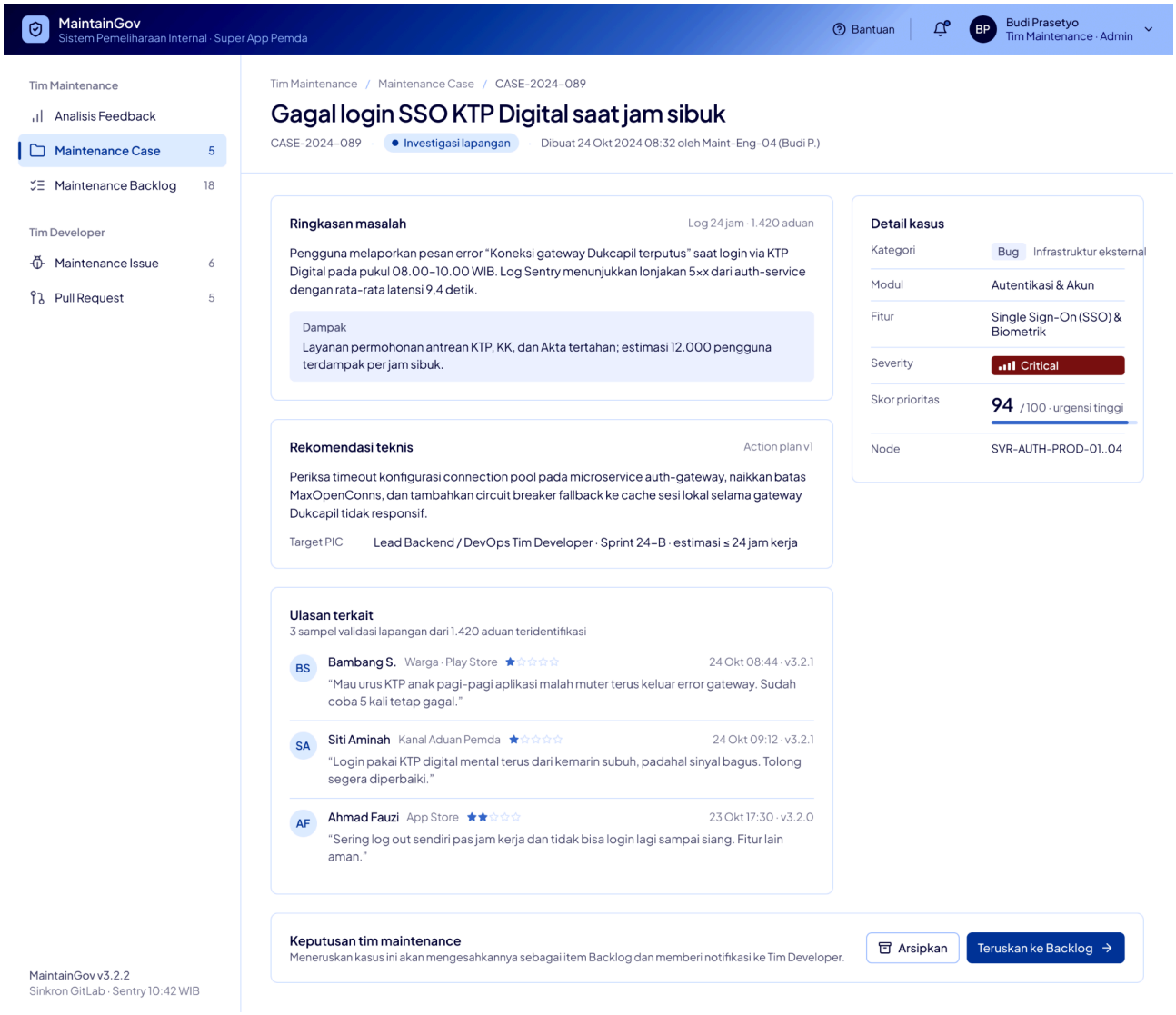
h. Halaman Maintenance Issue



Gambar 3.8 Halaman Maintenance Issue

Halaman Maintenance Issue menampilkan daftar pekerjaan pengembangan yang berasal dari Maintenance Backlog dan dapat ditindaklanjuti oleh Tim Developer. Informasi setiap pekerjaan mencakup ID issue, judul permasalahan, modul, severity, prioritas, pengguna yang ditugaskan, dan status pengerjaan. Fasilitas pencarian dan penyaringan memungkinkan Developer menemukan pekerjaan berstatus Ready for Development, Sedang Dikerjakan, maupun Ready for Pull Request sesuai kebutuhan. Pengguna dapat membuka detail issue untuk memeriksa spesifikasi pekerjaan sebelum mengambil atau melanjutkan aktivitas pengembangan.

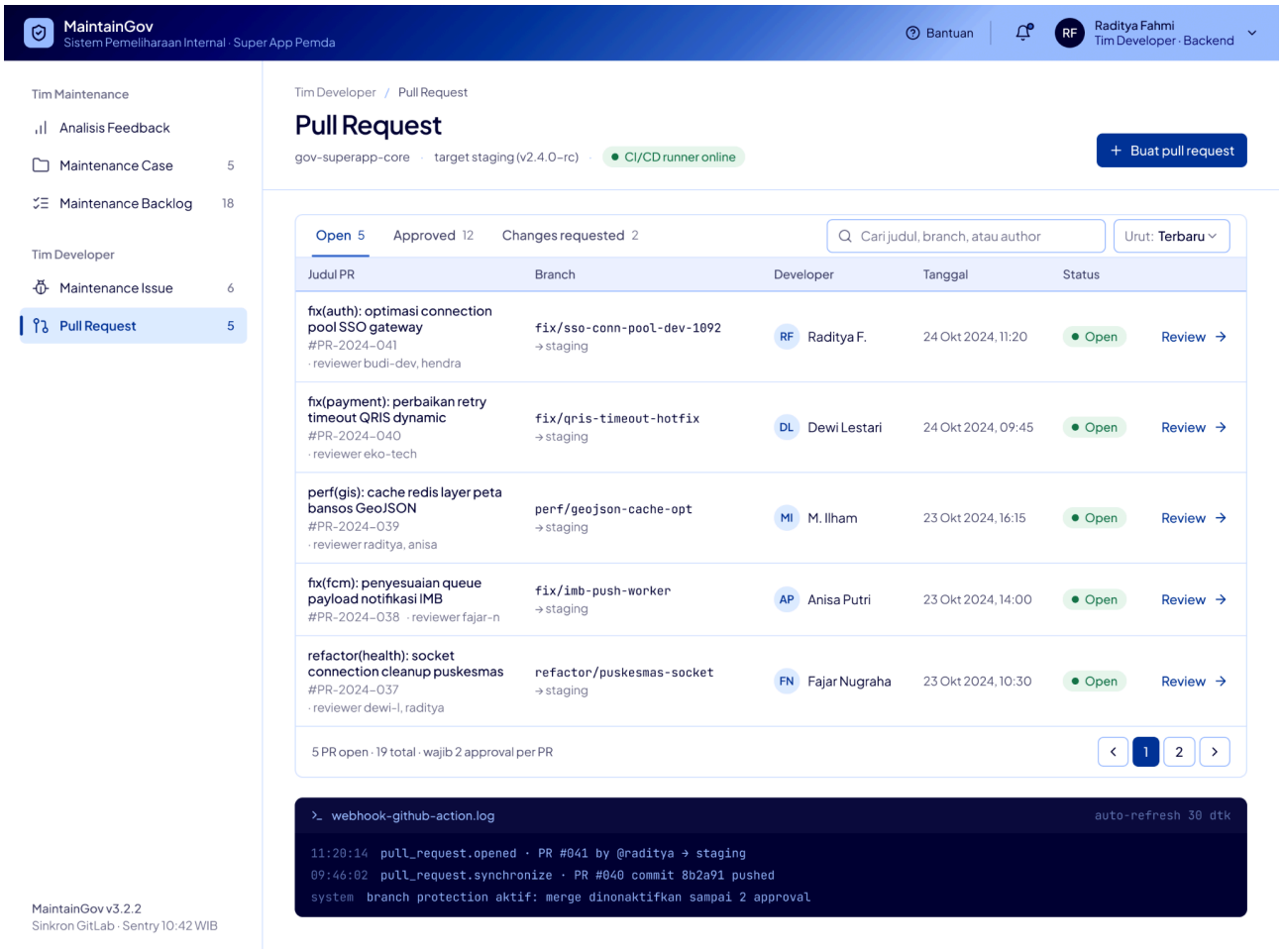
i. Halaman Detail Maintenance Issue



Gambar 3.9 Halaman Detail Maintenance Issue

Halaman Detail Maintenance Issue menyajikan spesifikasi teknis pekerjaan yang akan atau sedang ditangani oleh Tim Developer. Informasi yang tersedia meliputi modul, fitur, severity, prioritas, ringkasan masalah, penanggung jawab, estimasi pengerjaan, dan target rilis, disertai panel pipeline log yang menunjukkan aktivitas pengembangan serta pengujian. Tombol Kerjakan issue digunakan untuk mengambil pekerjaan yang masih tersedia dan menetapkan Developer sebagai penanggung jawab, sedangkan Ajukan pull request digunakan setelah issue memenuhi persyaratan pengajuan perubahan. Apabila issue belum siap diajukan atau telah diambil Developer lain, sistem membatasi tindakan yang tidak sesuai dengan status pekerjaan.

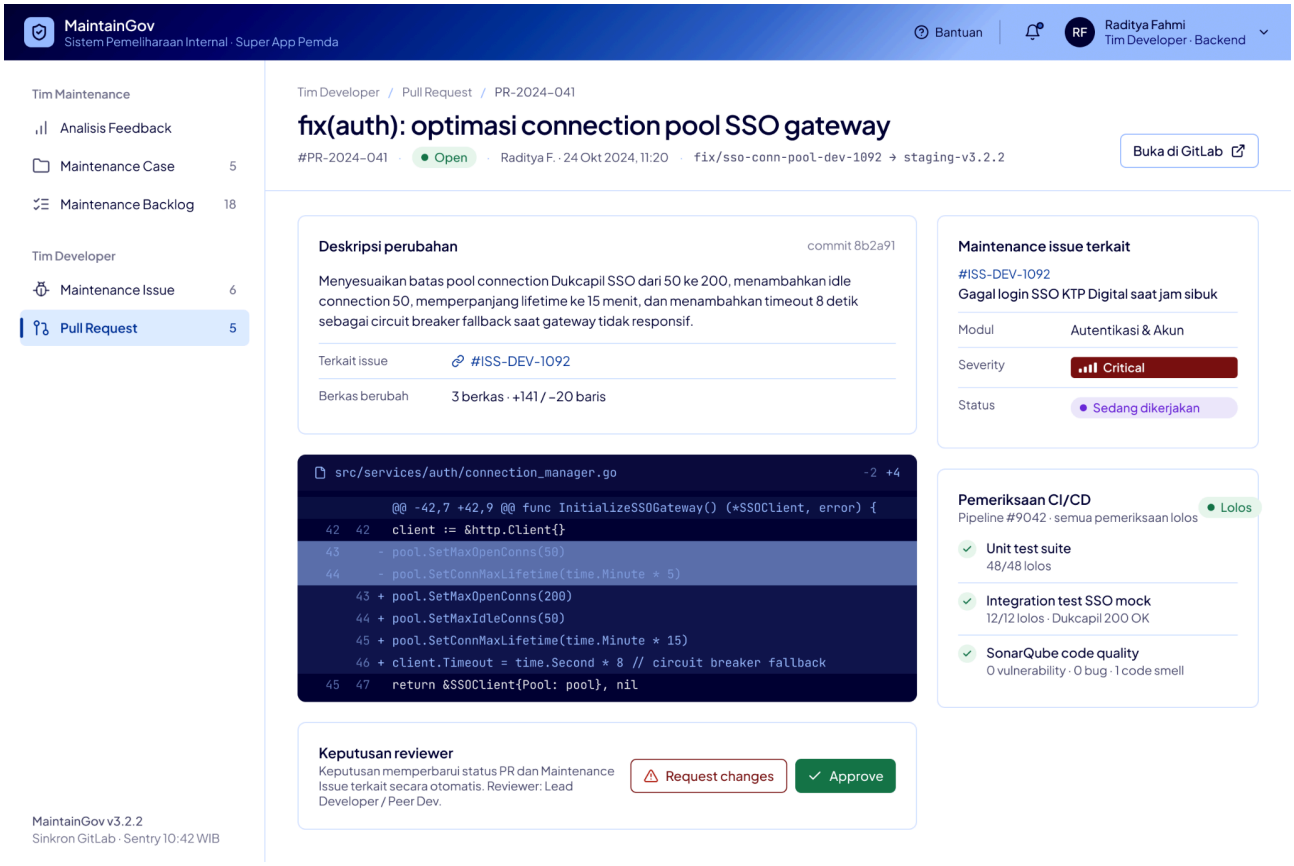
j. Halaman Pull Request



Gambar 3.10 Halaman Pull Request

Halaman Pull Request menampilkan daftar perubahan kode yang diajukan melalui repository dan berada pada tahap peninjauan. Antarmuka menyediakan pengelompokan berdasarkan status Open, Approved, dan Changes Requested, serta fasilitas pencarian dan pengurutan untuk menemukan perubahan tertentu. Setiap entri menampilkan judul Pull Request, branch, Developer, tanggal pengajuan, status, dan tombol Review yang mengarahkan pengguna menuju rincian perubahan. Panel aktivitas integrasi repository pada bagian bawah digunakan untuk menampilkan informasi sinkronisasi dan pembaruan teknis yang berkaitan dengan proses review.

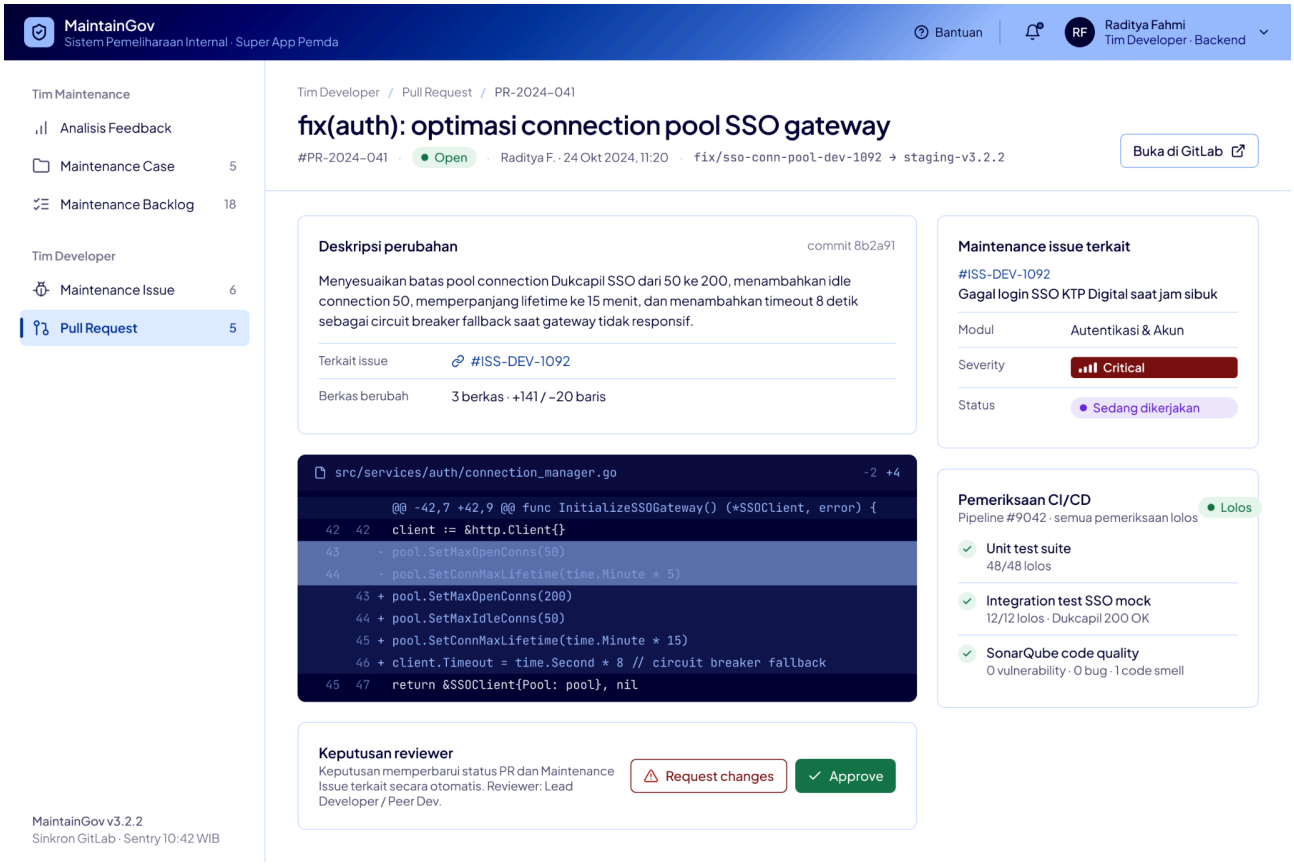
k. Halaman Detail Pull Request



Gambar 3.11 Halaman Detail Pull Request

Halaman Detail Pull Request digunakan Tim Developer untuk mengevaluasi perubahan source code sebelum keputusan review diberikan. Antarmuka menampilkan identitas dan deskripsi Pull Request, branch sumber dan tujuan, Maintenance Issue terkait, daftar berkas yang berubah, serta perbandingan kode melalui panel code diff. Informasi hasil pemeriksaan CI/CD, termasuk unit test, integration test, dan pemeriksaan kualitas kode, disediakan sebagai bahan pertimbangan terhadap kelayakan perubahan. Reviewer dapat memilih Approve apabila perubahan memenuhi kebutuhan atau Request Changes apabila diperlukan perbaikan sebelum proses dilanjutkan.

1. Halaman Buat Pull Request



Gambar 3.12 Halaman Buat Pull Request

Halaman Buat Pull Request menyediakan formulir pengajuan perubahan kode dari Maintenance Issue yang telah mencapai status Ready for Pull Request. Antarmuka menampilkan repository, branch sumber, branch tujuan, jumlah commit, dan daftar berkas yang mengalami perubahan sebelum Developer mengisi judul serta deskripsi Pull Request. Bagian pemeriksaan otomatis menampilkan informasi mengenai linter, kualitas kode, cakupan pengujian, dan pemeriksaan dependensi untuk mendukung penilaian kesiapan pengajuan. Sistem memvalidasi kelengkapan informasi wajib dan menampilkan pesan kesalahan apabila terdapat input yang belum sesuai, sedangkan tombol Buat pull request digunakan untuk mengirimkan perubahan setelah persyaratan terpenuhi.

3.2 Hardware Interfaces

MaintainGov tidak memerlukan perangkat keras khusus karena seluruh fungsi utama tersedia melalui aplikasi berbasis *web*. Interaksi perangkat keras pada sisi pengguna dilakukan melalui perangkat komputasi standar, sedangkan pemrosesan aplikasi berjalan pada infrastruktur server yang mendukung komponen backend, basis data, dan layanan analisis. Antarmuka perangkat

keras tersebut digunakan untuk memastikan sistem dapat dioperasikan tanpa ketergantungan pada perangkat periferal tertentu.

3.2.1 Supported Client Devices

MaintainGov dirancang untuk digunakan terutama melalui laptop atau PC yang memiliki *web browser* modern dan koneksi jaringan. Perangkat pengguna harus menyediakan antarmuka input standar seperti keyboard, mouse, atau *touchpad* untuk mendukung pengisian data, pemilihan file CSV, navigasi halaman, dan tindakan maintenance. Sistem tidak membutuhkan kamera, mikrofon, sensor, pemindai, maupun perangkat eksternal lain untuk menjalankan fungsi utamanya. Penggunaan perangkat seluler tidak menjadi target utama karena beberapa aktivitas, seperti pemeriksaan backlog, perubahan teknis, dan *Code Review*, membutuhkan tampilan informasi yang relatif kompleks.

Tabel 3.1 Supported Hardware

Perangkat	Kebutuhan Minimum	Penggunaan
Laptop atau PC	RAM 4 GB, prosesor dual-core, *browser* modern	Mengakses seluruh fungsi MaintainGov
Server Aplikasi	RAM 8 GB, prosesor minimal 2 core	Menjalankan *frontend*, backend, dan layanan pendukung
Server Analisis dan Database	RAM 8 GB atau lebih, penyimpanan yang memadai	Menjalankan *Analysis Service* dan MySQL

Spesifikasi server dapat ditingkatkan menyesuaikan jumlah pengguna dan volume *feedback* yang diproses. Beban komputasi terbesar diperkirakan berasal dari proses analisis ketika satu *analysis batch* berisi banyak ulasan. Pemisahan *Analysis Service* dari backend utama memungkinkan kapasitas komputasi ditingkatkan tanpa mengubah antarmuka pengguna. Kapasitas penyimpanan juga perlu mempertimbangkan pertumbuhan data ulasan, *Maintenance Case*, *Maintenance Backlog*, dan riwayat *Pull Request*.

3.2.2 Hardware Interaction Characteristics

Perangkat klien berinteraksi dengan MaintainGov melalui *web browser* dan tidak memiliki akses langsung terhadap database maupun layanan internal lainnya. Ketika Tim Maintenance memilih file CSV, *browser* menyediakan mekanisme pemilihan file dan mengirimkan file tersebut kepada backend melalui koneksi jaringan. Backend kemudian menangani validasi dan pemrosesan data tanpa memperoleh akses ke file lain pada perangkat pengguna. Pembatasan tersebut menjaga agar MaintainGov hanya memproses data yang secara eksplisit diberikan oleh pengguna.

Pada sisi server, komponen aplikasi menggunakan sumber daya CPU, memori, penyimpanan, dan antarmuka jaringan yang disediakan oleh sistem operasi. Backend, *Analysis Service*, dan MySQL berkomunikasi melalui lingkungan server yang terkontrol dan tidak diekspos langsung ke perangkat pengguna. Penyimpanan persisten digunakan untuk menjaga data pengguna, hasil analisis, case, backlog, dan aktivitas review. Kegagalan perangkat keras server tidak boleh menyebabkan sistem menyimpan status proses sebagai berhasil apabila transaksi belum selesai secara konsisten.

3.3 Software Interfaces

MaintainGov berinteraksi dengan beberapa komponen perangkat lunak internal dan eksternal yang mendukung keseluruhan proses maintenance. Komponen internal mencakup *frontend*, backend, *Analysis Service*, dan database, sedangkan GitHub menjadi layanan eksternal utama untuk mendukung repository, *Pull Request*, dan *Code Review*. Backend Laravel menjadi penghubung utama antar komponen sehingga logika bisnis, autentikasi, otorisasi, dan validasi tidak tersebar pada sisi klien. Pemisahan tanggung jawab tersebut membantu menjaga konsistensi proses dan mempermudah pengembangan setiap komponen secara independen.

3.3.1 Frontend Application

Frontend MaintainGov dikembangkan menggunakan Next.js dan TypeScript serta berfungsi sebagai antarmuka utama bagi Tim Maintenance dan Tim Developer. Komponen ini menangani tampilan halaman, navigasi, validasi awal input, pengelolaan *state*, serta penyajian data yang diperoleh dari backend. *Frontend* tidak mengakses database, *Analysis Service*, atau GitHub secara langsung karena seluruh operasi yang memengaruhi data dan workflow harus melalui backend. Pola tersebut memastikan hak akses dan aturan bisnis tetap dikendalikan secara terpusat.

3.3.2 Backend Application

Backend MaintainGov menggunakan Laravel dan menangani autentikasi, otorisasi, logika bisnis, validasi data, perubahan status, akses database, serta koordinasi dengan layanan lain. Backend memproses permintaan dari *frontend* dan menentukan apakah pengguna memiliki hak untuk menjalankan fungsi tertentu berdasarkan role yang dimiliki. Komponen ini juga menjaga konsistensi status antara *Maintenance Case*, *Maintenance Backlog*, *Maintenance Issue*, dan *Pull Request*. Semua operasi yang mengubah data utama sistem harus melewati backend agar tidak terjadi perubahan langsung yang mengabaikan aturan workflow.

3.3.3 Analysis Service

Analysis Service menggunakan Python dan FastAPI untuk menangani proses analisis *feedback* yang memiliki karakteristik pemrosesan berbeda dari logika bisnis aplikasi. Layanan ini

menerima data ulasan dari backend dan menjalankan preprocessing, klasifikasi kategori, analisis sentimen, penentuan *severity*, ekstraksi issue, pengelompokan issue serupa, serta perhitungan prioritas. Hasil analisis dikembalikan dalam bentuk data terstruktur untuk digunakan backend dalam membentuk *Maintenance Case*. Metode atau model yang digunakan dapat dikembangkan secara terpisah selama struktur input dan output tetap sesuai dengan kebutuhan MaintainGov.

3.3.4 Database Interface

MySQL digunakan sebagai database relasional utama untuk menyimpan data MaintainGov secara persisten. Data yang dikelola meliputi pengguna sistem, *analysis batch*, ulasan, *Maintenance Case*, *Maintenance Backlog*, repository GitHub, *Pull Request*, dan *Code Review*. Backend berinteraksi dengan MySQL melalui Laravel ORM untuk menjalankan operasi penyimpanan, pembacaan, pembaruan, dan relasi antardata. Penggunaan relasi antartabel mempertahankan keterlacakan antara *feedback* awal, keputusan validasi, pekerjaan maintenance, dan perubahan kode.

Operasi yang melibatkan beberapa data terkait perlu diproses secara konsisten agar tidak menghasilkan kondisi parsial. Sebagai contoh, *Maintenance Case* yang telah dinyatakan valid tidak boleh tercatat sebagai berhasil diteruskan apabila pembentukan *Maintenance Backlog* gagal. Perubahan status dan data terkait perlu dijalankan dalam transaksi apabila satu proses mencakup beberapa operasi database. Mekanisme tersebut digunakan untuk menjaga integritas workflow dan konsistensi data.

3.3.5 GitHub Interface

GitHub digunakan sebagai layanan eksternal yang mendukung repository, branch, *Pull Request*, dan proses review perubahan kode. MaintainGov berinteraksi dengan GitHub melalui GitHub REST API dan autentikasi GitHub App yang dikelola pada sisi backend. Integrasi ini digunakan untuk memperoleh informasi repository, membuat *Pull Request*, menyimpan nomor dan URL *Pull Request*, memperoleh status review, serta menyinkronkan perubahan status yang relevan. Credential integrasi tidak disimpan atau dikirimkan kepada *browser* pengguna.

MaintainGov tidak menggantikan GitHub sebagai platform pengelolaan *source code*. Perubahan kode tetap dilakukan melalui lingkungan pengembangan dan repository, sedangkan MaintainGov menyimpan metadata yang menghubungkan aktivitas tersebut dengan *Maintenance Backlog*. Satu repository dapat memiliki beberapa backlog dan satu backlog dapat berkaitan dengan lebih dari satu *Pull Request* apabila diperlukan revisi tambahan. Struktur tersebut memungkinkan riwayat perubahan tetap ditelusuri tanpa menduplikasi fungsi pengelolaan kode yang telah disediakan GitHub.

3.3.6 CSV Data Interface

File CSV menjadi antarmuka input utama untuk memasukkan *feedback* pada MaintainGov versi 1.0. Tim Maintenance memilih file melalui *browser*, kemudian backend memeriksa apakah file dapat dibaca dan memiliki struktur yang sesuai sebelum analisis dimulai. Pemeriksaan mencakup format file, keberadaan kolom wajib, kelengkapan struktur data, serta validitas data yang dibutuhkan oleh pipeline analisis. File yang gagal memenuhi ketentuan tidak diteruskan ke *Analysis Service* dan pengguna menerima informasi mengenai penyebab kegagalan.

3.4 Communications Interfaces

MaintainGov menetapkan mekanisme komunikasi antara pengguna, komponen internal, dan layanan eksternal. Komunikasi mencakup pola *request-response*, format pertukaran data, unggah file, autentikasi sesi, serta sinkronisasi informasi dengan GitHub. Setiap komunikasi yang melewati jaringan harus menggunakan mekanisme yang menjaga kerahasiaan dan integritas data selama transmisi. Struktur komunikasi juga harus menghasilkan respons yang konsisten agar kegagalan pada satu komponen tidak menyebabkan status workflow yang tidak sesuai.

3.4.1 Client–Server Communication

Komunikasi antara *frontend* dan backend menggunakan REST API melalui HTTP atau HTTPS dengan pola dua arah berbasis *request-response*. Pada lingkungan produksi, seluruh komunikasi antara *browser* dan server wajib menggunakan HTTPS untuk melindungi informasi pengguna dan data maintenance selama transmisi. Data terstruktur seperti informasi case, backlog, issue, dan *Pull Request* menggunakan JSON sebagai format utama. Pengiriman file CSV menggunakan multipart/form-data karena permintaan mengandung file yang perlu diterima dan divalidasi oleh backend.

Metode GET digunakan untuk membaca sumber daya tanpa mengubah kondisi sistem, sedangkan POST digunakan untuk pembuatan data atau pemicu proses baru. PATCH digunakan untuk pembaruan sebagian informasi atau perubahan status yang telah diizinkan oleh aturan bisnis. Penggunaan DELETE hanya diterapkan apabila fungsi penghapusan secara eksplisit didefinisikan dalam requirement. Pemilihan metode HTTP dilakukan secara konsisten agar operasi API mudah dipahami dan diuji.

3.4.2 Internal Service Communication

Backend Laravel berkomunikasi dengan *Analysis Service* FastAPI menggunakan REST API dengan format JSON. Data yang dikirim mencakup ulasan atau representasi data yang dibutuhkan untuk proses analisis, sedangkan hasil respons berisi kategori, sentimen, *severity*, hasil ekstraksi issue, kelompok issue, dan informasi prioritas. Backend tetap bertanggung jawab terhadap pembentukan objek bisnis seperti *Maintenance Case* setelah hasil analisis berhasil diterima.

Analysis Service tidak memiliki kewenangan langsung untuk mengubah status workflow utama tanpa melalui backend.

Proses analisis dapat membutuhkan waktu yang berbeda bergantung pada jumlah ulasan dalam satu *analysis batch*. Backend mencatat status proses sehingga pengguna dapat mengetahui apakah batch sedang diproses, telah selesai, atau mengalami kegagalan. Permintaan analisis yang belum selesai tidak boleh ditampilkan sebagai hasil final. Pola ini mengurangi risiko status yang tidak akurat ketika layanan analisis membutuhkan waktu pemrosesan yang lebih panjang.

3.4.3 GitHub Communication

Komunikasi antara backend dan GitHub dilakukan melalui GitHub REST API menggunakan HTTPS. Backend menggunakan GitHub App untuk memperoleh hak akses yang terbatas pada repository dan operasi yang memang diperlukan MaintainGov. Permintaan dapat mencakup pembacaan informasi repository, pembuatan *Pull Request*, pengambilan detail *Pull Request*, serta pengelolaan informasi review. Status yang diperoleh dari GitHub kemudian dipetakan ke informasi maintenance yang tersimpan pada MaintainGov.

GitHub juga dapat mengirimkan perubahan status kepada MaintainGov melalui webhook. Backend harus memverifikasi signature webhook sebelum event diperbolehkan mengubah data lokal agar permintaan palsu tidak dianggap berasal dari GitHub. Event yang diterima perlu diperiksa terhadap repository dan *Pull Request* yang telah terdaftar pada MaintainGov sebelum sinkronisasi dilakukan. Kegagalan sinkronisasi tidak boleh menghapus data lokal yang telah valid atau mengubah status proses menjadi berhasil.

3.4.4 Authentication and Session Communication

MaintainGov menggunakan autentikasi berbasis sesi untuk melindungi fitur yang hanya dapat digunakan oleh pengguna terdaftar. Setelah kredensial berhasil diverifikasi, backend membentuk sesi dan mengirimkan identitas sesi menggunakan *secure HTTP-only cookie*. Atribut Secure memastikan *cookie* hanya dikirim melalui HTTPS, sedangkan HttpOnly membatasi akses langsung melalui JavaScript pada *browser*. Setiap permintaan yang mengakses fungsi terproteksi tetap harus melalui pemeriksaan autentikasi dan role pada backend.

Proses Logout mengakhiri sesi aktif sehingga identitas sesi lama tidak lagi dapat digunakan untuk mengakses fungsi terproteksi. Permintaan yang mengubah data juga perlu dilindungi terhadap *Cross-Site Request Forgery* dan hanya diterima dari origin MaintainGov yang telah dikonfigurasi. Penyembunyian menu pada *frontend* tidak dianggap cukup untuk membatasi akses karena pengguna tetap dapat mencoba mengirim permintaan langsung ke endpoint. Otorisasi pada backend menjadi sumber keputusan akhir terhadap hak akses.

3.4.5 Data Format and Message Structure

JSON dengan encoding UTF-8 digunakan sebagai format standar untuk pertukaran data terstruktur antarkomponen. Setiap respons API memiliki struktur yang konsisten agar *frontend* dapat membedakan keberhasilan proses, kesalahan validasi, serta kegagalan layanan. Identifier objek seperti ID *analysis batch*, *Maintenance Case*, *Maintenance Backlog*, dan *Pull Request* tetap disertakan apabila dibutuhkan untuk menjaga keterlacakan. Nilai tanggal dan waktu menggunakan format ISO 8601 agar interpretasi waktu tetap konsisten antar komponen.

Pesan kesalahan yang dikirimkan kepada pengguna berisi informasi yang relevan untuk memperbaiki input atau tindakan yang dilakukan. Informasi internal seperti *stack trace*, query database, token akses, credential, dan konfigurasi server tidak boleh dikirimkan melalui respons API. Kesalahan internal dapat dicatat pada sistem log untuk kebutuhan troubleshooting oleh tim teknis. Pemisahan tersebut menjaga agar informasi diagnostik tetap tersedia tanpa mengekspos detail sensitif kepada pengguna.

3.4.6 HTTP Response and Error Handling

Backend menggunakan kode status HTTP untuk membedakan hasil pemrosesan permintaan secara konsisten. Kelompok 2xx digunakan untuk permintaan yang berhasil atau telah diterima untuk diproses, sedangkan kelompok 4xx digunakan untuk masalah pada permintaan, autentikasi, otorisasi, atau kondisi data. Kelompok 5xx digunakan apabila kegagalan berasal dari sisi server atau layanan yang dibutuhkan oleh MaintainGov. Pemilihan kode status perlu konsisten agar *frontend* dapat menangani kondisi kegagalan tanpa bergantung hanya pada isi pesan teks.

Kode yang digunakan mencakup 200 OK untuk permintaan berhasil, 201 Created untuk sumber daya baru, dan 202 Accepted untuk proses yang telah diterima tetapi belum selesai. 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, dan 422 Unprocessable Entity digunakan sesuai karakteristik kesalahan yang terjadi. 429 Too Many Requests dapat digunakan ketika batas permintaan diterapkan, sedangkan 500 Internal Server Error, 502 Bad Gateway, dan 503 Service Unavailable digunakan untuk kegagalan pada server atau layanan pendukung. Pesan pada antarmuka tetap menggunakan konteks aktivitas pengguna dan tidak hanya menampilkan kode HTTP.

3.4.7 Communication Failure Handling

Kegagalan komunikasi tidak boleh menghasilkan perubahan status yang menyatakan bahwa suatu proses telah berhasil sebelum seluruh operasi yang diperlukan selesai. Jika *Analysis Service* tidak tersedia, batch dipertahankan pada kondisi yang menunjukkan bahwa analisis belum berhasil dan pengguna menerima informasi kegagalan yang sesuai. Jika GitHub tidak dapat diakses, data case dan backlog yang telah tersimpan tetap tersedia, sedangkan operasi yang membutuhkan

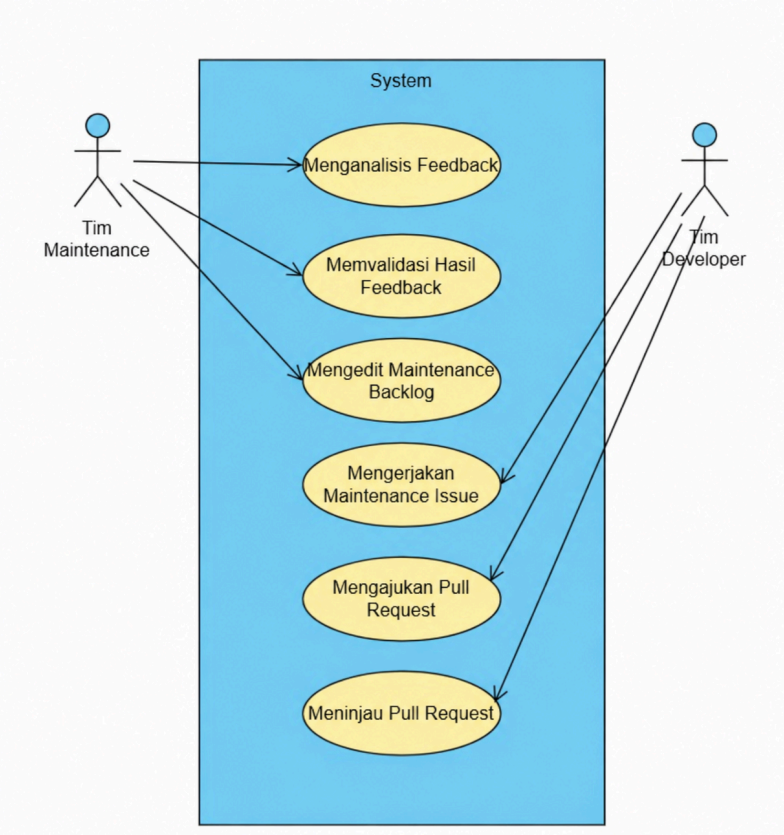
repository atau *Pull Request* tidak dilanjutkan. Prinsip ini mencegah gangguan pada satu komponen menyebabkan inkonsistensi pada keseluruhan proses maintenance.

Operasi yang aman untuk diulang dapat memiliki mekanisme *retry* ketika terjadi gangguan jaringan sementara. Tindakan yang berpotensi menghasilkan duplikasi, terutama pembuatan *Pull Request*, harus terlebih dahulu memeriksa hasil permintaan sebelumnya sebelum dikirim ulang. Waktu kegagalan dan informasi teknis yang relevan dapat dicatat pada log server untuk kebutuhan pemantauan dan troubleshooting. Respons kepada pengguna tetap disederhanakan agar menjelaskan kondisi yang terjadi tanpa mengekspos detail teknis internal.

4. System Features

4.1 System Feature Overview

MaintainGov sebagai aplikasi berbasis web yang menghubungkan feedback pengguna dengan proses software maintenance memiliki sejumlah fitur yang mendukung tujuan tersebut. Fitur-fitur ini terdiri atas dua fungsi pendukung, yaitu Authentication and Session Management dan enam use case utama, yaitu UC01 sampai UC06. Daftar fitur direpresentasikan dalam bentuk use case diagram pada gambar berikut.



Gambar 4.1 *Use Case Diagram* MaintainGov

Gambar tersebut merupakan Use Case Diagram MaintainGov yang menggambarkan aktor pada sistem beserta fitur yang dapat mereka gunakan. Terdapat dua aktor, yaitu Tim Maintenance dan Tim Developer. Kedua aktor menggunakan Login dan Logout, sedangkan enam use case utama dibagi sesuai tanggung jawab masing-masing aktor. Penjelasannya sebagai berikut:

1. Tim Maintenance
   Tim Maintenance menangani proses awal pengelolaan feedback hingga pembentukan Maintenance Backlog. Aksi yang dapat dilakukan Tim Maintenance di antaranya:

- Login, masuk ke MaintainGov menggunakan akun yang telah disediakan pada sistem. Akun tidak dibuat melalui registrasi mandiri.
- Logout, mengakhiri sesi aktif pada sistem.
- Menganalisis Feedback (UC01), mengunggah file CSV berisi feedback pengguna untuk diolah sistem menjadi Maintenance Case berstatus Pending.
- Memvalidasi Hasil Feedback (UC02), memeriksa Maintenance Case beserta ulasan terkait, lalu menetapkannya sebagai Valid atau Tidak Valid. Case yang valid membentuk Maintenance Backlog berstatus Ready for Development.
- Mengedit Maintenance Backlog (UC03), memperbarui informasi Maintenance Backlog sebelum issue dikerjakan Tim Developer.

2. Tim Developer
   Tim Developer bertanggung jawab atas proses teknis, mulai dari pengerjaan issue hingga peninjauan perubahan kode. Aksi yang dapat dilakukan Tim Developer di antaranya:
   - Login, masuk ke MaintainGov menggunakan akun yang telah disediakan pada sistem.
   - Logout, mengakhiri sesi aktif pada sistem.
   - Mengerjakan Maintenance Issue (UC04), mengambil issue berstatus Ready for Development, memperbaiki source code, menjalankan pengujian, dan mengirim perubahan ke GitHub hingga issue berstatus Ready for Pull Request.
   - Mengajukan Pull Request (UC05), membuat Pull Request di GitHub dari issue yang telah selesai dikerjakan, dengan menentukan branch tujuan, judul, dan deskripsi perubahan.
   - Meninjau Pull Request (UC06), memeriksa perubahan kode dan hasil pengujian, lalu memberikan keputusan Approve atau Request Changes.

Aktivitas merge dan deployment dilakukan di luar MaintainGov melalui GitHub dan CI/CD. MaintainGov hanya menyimpan atau menyinkronkan status yang relevan. Setiap fitur pada use case diagram diperjelas alur penggunaannya melalui use case specification, activity diagram, sequence diagram, user story, dan user story scenario pada subbab berikut.

4.2 System Feature 1 : Login

4.2.1 Description and Priority

Fitur login memungkinkan pengguna (Tim Maintenance atau Tim Developer) masuk ke MaintainGov menggunakan akun yang telah diprovision sebelumnya. Sistem ini tidak menyediakan self-registration, akun dan role pengguna sudah ditentukan di awal oleh pengelola sistem. Fitur ini merupakan prasyarat sebelum pengguna dapat mengakses UC01–UC06 sesuai role masing-masing.

Priority: Must Have.

4.2.2 Use Case Specification

Tabel 4.1 Use Case Specification Login

Nama Use Case	Login	
Aktor	Tim Maintenance, Tim Developer	
Deskripsi	Melakukan login untuk memasuki halaman beranda.	
Precondition	Aktor belum memiliki session aktif pada sistem dan memiliki akun yang telah disediakan.	
Postcondition	Aktor memiliki authenticated session dan berada pada halaman Analisis Feedback.	
Skenario Normal	Aktor	Sistem
	1. Membuka halaman Login.	
		2. Menampilkan halaman dan formulir Login.
	3. Mengisi email dan password pada formulir.	
	4. Menekan tombol "Login"	
		5. Memeriksa kelengkapan input.
		6. Memverifikasi credential terhadap data pengguna.
		7. Memperoleh role pengguna.
		8. Membuat authenticated session.

		9. Mengarahkan pengguna ke /analisis-feedback.
Skenario Alternatif 1 (Tidak mengisi field)	Aktor	Sistem
		5a. Menandai field wajib yang kosong dan menampilkan pesan kesalahan input.
		6a. Mempertahankan halaman Login.
	7a. Mengisi ulang formulir Login.	
Skenario Alternatif 2 (Credential tidak valid)	Aktor	Sistem
		6a. Menampilkan pesan "Login gagal" tanpa menyebutkan bagian credential yang salah.
		7a. Mempertahankan halaman Login.
	8a. Mengisi ulang formulir Login.	

4.2.3 Functional Requirements

- **FR-AUTH-01** Sistem harus menyediakan halaman Login pada /login.
- **FR-AUTH-02** Sistem harus menerima email pengguna.
- **FR-AUTH-03** Sistem harus menerima password dalam masked input.
- **FR-AUTH-04** Sistem harus menolak submit jika field wajib kosong.
- **FR-AUTH-05** Sistem harus memverifikasi credential terhadap account store.
- **FR-AUTH-06** Sistem harus menolak credential yang tidak valid dan menampilkan notifikasi "Login gagal" tanpa mengungkap apakah email atau password yang salah.
- **FR-AUTH-07** Sistem harus memperoleh role pengguna setelah autentikasi berhasil.
- **FR-AUTH-08** Sistem harus membuat authenticated session setelah credential valid.

- **FR-AUTH-09** Sistem harus mengarahkan pengguna yang berhasil login ke /analisis-feedback.
- **FR-AUTH-10** Sistem harus membatasi route/action berdasarkan role.

4.3 System Feature 2 : Menganalisis Feedback (UC01)

4.3.1 Description and Priority

Fitur Menganalisis Feedback memungkinkan Tim Maintenance mengunggah file CSV berisi feedback pengguna aplikasi, lalu sistem memproses file tersebut melalui tahapan preprocessing, klasifikasi kategori, analisis sentimen, penentuan severity, ekstraksi issue, dan pengelompokan kemiripan, sehingga terbentuk Maintenance Case berstatus Pending. Fitur ini merupakan titik awal pipeline MaintainGov sebelum Maintenance Case dapat divalidasi pada UC02.

Priority: Must Have.

4.3.2 Use Case Specification

Tabel 4.2 Use Case Specification Menganalisis Feedback

Nama Use Case	Menganalisis Feedback	
Aktor	Tim Maintenance	
Deskripsi	Memproses file CSV berisi feedback pengguna menjadi Maintenance Case terstruktur.	
Precondition	Tim Maintenance telah masuk ke MaintainGov. File CSV feedback telah tersedia.	
Postcondition	Maintenance Case berstatus Pending tersedia untuk proses validasi.	
Skenario Normal	**Aktor**	**Sistem**
	1. Membuka halaman Analisis Feedback.	
		2. Menampilkan fitur unggah file CSV.
	3. Memilih file CSV feedback.	
		4. Memeriksa format file CSV.

		5. Memeriksa struktur data CSV.
	6. Menekan tombol Analisis.	
		7. Membentuk Analysis Batch berstatus Diproses.
		8. Menyimpan data feedback sebagai ulasan.
		9. Melakukan preprocessing terhadap teks feedback.
		10. Melakukan analisis kategori feedback.
		11. Melakukan analisis sentimen feedback.
		12. Menentukan severity feedback.
		13. Mengekstraksi informasi issue.
		14. Mengelompokkan issue berdasarkan kemiripan.
		15. Membentuk Maintenance Case berstatus Pending berdasarkan hasil analisis.
Skenario Alternatif	**Aktor**	**Sistem**
	3a. Memilih file dengan format yang tidak sesuai (misalnya .txt atau .xlsx), atau file CSV dengan struktur kolom yang tidak lengkap.	
		4a. Memeriksa format dan struktur file CSV.
		5a. Mendeteksi file tidak

		memenuhi ketentuan.
		6a. Menampilkan daftar kolom wajib yang diperlukan.
		7a. Menampilkan notifikasi "Struktur CSV tidak sesuai".
		8a. Menonaktifkan tombol Analisis.
	9a. Memilih ulang file CSV yang sesuai dengan ketentuan.	
		10a. Mengaktifkan kembali tombol Analisis dan melanjutkan ke langkah 4.

4.3.3 Functional Requirements

- **FR-UC01-01** Sistem harus menyediakan kontrol unggah untuk file CSV pada /analisis-feedback.
- **FR-UC01-02** Sistem harus memvalidasi tipe file sebelum analisis dijalankan.
- **FR-UC01-03** Sistem harus memvalidasi struktur dan kolom wajib CSV.
- **FR-UC01-04** Sistem harus menolak CSV yang tidak sesuai, menampilkan notifikasi "Struktur CSV tidak sesuai" beserta daftar kolom wajib, dan menonaktifkan tombol Analisis.
- **FR-UC01-05** Sistem harus mengaktifkan kembali tombol Analisis setelah Tim Maintenance memilih file baru yang valid.
- **FR-UC01-06** Sistem harus membentuk record analysis_batch berstatus Diproses untuk file yang valid.
- **FR-UC01-07** Sistem harus menyimpan feedback dari batch sebagai baris ulasan sesuai PDM.
- **FR-UC01-08** Sistem harus menjalankan preprocessing terhadap teks feedback.
- **FR-UC01-09** Sistem harus menghasilkan hasil klasifikasi kategori feedback.
- **FR-UC01-10** Sistem harus menghasilkan hasil analisis sentimen.

- **FR-UC01-11** Sistem harus menentukan severity feedback.
- **FR-UC01-12** Sistem harus mengekstraksi informasi issue dari feedback.
- **FR-UC01-13** Sistem harus mengelompokkan issue berdasarkan kemiripan.
- **FR-UC01-14** Sistem harus menghitung priority score menggunakan metode yang telah dibaseline [TBD-ANL-06].
- **FR-UC01-15** Sistem harus membentuk Maintenance Case berstatus Pending dari hasil pengelompokan.
- **FR-UC01-16** Sistem harus mempertahankan keterkaitan antara ulasan dan maintenance_case yang terbentuk darinya.
- **FR-UC01-17** Sistem harus menampilkan notifikasi "Feedback berhasil diproses" setelah Maintenance Case terbentuk.
- **FR-UC01-18** Sistem menghasilkan rekomendasi teknis (recommendation) saat Maintenance Case terbentuk, sesuai kolom maintenance_case.recommendation pada PDM dan Sequence Diagram UC01. Langkah ini belum tertulis eksplisit pada Use Case Scenario UC01 versi terbaru dan perlu disinkronkan.

4.3.4 Data Mapping

pengguna_sistem, analysis_batch, ulasan, maintenance_case.

4.4 System Feature 3 : Memvalidasi Hasil Feedback (UC02)

4.4.1 Description and Priority

Fitur Memvalidasi Hasil Feedback memungkinkan Tim Maintenance memeriksa Maintenance Case berstatus Pending dan menentukan apakah case tersebut valid untuk diteruskan menjadi Maintenance Backlog. Validasi dilakukan pada level Maintenance Case, yang merupakan konsolidasi beberapa ulasan dengan masalah serupa, bukan pada tiap ulasan individual; ulasan-ulasan terkait hanya ditampilkan sebagai bahan pertimbangan.

Priority: Must Have.

4.4.2 Use Case Specification

Tabel 4.3 Use Case Specification Memvalidasi Hasil Feedback

Nama Use Case	Memvalidasi Hasil Feedback

Aktor	Tim Maintenance	
Deskripsi	Memeriksa Maintenance Case hasil analisis untuk menentukan kelayakan case menuju Maintenance Backlog.	
Precondition	Maintenance Case berstatus Pending telah tersedia.	
Postcondition	Status validasi Maintenance Case tersimpan. Case valid menghasilkan Maintenance Backlog.	
Skenario Normal	**Aktor**	**Sistem**
	1. Membuka halaman Maintenance Case.	
		2. Menampilkan daftar Maintenance Case berstatus Pending.
	3. Memilih satu Maintenance Case.	
		4. Menampilkan detail Maintenance Case beserta ulasan-ulasan terkait yang membentuknya.
	5. Memeriksa detail Maintenance Case dan ulasan terkait.	
	6. Menekan tombol Ya.	
		7. Mengubah status Maintenance Case menjadi Valid, lalu membentuk dan menghubungkan Maintenance Backlog berstatus Ready for Development.
		8. Menampilkan notifikasi "Maintenance Case berhasil divalidasi".
Skenario Alternatif	**Aktor**	**Sistem**

	6a. Menekan tombol Tidak.	
		7a. Mengubah status Maintenance Case menjadi Tidak Valid dan menghapus case dari daftar berstatus Pending.
		8a. Menampilkan notifikasi "Maintenance Case tidak diteruskan".
	9a. Kembali ke daftar Maintenance Case untuk memeriksa case lain.	

4.4.3 Functional Requirements

- **FR-UC02-01** Sistem harus menampilkan daftar Maintenance Case berstatus Pending pada /maintenance-case.
- **FR-UC02-02** Sistem harus menampilkan detail satu Maintenance Case meliputi judul issue, kategori, modul, fitur, severity, priority score, dan ringkasan masalah.
- **FR-UC02-03** Sistem harus menampilkan ulasan-ulasan yang menjadi dasar terbentuknya Maintenance Case tersebut.
- **FR-UC02-04** Sistem harus menyediakan keputusan validasi Ya dan Tidak.
- **FR-UC02-05** Sistem harus mengubah status Maintenance Case menjadi Valid ketika keputusan Ya dipilih.
- **FR-UC02-06** Sistem harus mencatat identitas Tim Maintenance yang melakukan validasi beserta waktu validasi.
- **FR-UC02-07** Sistem harus membentuk satu entri Maintenance Backlog berstatus Ready for Development ketika Maintenance Case dinyatakan Valid.
- **FR-UC02-08** Sistem harus menghubungkan Maintenance Backlog yang terbentuk dengan Maintenance Case asalnya.
- **FR-UC02-09** Sistem harus menampilkan notifikasi "Maintenance Case berhasil divalidasi" setelah validasi Ya berhasil.

- **FR-UC02-10** Sistem harus mengubah status Maintenance Case menjadi Tidak Valid ketika keputusan Tidak dipilih.
- **FR-UC02-11** Sistem tidak boleh membentuk Maintenance Backlog ketika Maintenance Case dinyatakan Tidak Valid.
- **FR-UC02-12** Sistem harus menghapus Maintenance Case yang Tidak Valid dari daftar berstatus Pending.
- **FR-UC02-13** Sistem harus menampilkan notifikasi "Maintenance Case tidak diteruskan" setelah validasi Tidak berhasil.
- **FR-UC02-14** Sistem menampilkan rekomendasi teknis (recommendation) pada detail Maintenance Case, sesuai kolom pada PDM dan Lo-Fi Halaman Detail Maintenance Case. Lihat FR-UC01-18.

4.4.4 Data Mapping

maintenance_case, ulasan, maintenance_backlog, pengguna_sistem.

4.5 System Feature 4 : Mengedit Maintenance Backlog (UC03)

4.5.1 Description and Priority

Fitur Mengedit Maintenance Backlog memungkinkan Tim Maintenance memperbarui informasi item Maintenance Backlog, priority, severity, modul, dan ringkasan masalah, sebelum item tersebut dikerjakan oleh Tim Developer pada UC04.

Priority: Must Have.

4.5.2 Use Case Specification

Tabel 4.4 Use Case Specification Mengedit Maintenance Backlog

Nama Use Case	Mengedit Maintenance Backlog
Aktor	Tim Maintenance
Deskripsi	Memperbarui informasi Maintenance Backlog sebelum issue mulai dikerjakan Tim Developer.
Precondition	Maintenance Backlog berstatus Ready for Development tersedia.
Postcondition	Informasi terbaru Maintenance Backlog tersimpan.

Skenario Normal	**Aktor**	**Sistem**
	1. Membuka halaman Maintenance Backlog.	
		2. Menampilkan daftar Maintenance Backlog.
	3. Memilih item backlog.	
		4. Menampilkan detail backlog.
	5. Menekan tombol Edit.	
		6. Menampilkan formulir perubahan backlog.
	7. Mengubah informasi backlog.	
	8. Menekan tombol Simpan.	
		9. Memvalidasi kelengkapan dan format data.
		10. Memperbarui data Maintenance Backlog dan mencatat waktu perubahan terakhir.
		11. Menampilkan notifikasi "Perubahan berhasil disimpan".
Skenario Alternatif	**Aktor**	**Sistem**
		9a. Mendeteksi field wajib yang kosong.

4.5.3 Functional Requirements

- **FR-UC03-01** Sistem harus menampilkan daftar Maintenance Backlog pada /maintenance-backlog.

- **FR-UC03-02** Sistem harus menampilkan detail satu item backlog.
- **FR-UC03-03** Sistem harus menyediakan tindakan Edit pada detail backlog.
- **FR-UC03-04** Sistem harus menyediakan formulir perubahan yang dapat mengubah priority, severity, modul, dan ringkasan masalah.
- **FR-UC03-05** Sistem harus memvalidasi kelengkapan field wajib sebelum menyimpan perubahan.
- **FR-UC03-06** Sistem harus menandai field yang belum lengkap dan menampilkan notifikasi "Data wajib belum lengkap" jika validasi gagal.
- **FR-UC03-07** Sistem harus mempertahankan formulir dalam kondisi terbuka dan data sebelumnya tidak hilang jika validasi gagal.
- **FR-UC03-08** Sistem harus memperbarui data Maintenance Backlog ketika validasi berhasil.
- **FR-UC03-09** Sistem harus mencatat waktu perubahan terakhir (updated_at) setiap kali data backlog diperbarui.
- **FR-UC03-10** Sistem harus menampilkan notifikasi "Perubahan berhasil disimpan" setelah data tersimpan.

4.5.4 Data Mapping

maintenance_backlog, maintenance_case.

4.6 System Feature 5 : Mengerjakan Maintenance Issue (UC04)

4.6.1 Description and Priority

Fitur Mengerjakan Maintenance Issue memungkinkan Tim Developer mengambil item Maintenance Backlog berstatus Ready for Development untuk dikerjakan. Perbaikan source code dilakukan di luar MaintainGov (repository lokal Developer), MaintainGov hanya mencatat penugasan, perubahan status, dan event git push yang diterima dari GitHub.

Priority: Must Have.

4.6.2 Use Case Specification

Tabel 4.5 Use Case Specification Mengerjakan Maintenance Issue

Nama Use Case	Mengerjakan Maintenance Issue

Aktor	Tim Developer	
Deskripsi	Mengambil Maintenance Issue dari backlog untuk proses implementasi perubahan source code.	
Precondition	Maintenance Issue berstatus Ready for Development telah tersedia.	
Postcondition	Perubahan kode tersedia pada GitHub. Status issue menjadi Ready for Pull Request.	
Skenario Normal	**Aktor**	**Sistem**
	1. Membuka halaman Maintenance Backlog.	
		2. Menampilkan issue berstatus Ready for Development.
	3. Memilih Maintenance Issue.	
		4. Menampilkan detail Maintenance Issue.
	5. Menekan tombol Kerjakan Issue.	
		6. Memeriksa ketersediaan issue.
		7. Menetapkan issue kepada Tim Developer dan mengubah status menjadi Sedang Dikerjakan.
	8. Mengirim perubahan kode melalui git push (perbaikan source code dan pengujian dilakukan di luar sistem MaintainGov, pada repository	

	lokal Developer).	
		9. Menerima informasi perubahan dari GitHub.
		10. Mengubah status issue menjadi Ready for Pull Request.
Skenario Alternatif	**Aktor**	**Sistem**
		6a. Mendeteksi assignment aktif pada Maintenance Issue (issue sudah diambil Developer lain).
		7a. Membatalkan proses pengambilan issue dan menampilkan status Sedang Dikerjakan beserta identitas Developer penerima issue.

4.6.3 Functional Requirements

- **FR-UC04-01** Sistem harus menampilkan issue yang dapat dikerjakan Developer.
- **FR-UC04-02** Sistem harus menampilkan detail Maintenance Issue.
- **FR-UC04-03** Sistem harus menyediakan tindakan Kerjakan Issue hanya untuk issue berstatus Ready for Development.
- **FR-UC04-04** Sistem harus memeriksa ketersediaan/assignment issue sebelum penugasan dilakukan.
- **FR-UC04-05** Sistem harus menolak klaim dan menampilkan notifikasi "Issue telah diambil Developer lain" jika issue sudah ditugaskan ke Developer lain.
- **FR-UC04-06** Sistem harus menonaktifkan tombol Kerjakan Issue pada issue yang sudah ditugaskan.
- **FR-UC04-07** Sistem harus menyimpan identitas Developer yang menerima penugasan (assigned_to). Kolom ini belum dikonfirmasi tersedia pada CDM/PDM terbaru dan perlu ditambahkan sebelum requirement ini dapat diverifikasi.

- **FR-UC04-08** Sistem harus mengubah status Maintenance Issue menjadi Sedang Dikerjakan setelah penugasan berhasil.
- **FR-UC04-09** Sistem harus menerima event perubahan kode (git push) dari GitHub untuk issue yang sedang dikerjakan.
- **FR-UC04-10** Sistem harus mengubah status Maintenance Issue menjadi Ready for Pull Request setelah event perubahan kode diterima.
- **FR-UC04-11** Sistem tidak boleh menampilkan panel log aktivitas teknis (perintah git, hasil pengujian lokal) pada antarmuka MaintainGov, karena aktivitas tersebut terjadi di luar batas sistem.

4.6.4 Data Mapping

maintenance_backlog, pengguna_sistem, github_repository.

4.7 System Feature 6 : Mengajukan Pull Request (UC05)

4.7.1 Description and Priority

Fitur Mengajukan Pull Request memungkinkan Tim Developer mengajukan perubahan kode yang telah di-push, melalui integrasi langsung dengan GitHub API, untuk item Maintenance Issue berstatus Ready for Pull Request.

Priority: Must Have.

4.7.2 Use Case Specification

Tabel 4.6 Use Case Specification Mengajukan Pull Request

Nama Use Case	Mengajukan Pull Request	
Aktor	Tim Developer	
Deskripsi	Mengajukan perubahan kode melalui Pull Request GitHub untuk proses review.	
Precondition	Maintenance Issue berstatus Ready for Pull Request.	
Postcondition	Pull Request berstatus Open tersedia pada GitHub. Metadata PR tersimpan pada MaintainGov.	
Skenario Normal	**Aktor**	**Sistem**
	1. Membuka Maintenance	

	Issue berstatus Ready for Pull Request.	
		2. Menampilkan detail Maintenance Issue dan informasi branch sumber.
	3. Menekan tombol Ajukan Pull Request.	
		4. Memeriksa koneksi GitHub.
		5. Menampilkan formulir Pull Request.
	6. Memilih branch tujuan.	
	7. Mengisi judul Pull Request.	
	8. Mengisi deskripsi perubahan.	
	9. Menekan tombol Create Pull Request.	
		10. Memvalidasi informasi Pull Request.
		11. Mengirim permintaan pembuatan Pull Request ke GitHub dan menyimpan metadata hasilnya.
		12. Mengubah status Pull Request menjadi Open.
		13. Menampilkan notifikasi "Pull Request berhasil dibuat".

4.7.3 Functional Requirements

- **FR-UC05-01** Sistem harus memastikan Maintenance Issue berstatus Ready for Pull Request sebelum menampilkan tindakan Ajukan Pull Request.

- **FR-UC05-02** Sistem harus menampilkan informasi branch sumber pada detail Maintenance Issue.
- **FR-UC05-03** Sistem harus memeriksa koneksi GitHub ketika tombol Ajukan Pull Request ditekan.
- **FR-UC05-04** Sistem harus menghentikan proses dan menampilkan notifikasi "Koneksi GitHub tidak tersedia, pengajuan belum diproses" jika koneksi GitHub gagal, tanpa mengubah status Maintenance Issue.
- **FR-UC05-05** Sistem harus menampilkan formulir Pull Request setelah koneksi GitHub berhasil diperiksa.
- **FR-UC05-06** Sistem harus menyediakan pemilihan branch tujuan.
- **FR-UC05-07** Sistem harus menerima judul Pull Request.
- **FR-UC05-08** Sistem harus menerima deskripsi Pull Request.
- **FR-UC05-09** Sistem harus memvalidasi kelengkapan informasi Pull Request sebelum pengajuan dikirim.
- **FR-UC05-10** Sistem harus mengirim permintaan pembuatan Pull Request ke GitHub melalui GitHub API/App setelah validasi berhasil.
- **FR-UC05-11** Sistem harus menyimpan branch sumber, branch tujuan, judul, deskripsi, Developer pengaju, github_pr_number, dan github_pr_url sesuai PDM.
- **FR-UC05-12** Sistem harus menetapkan status Pull Request menjadi Open setelah pembuatan pada GitHub berhasil.
- **FR-UC05-13** Sistem tidak boleh menandai pengajuan berhasil apabila GitHub gagal membuat Pull Request.
- **FR-UC05-14** Sistem harus menampilkan notifikasi "Pull Request berhasil dibuat" setelah status Open tersimpan.

4.7.4 Data Mapping

maintenance_backlog, pull_request, github_repository, pengguna_sistem.

4.8 System Feature 7 : Meninjau Pull Request (UC06)

4.8.1 Description and Priority

Fitur Meninjau Pull Request memungkinkan Tim Developer memeriksa perubahan kode pada Pull Request berstatus Open, lalu memberikan keputusan Approve atau Request Changes.

Priority: Must Have.

4.8.2 Use Case Specification

Tabel 4.7 Use Case Specification Meninjau Pull Request

Nama Use Case	Meninjau Pull Request	
Aktor	Tim Developer	
Deskripsi	Memeriksa perubahan kode pada Pull Request untuk menentukan kelayakan menuju proses merge.	
Precondition	Pull Request berstatus Open telah tersedia.	
Postcondition	Hasil review Pull Request tersimpan. Status PR tersinkron dengan GitHub.	
Skenario Normal	**Aktor**	**Sistem**
	1. Membuka halaman Pull Request.	
		2. Menampilkan Pull Request berstatus Open.
	3. Memilih Pull Request.	
		4. Menampilkan detail Pull Request dan Maintenance Issue terkait.
	5. Memeriksa perubahan source code, kesesuaiannya terhadap Maintenance Issue, dan hasil pengujian.	
	6. Menekan tombol Approve.	
		7. Mengirim keputusan review ke GitHub dan menyimpan hasil Code Review.

		8. Mengubah status Pull Request menjadi Approved.
		9. Menampilkan status Ready for Merge.
Skenario Alternatif	**Aktor**	**Sistem**
	6a. Menekan tombol Request Changes.	
		7a. Menampilkan field catatan perbaikan.
	8a. Mengisi catatan perbaikan.	

4.8.3 Functional Requirements

- **FR-UC06-01** Sistem harus menampilkan daftar Pull Request berstatus Open.
- **FR-UC06-02** Sistem harus menampilkan detail Pull Request dan Maintenance Issue terkait.
- **FR-UC06-03** Sistem harus menampilkan branch sumber dan branch tujuan.
- **FR-UC06-04** Sistem harus menampilkan Developer pengaju.
- **FR-UC06-05** Sistem harus menampilkan hasil pengujian yang tersedia dari GitHub.
- **FR-UC06-06** Sistem harus menampilkan perubahan source code (diff) yang tersedia dari GitHub.
- **FR-UC06-07** Sistem harus menyediakan tindakan Approve.
- **FR-UC06-08** Sistem harus menyediakan tindakan Request Changes beserta field catatan perbaikan.
- **FR-UC06-09** Sistem harus mencatat identitas reviewer dan waktu review.
- **FR-UC06-10** Sistem harus menyimpan catatan perbaikan ketika Request Changes dipilih.
- **FR-UC06-11** Sistem harus mengirim keputusan review (Approved/Changes Requested) ke GitHub.
- **FR-UC06-12** Sistem harus mengubah status Pull Request menjadi Approved dan menampilkan status Ready for Merge ketika keputusan Approve berhasil disinkronkan.

- **FR-UC06-13** Sistem harus mengubah status Pull Request menjadi Changes Requested ketika keputusan Request Changes berhasil disinkronkan.
- **FR-UC06-14** Sistem harus mengubah status Maintenance Issue menjadi Sedang Dikerjakan setelah Request Changes.
- **FR-UC06-15** Sistem harus menampilkan notifikasi "Perubahan memerlukan perbaikan" setelah Request Changes tersimpan.
- **FR-UC06-16** Sistem harus menyimpan/menyinkronkan status Merged apabila merge dilakukan secara eksternal pada GitHub.

4.8.4 Data Mapping

pull_request, code_review, maintenance_backlog, pengguna_sistem.

4.9 System Feature 8 : Logout

4.9.1 Description and Priority

Fitur logout memungkinkan pengguna (Tim Maintenance atau Tim Developer) mengakhiri sesi penggunaan MaintainGov dengan keluar dari akunnya. Saat logout, sistem menghapus authenticated session pengguna sehingga halaman dan fitur yang membutuhkan autentikasi tidak dapat diakses lagi tanpa login ulang. Fitur ini penting untuk menjaga keamanan akun, terutama ketika sistem diakses dari perangkat yang digunakan bersama. Fitur ini hanya dapat digunakan oleh pengguna yang telah berhasil login.

Priority: Must Have.

4.9.2 Use Case Specification

Tabel 4.8 Use Case Specification Logout

Nama Use Case	Logout
Aktor	Tim Maintenance, Tim Developer
Deskripsi	Aktor mengakhiri sesi penggunaan sistem dengan keluar dari akun. Sistem menghapus authenticated session aktor sehingga halaman yang membutuhkan autentikasi tidak dapat diakses lagi tanpa login ulang.

Precondition	Aktor telah login dan memiliki authenticated session.	
Postcondition	Session aktor berakhir dan aktor berada pada halaman /login.	
Skenario Normal	**Aktor**	**Sistem**
	1. Menekan tombol "Logout".	
		2. Memeriksa session aktif.
		3. Mengakhiri session pengguna.
		4. Mengarahkan pengguna ke /login.
		5. Menampilkan halaman Login.
Skenario Exception	**Aktor**	**Sistem**
		3a. Mendeteksi session tidak valid dan tidak memprosesnya sebagai session aktif.
		4a. Mengarahkan pengguna ke /login.

4.9.3 Functional Requirements

- **FR-AUTH-11** Sistem harus menyediakan tindakan Logout pada authenticated shell.
- **FR-AUTH-12** Sistem harus mengakhiri session saat Logout.
- **FR-AUTH-13** Sistem harus mengarahkan pengguna ke /login setelah Logout.
- **FR-AUTH-14** Sistem harus menolak reuse session yang telah invalid/expired.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Kebutuhan performa menetapkan batas waktu respons, kapasitas, dan jumlah aktivitas yang harus dapat dilayani MaintainGov pada kondisi operasional normal. Waktu pemrosesan Feedback Analysis dipisahkan dari waktu respons halaman biasa karena proses analisis mengolah teks dalam jumlah besar. Pengukuran dilakukan pada lingkungan pengujian yang mengikuti Tabel 2.1 dengan koneksi jaringan normal. Target berikut merupakan batas penerimaan awal dan diverifikasi melalui pengujian performa pada tahap implementasi.

- NFR-PERF-01 Sistem harus menampilkan halaman utama, yaitu Maintenance Case, Maintenance Backlog, Maintenance Issue, dan Pull Request, dalam waktu paling lama 3 detik untuk sekurang-kurangnya 95% permintaan pada kondisi beban normal.
- NFR-PERF-02 Sistem harus memberikan respons dalam waktu paling lama 2 detik untuk sekurang-kurangnya 95% operasi API yang tidak melibatkan Feedback Analysis atau layanan eksternal.
- NFR-PERF-03 Sistem harus melayani sekurang-kurangnya 25 pengguna aktif secara bersamaan tanpa kegagalan fungsi, dengan rata-rata waktu respons operasi standar tidak melebihi 5 detik.
- NFR-PERF-04 Sistem harus menerima file CSV berukuran hingga 10 MB atau paling banyak 10.000 baris feedback dalam satu Analysis Batch, dan menolak file yang melebihi batas tersebut.
- NFR-PERF-05 Sistem harus menyelesaikan validasi format dan struktur file CSV dalam waktu paling lama 5 detik untuk file yang berada dalam batas ukuran yang didukung.
- NFR-PERF-06 Sistem ditargetkan menyelesaikan Analysis Batch berisi hingga 5.000 feedback dalam waktu paling lama 120 detik pada lingkungan pengujian, tidak termasuk waktu antre apabila terdapat proses analisis lain yang berjalan. Nilai ini merupakan target awal dan disesuaikan setelah metode analisis ditetapkan.
- NFR-PERF-07 Sistem harus menampilkan status proses analisis (Diproses, Selesai, atau Gagal) sehingga pengguna tidak perlu mempertahankan satu koneksi HTTP terbuka sampai analisis selesai.
- NFR-PERF-08 Sistem harus menyelesaikan pencarian, filter, dan pengurutan pada daftar case, backlog, issue, dan Pull Request dalam waktu paling lama 2 detik untuk data operasional hingga 10.000 record.

- NFR-PERF-09 Sistem harus menampilkan indikator proses apabila permintaan ke GitHub membutuhkan waktu lebih dari 2 detik, dan menangani *timeout* apabila GitHub tidak merespons dalam 30 detik. Waktu respons GitHub tidak dihitung sebagai performa internal MaintainGov.
- NFR-PERF-10 Sistem harus mencatat waktu mulai dan waktu selesai setiap Analysis Batch agar kecepatan pemrosesan dapat diukur.

5.2 Safety Requirements

MaintainGov tidak mengendalikan perangkat fisik sehingga kebutuhan safety berkaitan dengan integritas data dan pencegahan perubahan workflow yang keliru. Risiko utamanya adalah kehilangan data, perubahan status yang tidak konsisten, kegagalan proses analisis, dan kegagalan komunikasi dengan GitHub. Kegagalan pada satu proses tidak boleh merusak data lain yang telah tersimpan dengan benar.

- NFR-SAF-01 Sistem harus menggunakan transaksi basis data pada operasi yang mengubah beberapa data terkait, sehingga seluruh perubahan berhasil secara utuh atau dibatalkan apabila salah satu operasi gagal.
- NFR-SAF-02 Sistem tidak boleh membentuk Maintenance Backlog apabila hasil validasi Maintenance Case belum berhasil disimpan.
- NFR-SAF-03 Sistem tidak boleh menganggap Maintenance Case berhasil dibentuk ketika Analysis Service gagal, dan harus menandai Analysis Batch terkait sebagai Gagal.
- NFR-SAF-04 Sistem tidak boleh mengubah status Pull Request menjadi Open, Approved, atau status berhasil lain sebelum respons GitHub yang sesuai diterima dan diverifikasi.
- NFR-SAF-05 Sistem harus mencegah satu Maintenance Issue diklaim secara bersamaan oleh lebih dari satu Developer.
- NFR-SAF-06 Sistem tidak boleh menghapus data yang telah tersimpan sebagai akibat kegagalan layanan eksternal.
- NFR-SAF-07 Sistem harus mencadangkan basis data sekurang-kurangnya satu kali setiap 24 jam.
- NFR-SAF-08 Mekanisme pemulihan basis data harus memenuhi *Recovery Point Objective* paling lama 24 jam dan *Recovery Time Objective* paling lama 4 jam.
- NFR-SAF-09 Sistem harus memeriksa apakah permintaan sebelumnya telah berhasil sebelum mengirim ulang operasi eksternal yang berpotensi menghasilkan duplikasi, terutama pembuatan Pull Request.

- NFR-SAF-10 Sistem harus memastikan kesalahan pada satu Analysis Batch tidak menghentikan akses ke case, backlog, issue, atau Pull Request lain yang tidak terkait.
- NFR-SAF-11 Sistem harus meminta konfirmasi sebelum menyimpan keputusan Valid atau Tidak Valid pada Maintenance Case dan keputusan Approve atau Request Changes pada Pull Request.

5.3 Security Requirements

Kebutuhan keamanan berfokus pada autentikasi, pembatasan hak akses, perlindungan kredensial, keamanan komunikasi, dan pencatatan aktivitas penting. Akses tidak sah dapat memengaruhi proses pengembangan karena MaintainGov terhubung dengan repository GitHub. Kontrol keamanan diterapkan pada backend dan tidak bergantung pada elemen antarmuka.

- Semua fitur yang bersifat sensitif harus memerlukan autentikasi dalam mengaksesnya, di mana aktor harus *login* menggunakan kredensial yang valid. JWT kemudian dihasilkan dengan waktu kadaluarsa yang terbatas untuk mengurangi risiko penyalahgunaan.
- Sistem harus menerapkan *Role-Based Access Control* (RBAC) baik untuk pengguna maupun admin.
- Semua data sensitif yang masuk ke *database* harus melalui proses *hashing* dengan algoritma yang kuat
- Seluruh komunikasi antara klien dan server wajib menggunakan enkripsi HTTPS (SSL/TLS) untuk melindungi data dari penyadapan

5.4 Software Quality Attributes

Atribut kualitas yang relevan bagi MaintainGov adalah reliability, usability, maintainability, dan portability. Keempatnya dipilih karena sistem terdiri atas beberapa komponen internal dan satu layanan eksternal yang harus tetap dapat digunakan, dikembangkan, dan diuji secara terpisah.

a. Reliability

- NFR-REL-01 Sistem harus memiliki tingkat ketersediaan sekurang-kurangnya 99,5% per bulan, tidak termasuk pemeliharaan terjadwal.
- NFR-REL-02 Sistem harus memastikan kegagalan satu permintaan tidak menyebabkan kegagalan pada sesi atau proses pengguna lain yang tidak terkait.
- NFR-REL-03 Sistem harus mempertahankan data yang telah berhasil di-*commit* ketika layanan eksternal mengalami gangguan.

- NFR-REL-04 Sistem harus membedakan status proses yang gagal dari proses yang berhasil atau masih berjalan.

b. Usability

- NFR-USA-01 Sistem harus menampilkan menu dan tindakan sesuai role pengguna.
- NFR-USA-02 Sistem harus menampilkan indikator proses atau status pada setiap proses yang membutuhkan waktu lebih dari 2 detik.
- NFR-USA-03 Sistem harus memberikan umpan balik yang menjelaskan hasil setiap tindakan, baik berhasil maupun gagal.
- NFR-USA-04 Sistem harus menggunakan istilah, status, format tanggal, label field, dan pola navigasi secara konsisten pada seluruh halaman.
- NFR-USA-05 Sistem harus menampilkan setiap status dalam bentuk teks, tidak hanya melalui warna atau ikon.
- NFR-USA-06 Pada user acceptance testing, sekurang-kurangnya 90% skenario utama harus dapat diselesaikan pengguna tanpa bantuan langsung dari pengembang.

c. Maintainability

- NFR-MNT-01 Sistem harus memisahkan frontend, backend, Analysis Service, dan integrasi GitHub menjadi komponen dengan tanggung jawab yang jelas.
- NFR-MNT-02 Endpoint, struktur data utama, konfigurasi lingkungan, dan dependensi sistem harus terdokumentasi.
- NFR-MNT-03 Business rule kritis mengenai validasi case, assignment issue, perubahan status, dan Pull Request harus memiliki pengujian otomatis.
- NFR-MNT-04 Perubahan model pada Analysis Service tidak boleh mewajibkan perubahan frontend selama kontrak API dipertahankan.
- NFR-MNT-05 Perubahan struktur basis data harus dikelola melalui migrasi yang dikendalikan versinya.

d. Portability and Compatibility

- NFR-PORT-01 Antarmuka harus berfungsi pada dua versi mayor terbaru Chrome, Edge, Firefox, dan Safari.
- NFR-PORT-02 Komponen server harus dapat dijalankan pada lingkungan Linux yang memenuhi dependensi aplikasi.

- NFR-PORT-03 Konfigurasi aplikasi harus dipisahkan dari source code menggunakan konfigurasi lingkungan (environment configuration).
- NFR-PORT-04 Komunikasi antarlayanan harus menggunakan protokol dan format standar, yaitu HTTPS, REST, dan JSON.

5.5 Business Rules

Aturan bisnis menetapkan kondisi operasional yang mengendalikan perpindahan objek dan aktor MaintainGov dari satu tahap proses ke tahap berikutnya. Aturan ini tidak menjelaskan kualitas sistem, melainkan batas proses yang harus selalu dipatuhi. Aturan berikut menjadi dasar validasi workflow pada backend dan konsisten dengan use case serta status.

- BR-01 Hanya akun yang disediakan secara internal dan memiliki role aktif yang dapat mengakses MaintainGov.
- BR-02 Tim Maintenance berwenang atas Feedback Analysis, validasi Maintenance Case, dan pengelolaan Maintenance Backlog.
- BR-03 Tim Developer berwenang atas pengerjaan Maintenance Issue, pengajuan Pull Request, dan Code Review.
- BR-04 File feedback hanya dapat diproses apabila memenuhi struktur CSV dan memiliki kolom wajib yang ditetapkan sistem.
- BR-05 Setiap proses analisis yang diterima sistem harus dicatat sebagai satu Analysis Batch.
- BR-06 Maintenance Case yang baru dibentuk harus berstatus Pending.
- BR-07 Hanya Tim Maintenance yang dapat menetapkan Maintenance Case sebagai Valid atau Tidak Valid.
- BR-08 Hanya Maintenance Case berstatus Valid yang dapat membentuk Maintenance Backlog.
- BR-09 Maintenance Case berstatus Tidak Valid tidak boleh menghasilkan Maintenance Backlog.
- BR-10 Satu Maintenance Case yang valid hanya boleh membentuk satu Maintenance Backlog utama untuk mencegah duplikasi pekerjaan.
- BR-11 Maintenance Backlog yang baru terbentuk harus berstatus Ready for Development.
- BR-12 Tim Maintenance hanya dapat mengubah informasi backlog sebelum item memasuki tahap pengerjaan Developer.

- BR-13 Maintenance Issue hanya dapat diambil apabila backlog berstatus Ready for Development dan belum memiliki Developer yang bertanggung jawab.
- BR-14 Setelah issue berhasil diambil, sistem harus mencatat Developer yang bertanggung jawab dan mengubah status menjadi Sedang Dikerjakan.
- BR-15 Issue yang telah diambil Developer lain tidak boleh diklaim kembali selama *assignment* masih aktif.
- BR-16 Pull Request hanya dapat diajukan untuk issue berstatus Ready for Pull Request.
- BR-17 Pull Request yang berhasil dibuat harus berstatus Open dan tetap terhubung dengan Maintenance Backlog sumbernya.
- BR-18 Satu Maintenance Backlog dapat terhubung dengan satu atau lebih Pull Request, misalnya ketika diperlukan revisi tambahan.
- BR-19 Setiap keputusan Code Review harus mencatat reviewer, keputusan, waktu review, dan catatan review apabila diperlukan.
- BR-20 Keputusan Request Changes harus mengubah Pull Request menjadi Changes Requested dan mengembalikan pekerjaan terkait ke status Sedang Dikerjakan.
- BR-21 Keputusan Approve harus mengubah Pull Request menjadi Approved dan memungkinkan pekerjaan dilanjutkan ke status Ready for Merge.
- BR-22 MaintainGov versi 1.0 tidak menjalankan merge atau deployment secara langsung. Status terkait dapat disimpan atau disinkronkan dari lingkungan pengembangan eksternal.
- BR-23 Hubungan antara feedback, Maintenance Case, Maintenance Backlog, repository, Pull Request, dan Code Review tidak boleh diputus selama objek terkait masih tersimpan.
- BR-24 Perubahan status tidak boleh melewati tahapan workflow yang diwajibkan oleh proses bisnis MaintainGov.

6. Other Requirements

Appendix A: Glossary

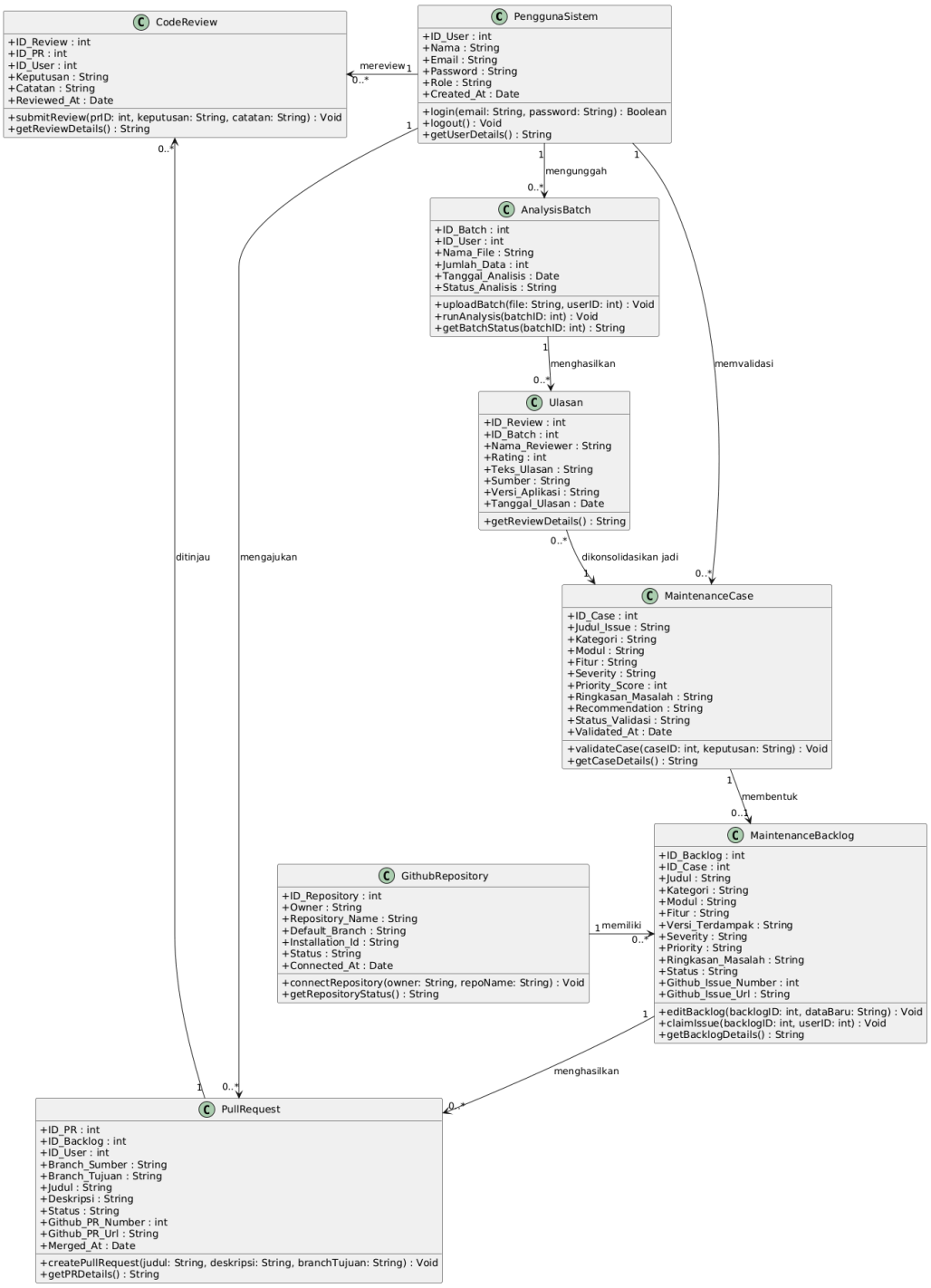
- Analysis Batch: Satu sesi pemrosesan file CSV yang berisi sejumlah *feedback* pengguna. Setiap batch memiliki informasi file, jumlah data, serta status proses seperti Diproses, Selesai, atau Gagal.
- Category Classification: Proses pengelompokan *feedback* berdasarkan kategori permasalahan yang teridentifikasi dari isi ulasan.
- Code Review: Proses peninjauan perubahan *source code* yang diajukan melalui *Pull Request*. Hasil review dapat berupa Approved atau Changes Requested.
- Feedback: Ulasan pengguna aplikasi yang menjadi sumber awal identifikasi permasalahan dan kebutuhan maintenance.
- Issue Extraction: Proses mengidentifikasi informasi permasalahan utama dari teks *feedback* agar dapat digunakan sebagai dasar pembentukan pekerjaan maintenance.
- Issue Grouping: Proses menggabungkan *feedback* yang membahas permasalahan serupa sehingga beberapa ulasan tidak menghasilkan pekerjaan maintenance yang redundan.
- Maintenance Backlog: Pekerjaan maintenance yang dibentuk dari *Maintenance Case* yang telah dinyatakan valid. Backlog menjadi acuan kerja bagi Tim Developer.
- Maintenance Case: Representasi terstruktur dari suatu masalah yang dihasilkan berdasarkan analisis satu atau beberapa *feedback* terkait. Case memuat informasi seperti kategori, modul, fitur terdampak, *severity*, prioritas, ringkasan masalah, dan hasil validasi.
- Maintenance Issue: Representasi *Maintenance Backlog* ketika telah memasuki tahap pengerjaan oleh Tim Developer. Istilah ini digunakan pada konteks workflow pengembangan dan tidak merepresentasikan entitas data yang terpisah dari backlog.
- Preprocessing: Tahapan awal pengolahan teks *feedback* sebelum dilakukan klasifikasi, analisis sentimen, ekstraksi issue, dan analisis lanjutan lainnya.
- Priority Score: Nilai yang merepresentasikan tingkat prioritas suatu permasalahan untuk membantu menentukan urutan penanganan maintenance.
- Pull Request: Pengajuan perubahan *source code* untuk ditinjau sebelum perubahan dapat digabungkan ke branch tujuan.
- Repository: Lokasi penyimpanan dan pengelolaan *source code* yang terhubung dengan pekerjaan maintenance.

- Sentiment Analysis: Proses identifikasi kecenderungan sentimen pada *feedback* sebagai salah satu informasi pendukung dalam analisis permasalahan.
- Severity: Tingkat keparahan suatu permasalahan berdasarkan dampaknya terhadap aplikasi atau pengguna.
- Tim Developer: Aktor yang bertanggung jawab terhadap pengerjaan *Maintenance Issue*, pengajuan *Pull Request*, serta *Code Review*.
- Tim Maintenance: Aktor yang bertanggung jawab terhadap analisis *feedback*, validasi *Maintenance Case*, dan pengelolaan *Maintenance Backlog*.
- Traceability: Kemampuan untuk menelusuri keterkaitan antara *feedback*, *Maintenance Case*, *Maintenance Backlog*, pekerjaan Developer, *Pull Request*, dan *Code Review*.
- Pending: Status awal *Maintenance Case* yang menunjukkan bahwa case belum memperoleh keputusan validasi dari Tim Maintenance.
- Valid: Status *Maintenance Case* yang menunjukkan bahwa case telah disetujui untuk diteruskan menjadi *Maintenance Backlog*.
- Tidak Valid: Status *Maintenance Case* yang menunjukkan bahwa case tidak diteruskan menjadi pekerjaan maintenance.
- Ready for Development: Status yang menunjukkan bahwa *Maintenance Backlog* telah tersedia untuk dikerjakan oleh Tim Developer.
- Sedang Dikerjakan: Status yang menunjukkan bahwa pekerjaan telah diambil dan sedang ditangani oleh Tim Developer.
- Ready for Pull Request: Status yang menunjukkan bahwa pengerjaan dan pengujian perubahan telah selesai sehingga dapat diajukan sebagai *Pull Request*.
- Open: Status awal *Pull Request* setelah berhasil dibuat dan sebelum memperoleh keputusan review.
- Changes Requested: Status *Pull Request* yang menunjukkan bahwa reviewer meminta perubahan atau perbaikan tambahan.
- Approved: Status *Pull Request* yang menunjukkan bahwa perubahan telah memperoleh persetujuan reviewer.
- Ready for Merge: Status yang menunjukkan bahwa perubahan telah disetujui dan siap dilanjutkan ke proses *merge*.

- Merged: Status yang menunjukkan bahwa perubahan telah berhasil digabungkan ke branch tujuan.
- Selesai: Status akhir pekerjaan maintenance setelah proses yang diperlukan terhadap pekerjaan tersebut telah diselesaikan.

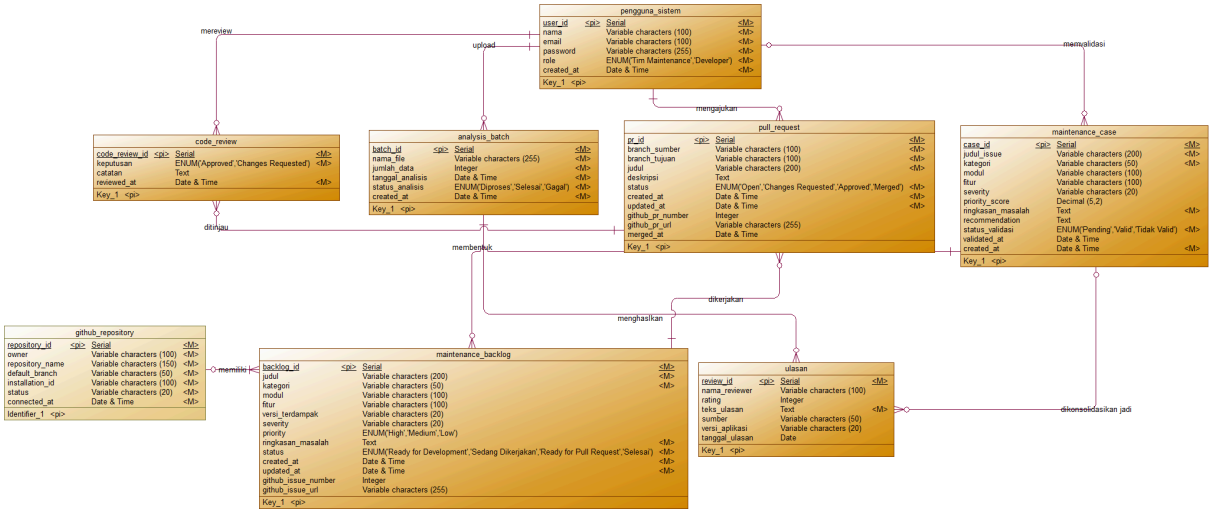
Appendix B: Analysis Models

A. Class Diagram

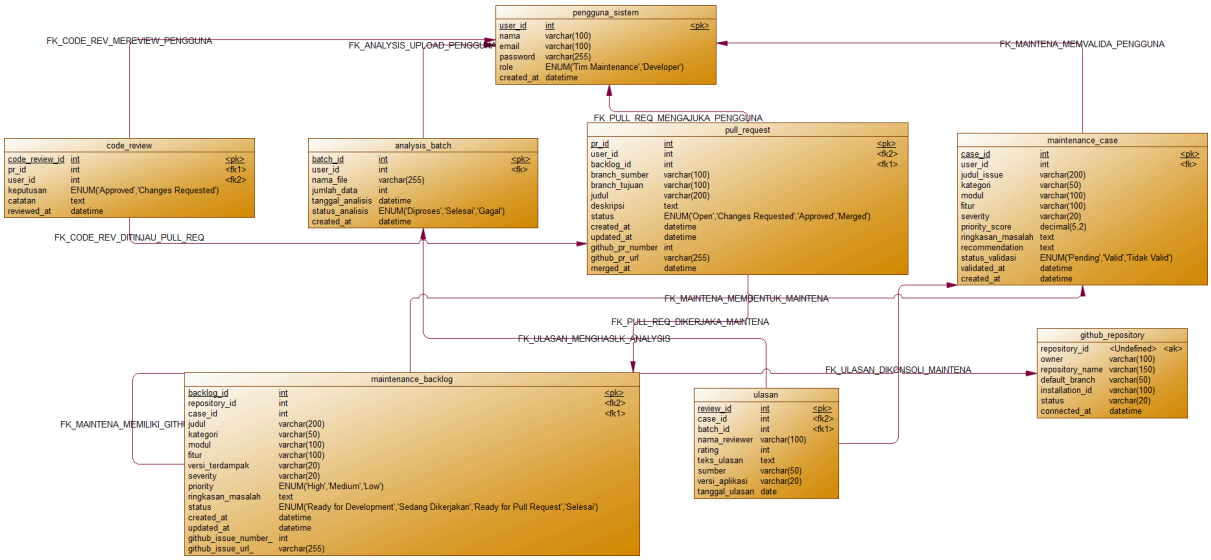


Gambar 6.1 *Class Diagram* MaintainGov

B. Desain Database



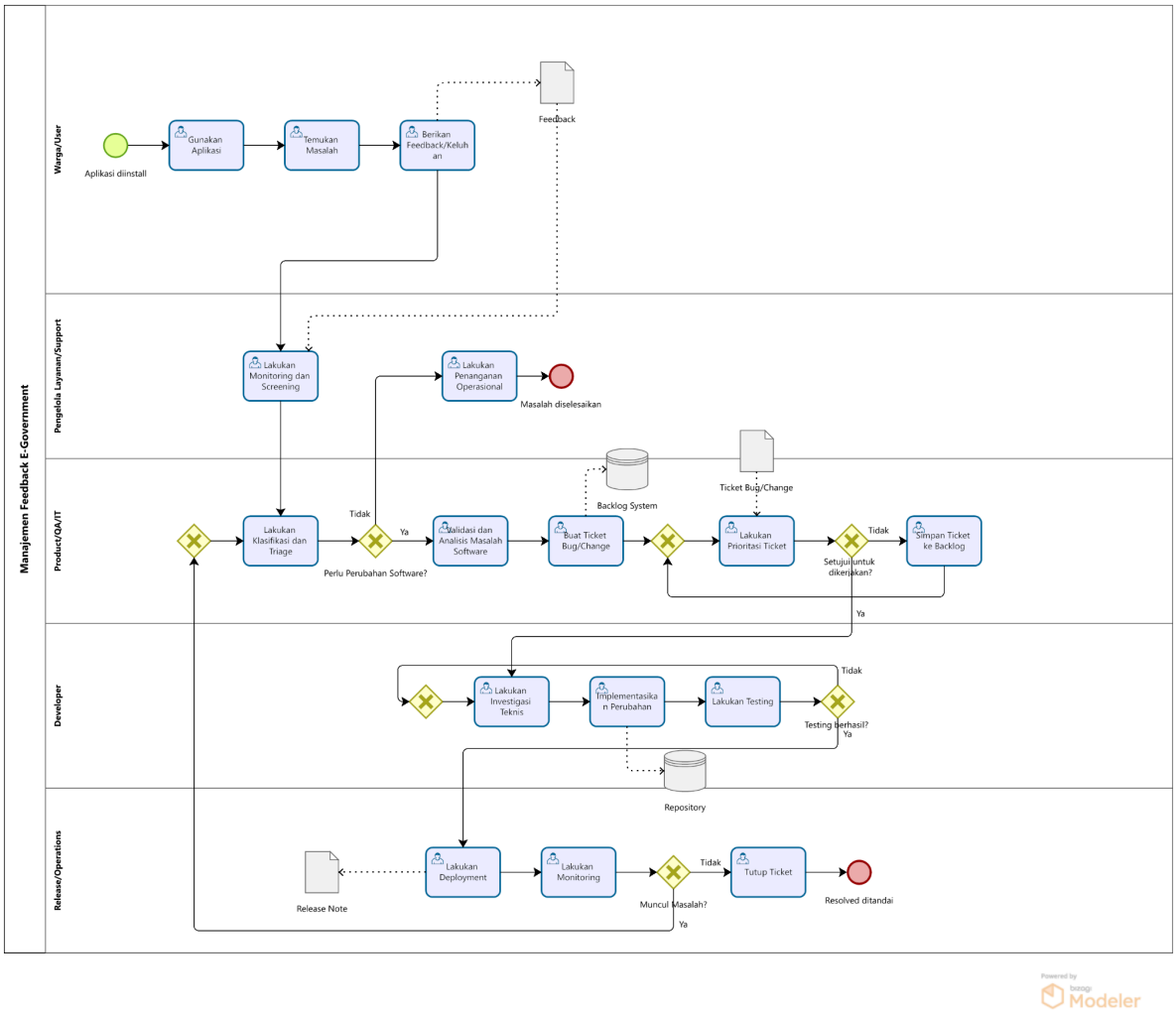
Gambar 6.2 *Conceptual Data Model* MaintainGov



Gambar 6.3 *Physical Data Model* MaintainGov

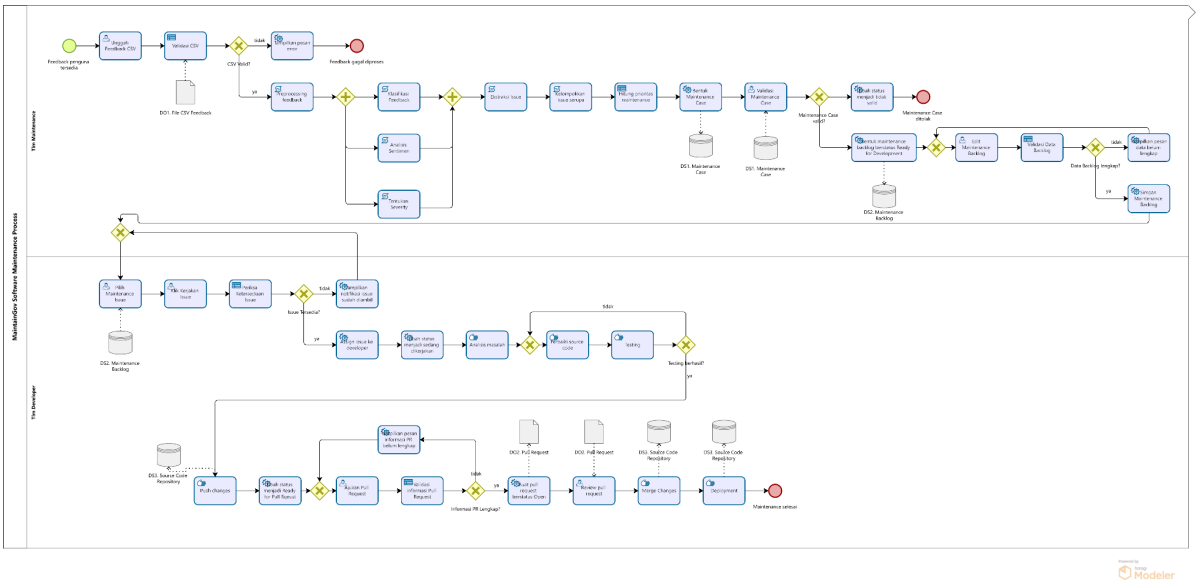
C. BPMN

1) BPMN As-Is



Gambar 6.4 *BPMN As-Is* MaintainGov

2) BPMN To-Be



Gambar 6.5 *BPMN To-Be* MaintainGov

D. User Story Scenario

1) Login

Tabel 6.1 User Story Scenario Login

Login
Feature: Login **In order to** access MaintainGov functions according to my role **As a** MaintainGov user **I want to** to log in to the system
Scenario: Successfully login with valid credentials **Given** I am on "/login" **And** I fill in "email" with a valid account email **And** I fill in "password" with a valid password **When** I press "Login" **Then** I should be on "/analisis-feedback"
Scenario: Login fails because credentials are invalid **Given** I am on "/login" **And** I fill in "email" with a valid account email **And** I fill in "password" with an incorrect password **When** I press "Login" **Then** I should see "Login gagal"

And I should still be on "/login"
Scenario: Login fails because a required field is empty **Given** I am on "/login" **And** I fill in "email" with a valid account email **And** I fill in "password" with "" **When** I press "Login" **Then** I should still be on "/login"

2) Menganalisis Feedback

Tabel 6.2 User Story Scenario Menganalisis Feedback

Analyze User Feedback
Feature: Analyze User Feedback **In order to** generate structured maintenance information from user feedback **As a** Maintenance Team **I want to** analyze feedback from a CSV file
Scenario: Successfully analyze feedback from a valid CSV file **Given** I am on “/analisis-feedback” **When** I attach the file “feedback_user.csv” to “form_file” **And** I press “Analisis” **Then** I should see “Hasil Analisis Feedback” **And** I should see “Maintenance Case”
Scenario: Fail to analyze because the CSV file is invalid **Given** I am on “/analisis-feedback” **When** I attach the file “feedback_user.txt” to “form_file” **And** I press “Analisis” **Then** I should see “File tidak dapat diproses”

3) Memvalidasi Hasil Feedback

Tabel 6.3 User Story Scenario Memvalidasi Hasil Feedback

Validate Maintenance Case
Feature: Validate Maintenance Case **In order to** ensure only valid Maintenance Cases are forwarded to the Maintenance Backlog

As a Maintenance Team **I want to** validate the feedback analysis result
Scenario: Mark the Maintenance Case as valid **Given** I am on "/maintenance-case" **When** I follow "MC-001" **And** I press "Ya" **Then** I should be on "/maintenance-case/MC-001" **And** I should see "Valid" **And** I should see "Maintenance Case berhasil divalidasi" **And** I should see "Ready for Development"
Scenario: Mark the Maintenance Case as invalid **Given** I am on "/maintenance-case/MC-001" **When** I press "Tidak" **Then** I should see "Tidak Valid" **And** I should see "Maintenance Case tidak diteruskan"

4) Mengedit Maintenance Backlog

Tabel 6.4 User Story Scenario Mengedit Maintenance Backlog

Edit Maintenance Backlog
Feature: Edit Maintenance Backlog **In order to** keep maintenance information accurate as a reference for the Developer **As a** Maintenance Team **I want to** edit an item in the Maintenance Backlog
Scenario: Successfully update a Maintenance Backlog item **Given** I am on "/maintenance-backlog" **When** I follow "BL-001" **And** I press "Edit" **And** I select "High" from "priority" **And** I press "Simpan" **Then** I should see "Perubahan berhasil disimpan" **And** the "priority" field should contain "High"
Scenario: Fail to save because required data is incomplete **Given** I am on "/maintenance-backlog/BL-001/edit"

When I fill in “priority” with “” **And** I press “Simpan” **Then** I should see “Data wajib belum lengkap”

5) Mengerjakan Maintenance Issue

Tabel 6.5 User Story Scenario Mengerjakan Maintenance Issue

Work on Maintenance Issue
Feature: Work on Maintenance Issue **In order to** complete the software fix based on the Maintenance Issue **As a** Developer **I want to** work on a Maintenance Issue
Scenario: Successfully work on a Maintenance Issue **Given** I am on “/maintenance-backlog” **When** I follow “M1-001” **And** I press “Kerjakan Issue” **Then** I should see “Sedang Dikerjakan” **When** I reload the page **Then** I should see “Ready for Pull Request”
Scenario: Maintenance issue has already been claimed **Given** I am on “/maintenance-backlog” **When** I follow “M1-001” **And** I press “Kerjakan Issue” **Then** I should see “Issue telah diambil Developer lain” **And** I should see “Sedang Dikerjakan”

6) Mengajukan Pull Request

Tabel 6.6 User Story Scenario Mengajukan Pull Request

Submit Pull Request
Feature: Submit Pull Request **In order to** submit code changes for review **As a** Developer **I want to** create a Pull Request
Scenario: Successfully create a Pull Request

Given I am on “/maintenance-issue/M1-001” **And** I should see "Ready for Pull Request" **When** I press "Ajukan Pull Request" **And** I select “main” from “branch_tujuan” **And** I fill in “judul” with “Fix upload lampiran LAKSA” **And** I fill in “deskripsi” with “Memperbaiki validasi attachment pada versi 5.2.1” **And** I press “Create Pull Request” **Then** I should see “Open” **And** I should see “Pull request berhasil dibuat”
Scenario: Pull Request information is incomplete **Given** I am on “/pull-request/new” **When** I press “Create Pull Request” **Then** I should see “Lengkapi informasi Pull Request”

7) Meninjau Pull Request

Tabel 6.7 User Story Scenario Meninjau Pull Request

Review Pull Request
Feature: Review Pull Request **In order to** ensure the code changes match the Maintenance Issue **As a** Developer **I want to** review a Pull Request
Scenario: Approve the Pull Request **Given** I am on “/pull-request/PR-001” **When** I press “Approve” **Then** I should see “Approved” **And** I should see “Ready for Merge”
Scenario: Request changes on the Pull Request **Given** I am on “/pull-request/PR-001” **When** I fill in “catatan_review” with “Perbaiki validasi input sebelum merge” **And** I press “Request Changes” **Then** I should see “Changes Requested” **And** I should see “Sedang Dikerjakan”

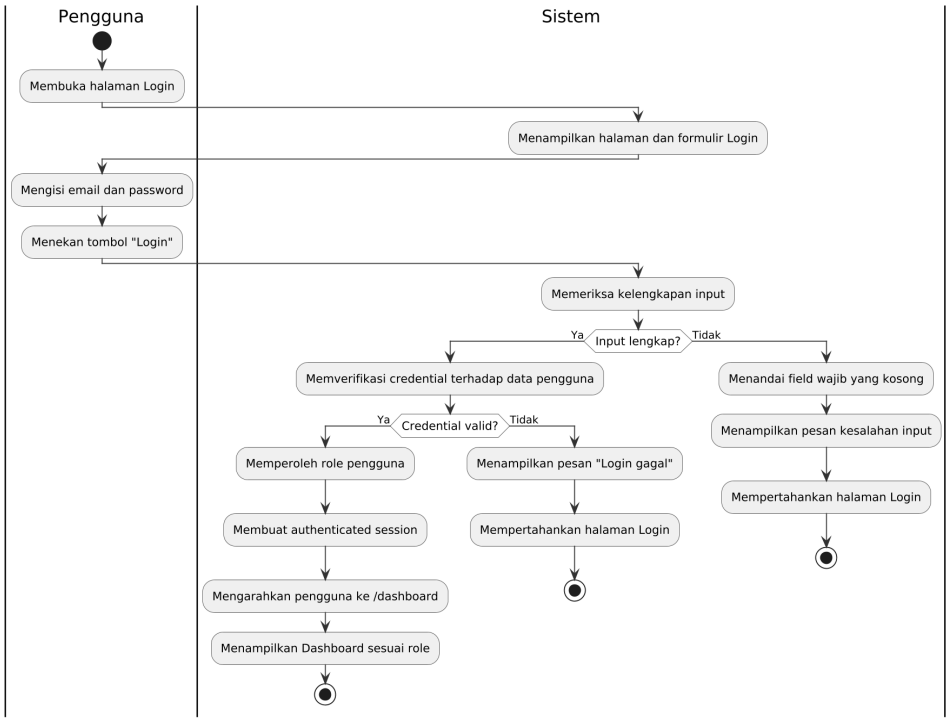
8) Logout

Tabel 6.8 User Story Scenario Logout

Logout
Feature: Logout **In order to** end my active session securely **As a** MaintainGov user **I want to** log out from the system
Scenario: Successfully logout **Given** I am authenticated **When** I press "Logout" **Then** I should be on "/login" **And** I should no longer be authenticated
Scenario: Logout when the session has already expired **Given** my session has expired **When** I press "Logout" **Then** I should be on "/login"

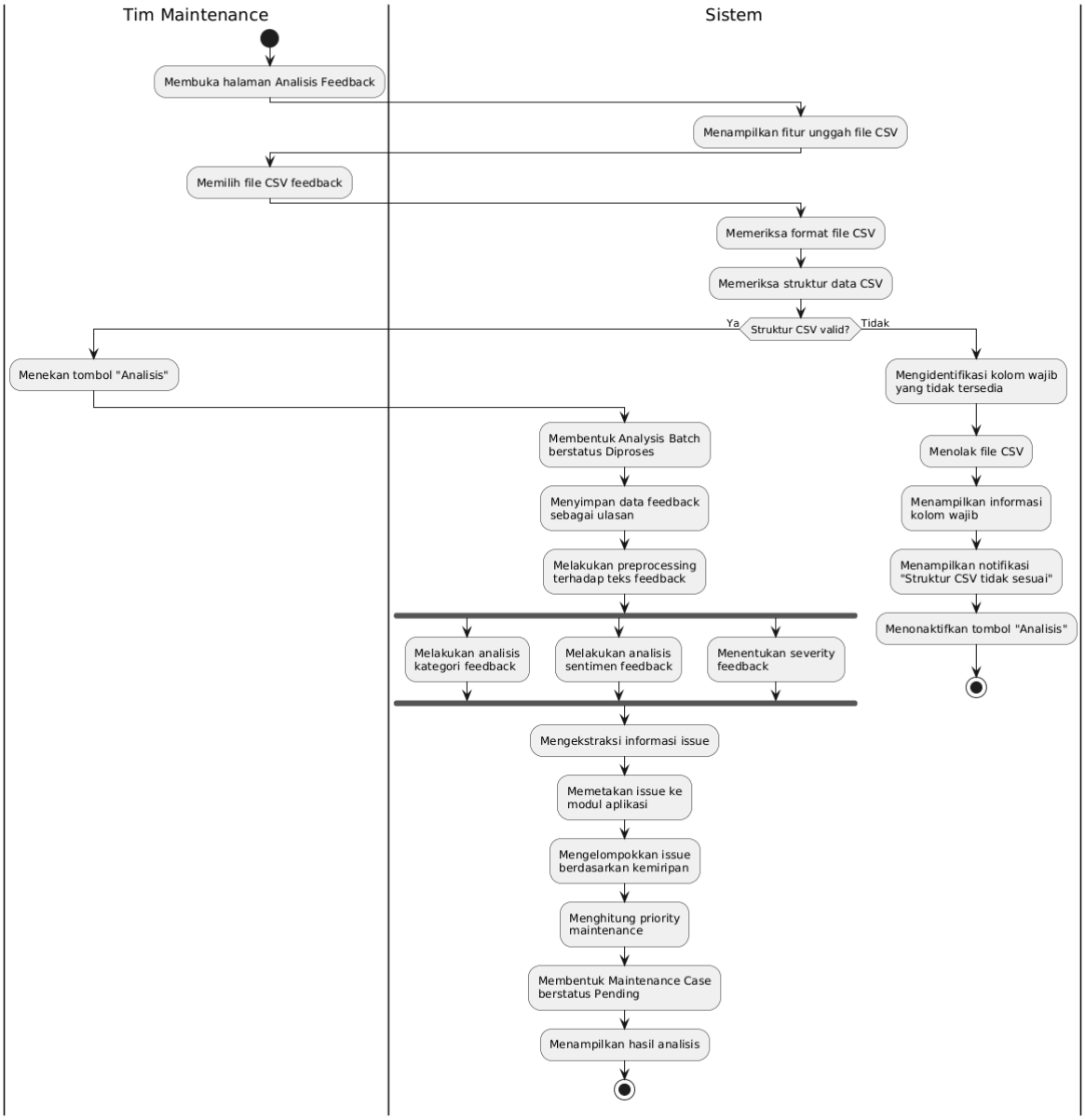
E. Activity Diagram

1) Login



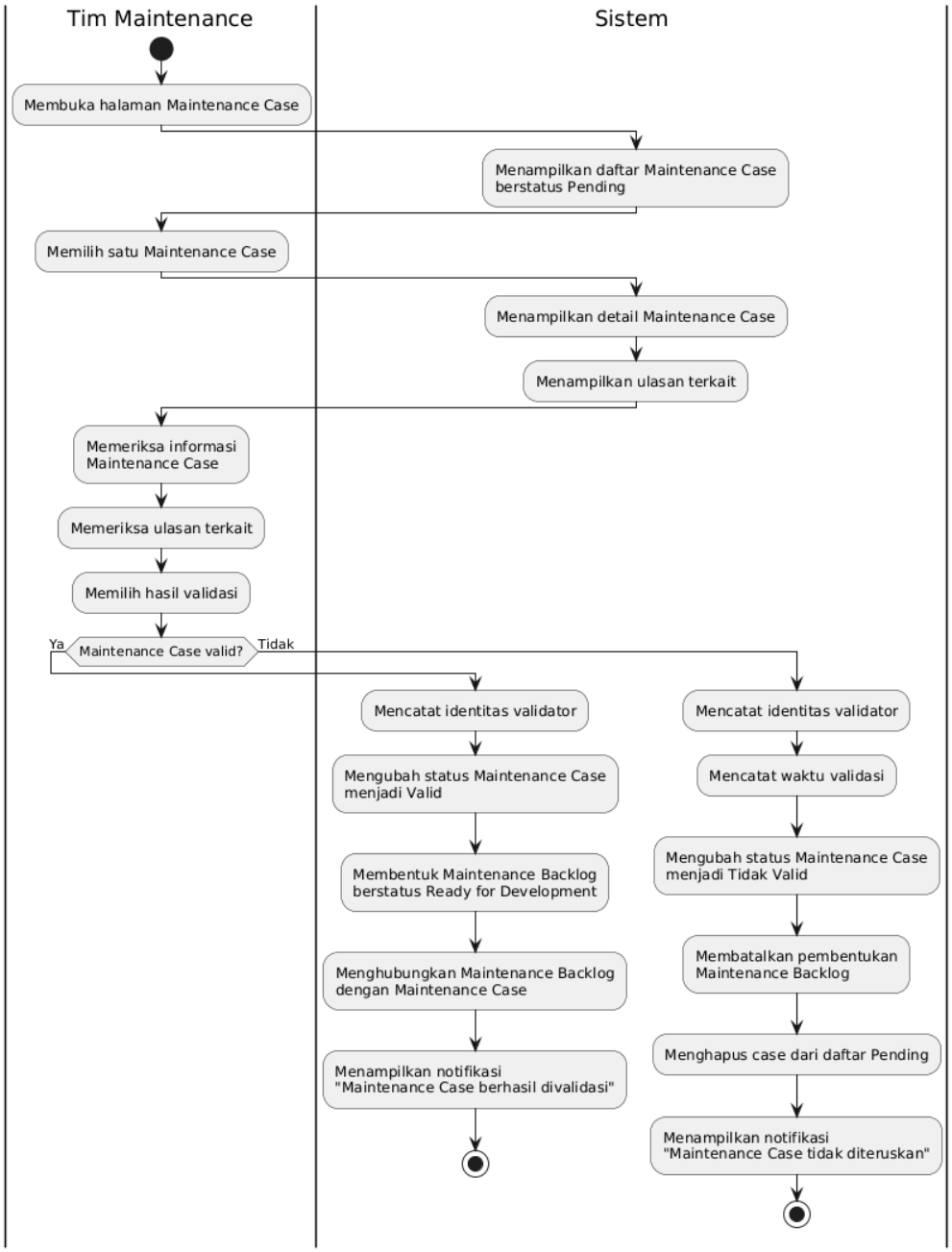
Gambar 6.6 *Activity Diagram* Login

2) Menganalisis Feedback



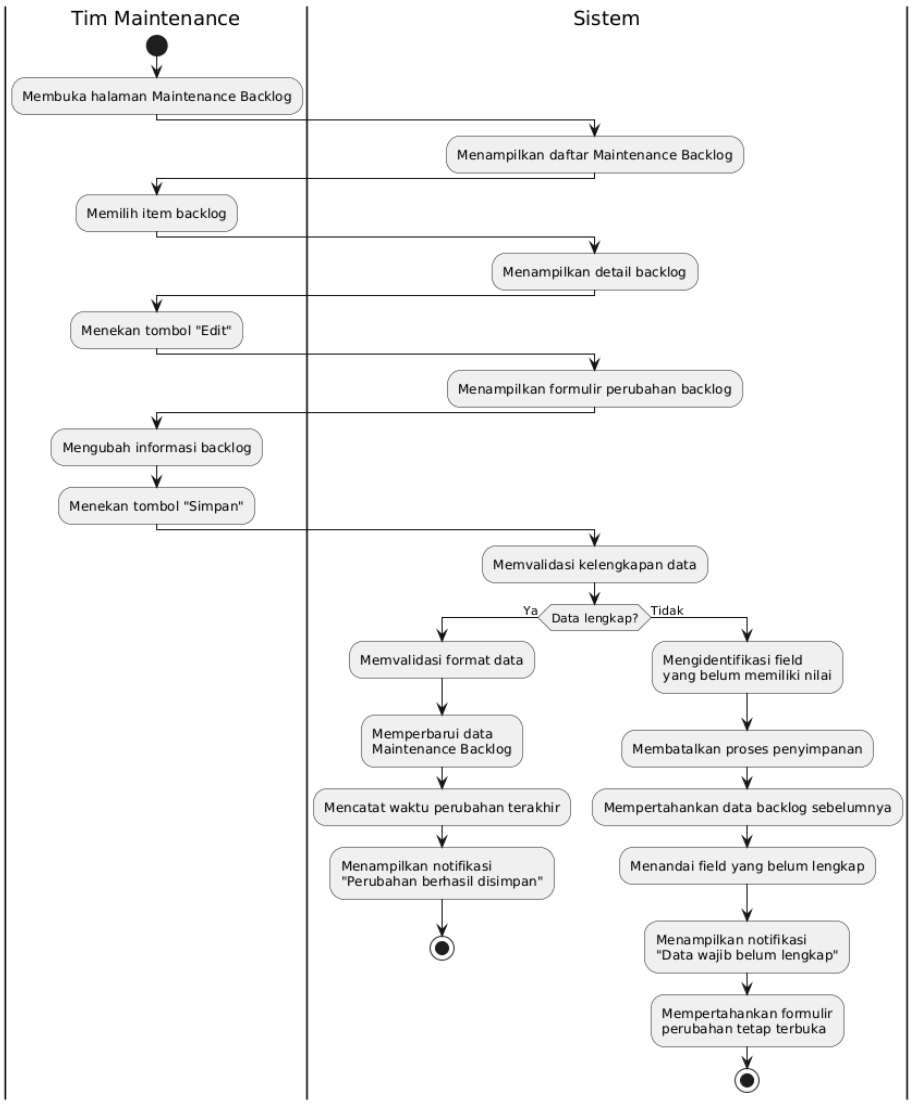
Gambar 6.7 *Activity Diagram* Menganalisis Feedback

3) Memvalidasi Hasil Feedback



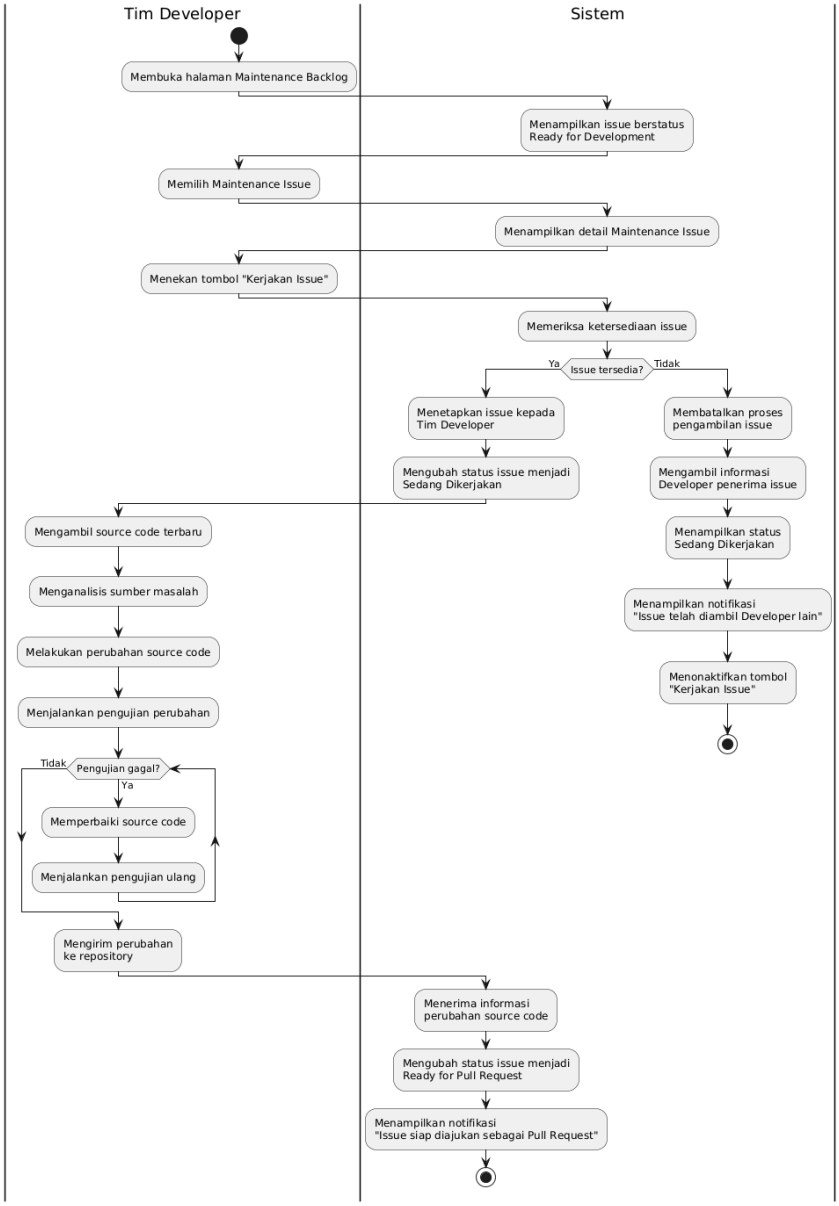
Gambar 6.8 *Activity Diagram* Memvalidasi Hasil Feedback

4) Mengedit Maintenance Backlog



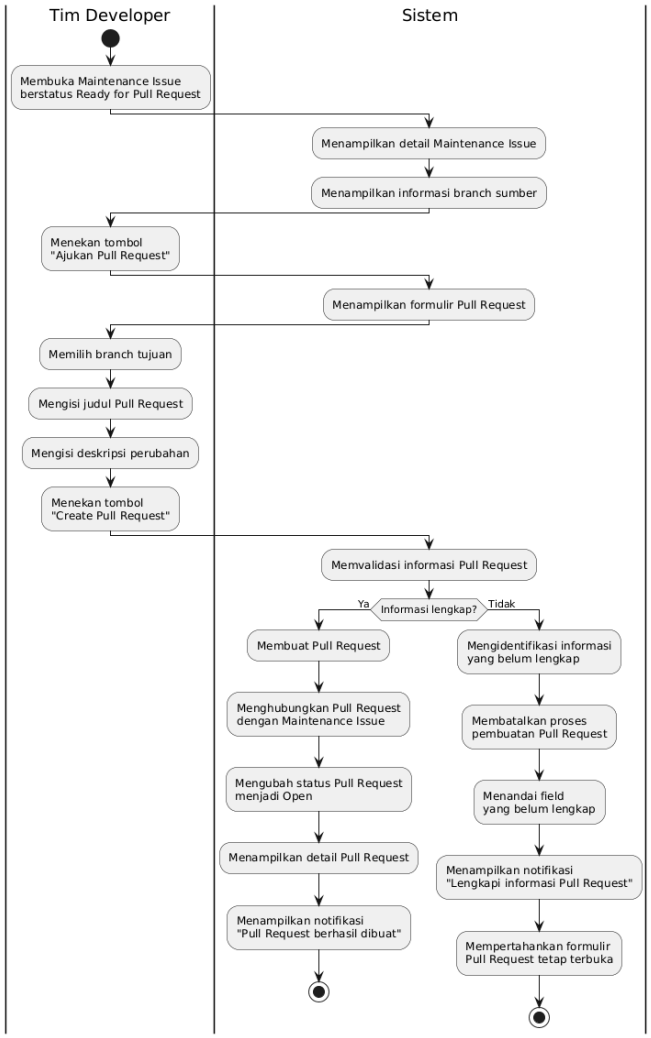
Gambar 6.9 *Activity Diagram* Mengedit Maintenance Backlog

5) Mengerjakan Maintenance Issue



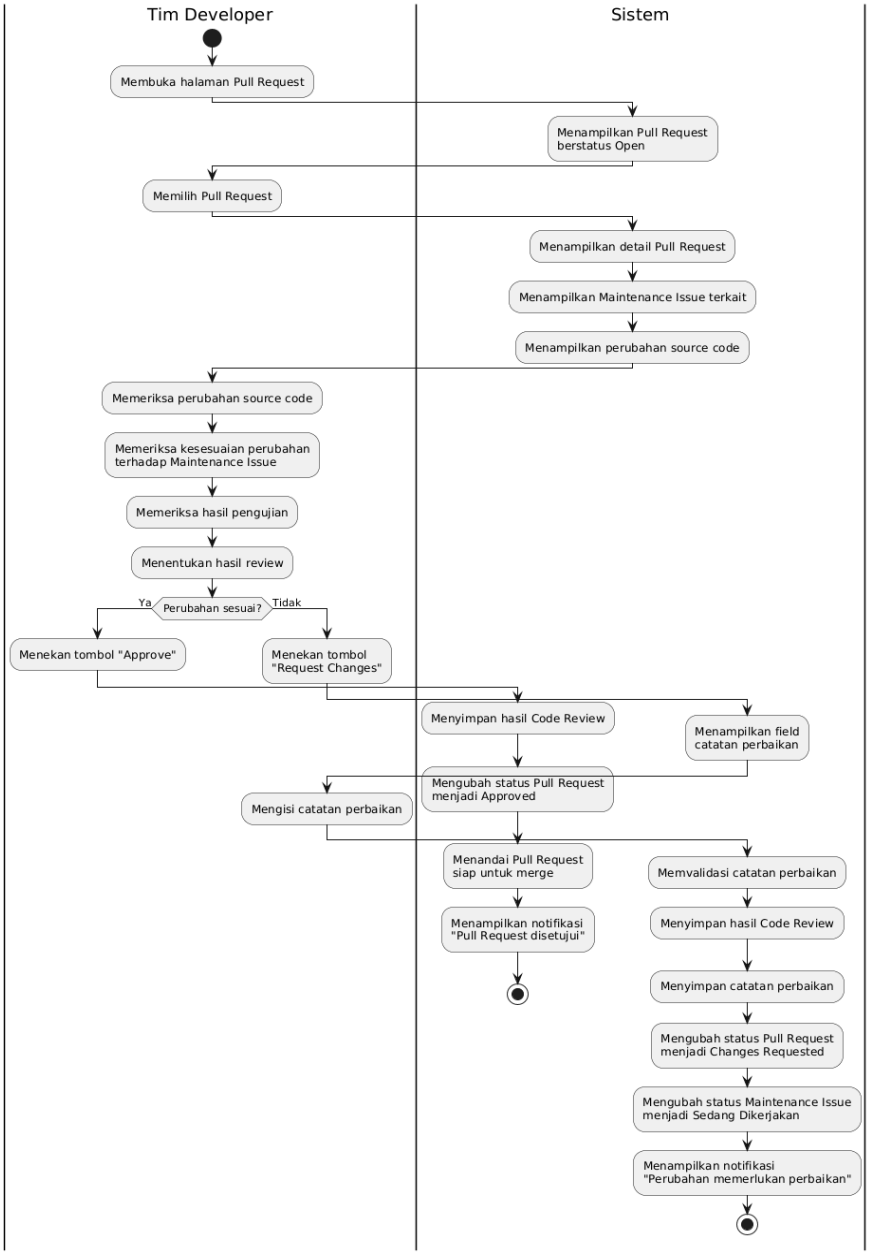
Gambar 6.10 *Activity Diagram* Mengerjakan Maintenance Issue

6) Mengajukan Pull Request



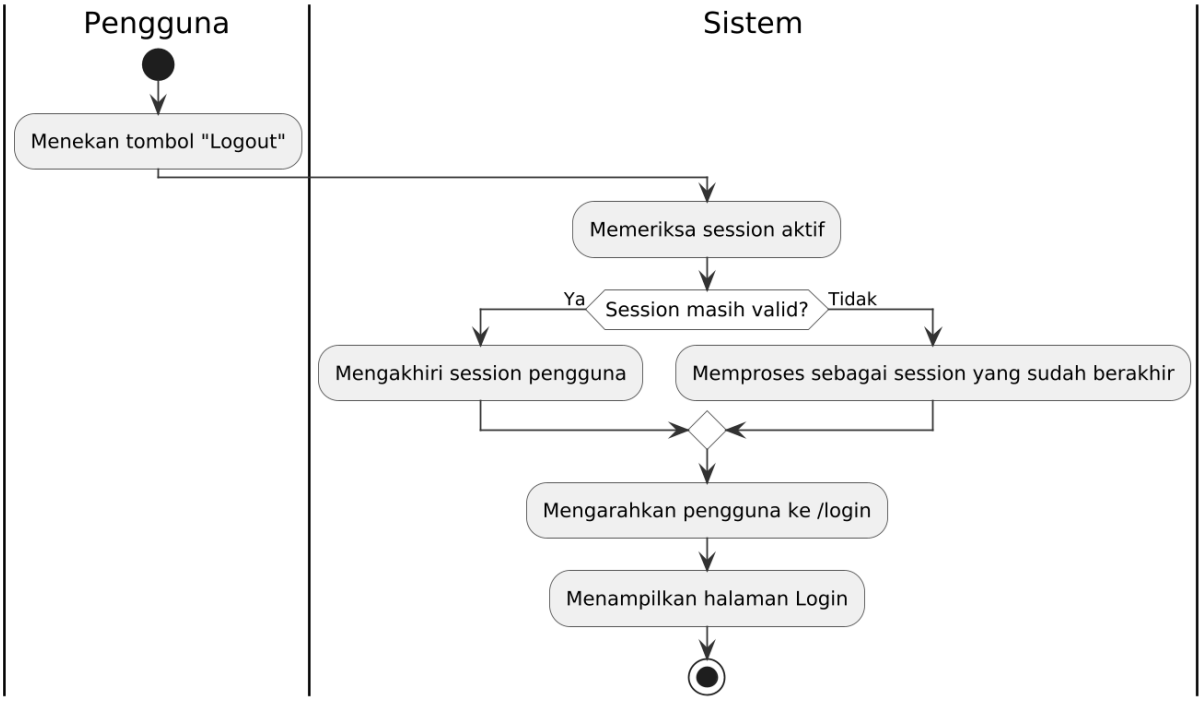
Gambar 6.11 *Activity Diagram* Mengajukan Pull Request

7) Meninjau Pull Request



Gambar 6.12 *Activity Diagram* Meninjau Pull Request

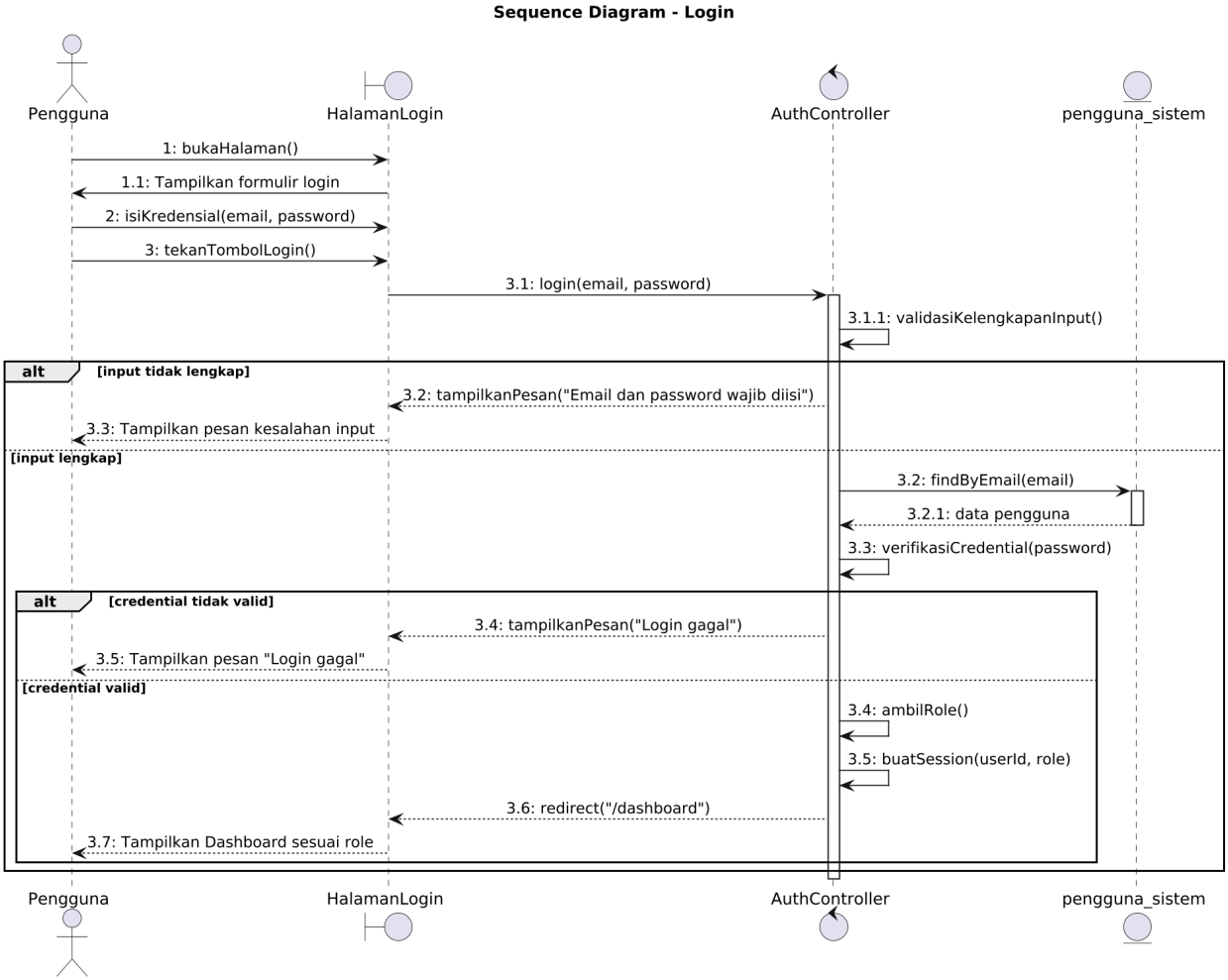
8) Logout



Gambar 6.13 *Activity Diagram* Logout

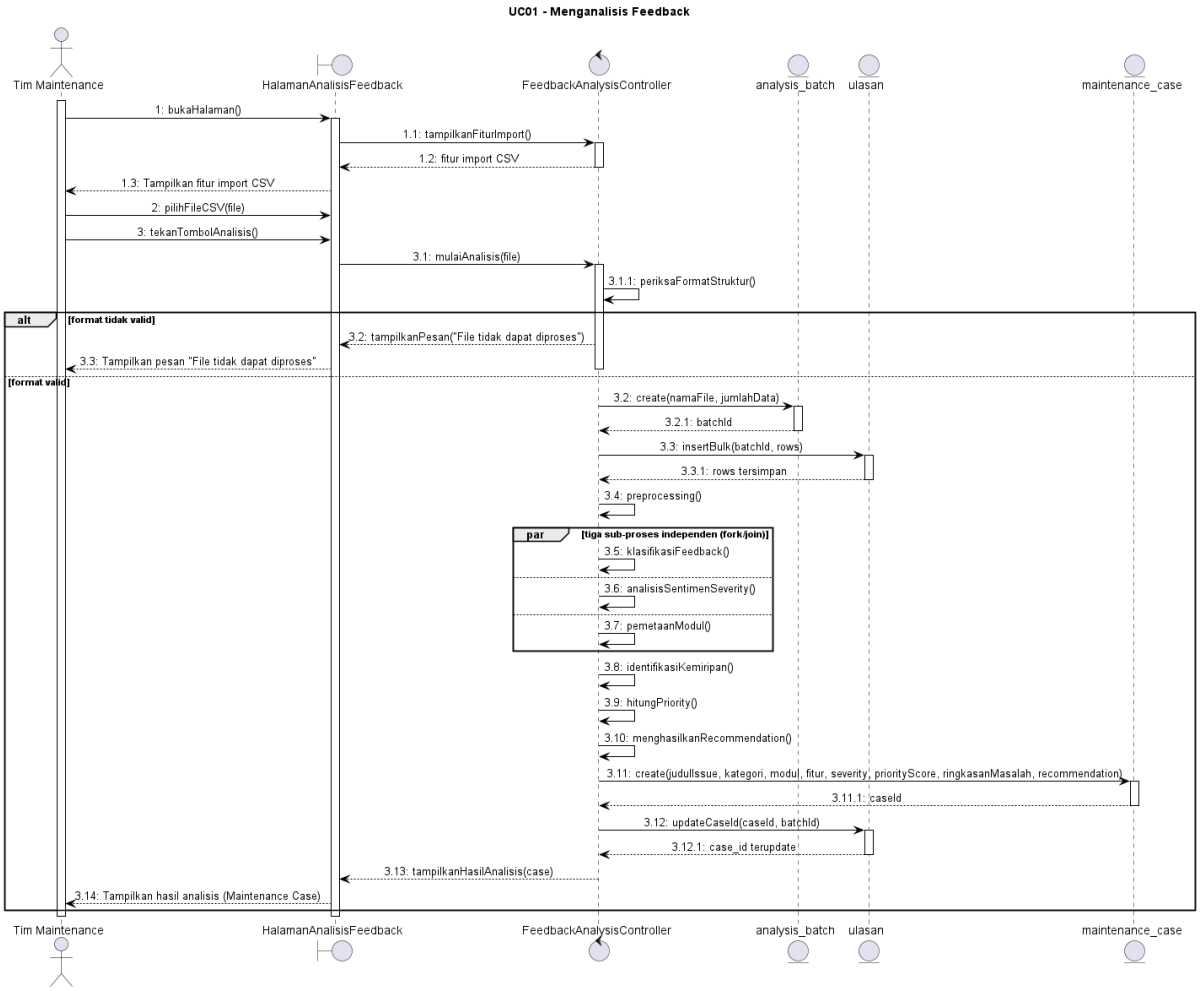
F. Sequence Diagram

1) Login



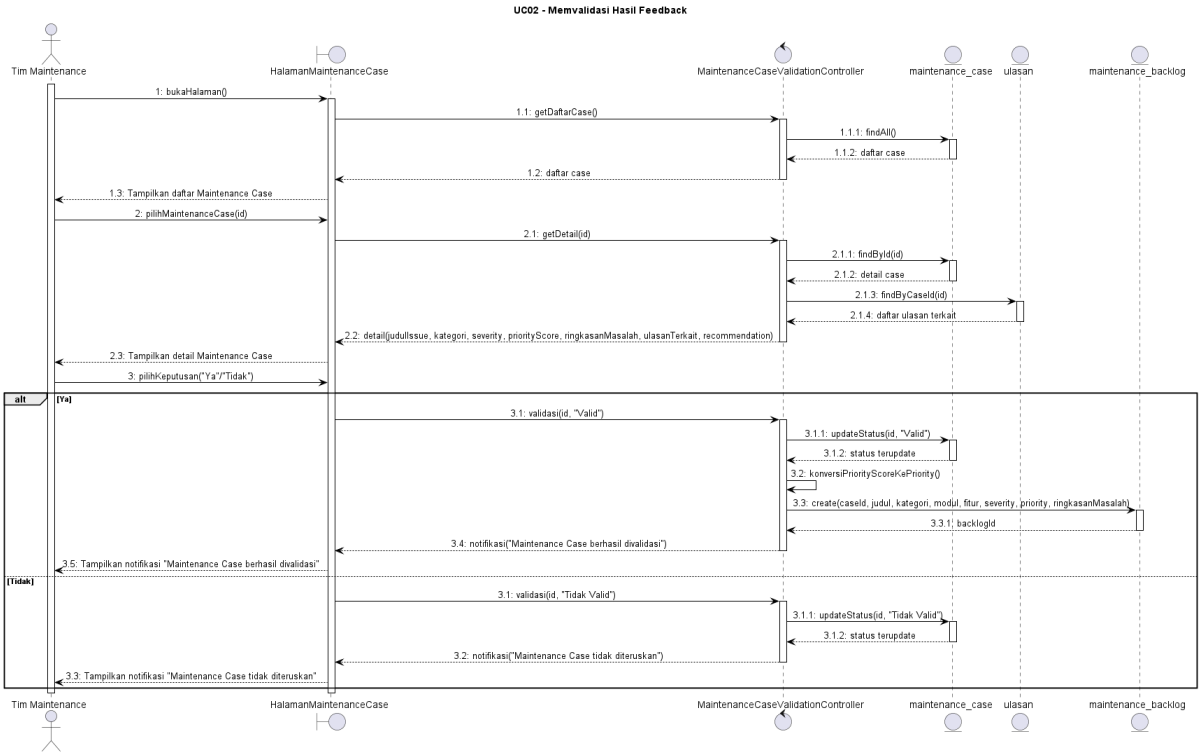
Gambar 6.14 *Sequence Diagram* Login

2) Menganalisis Feedback



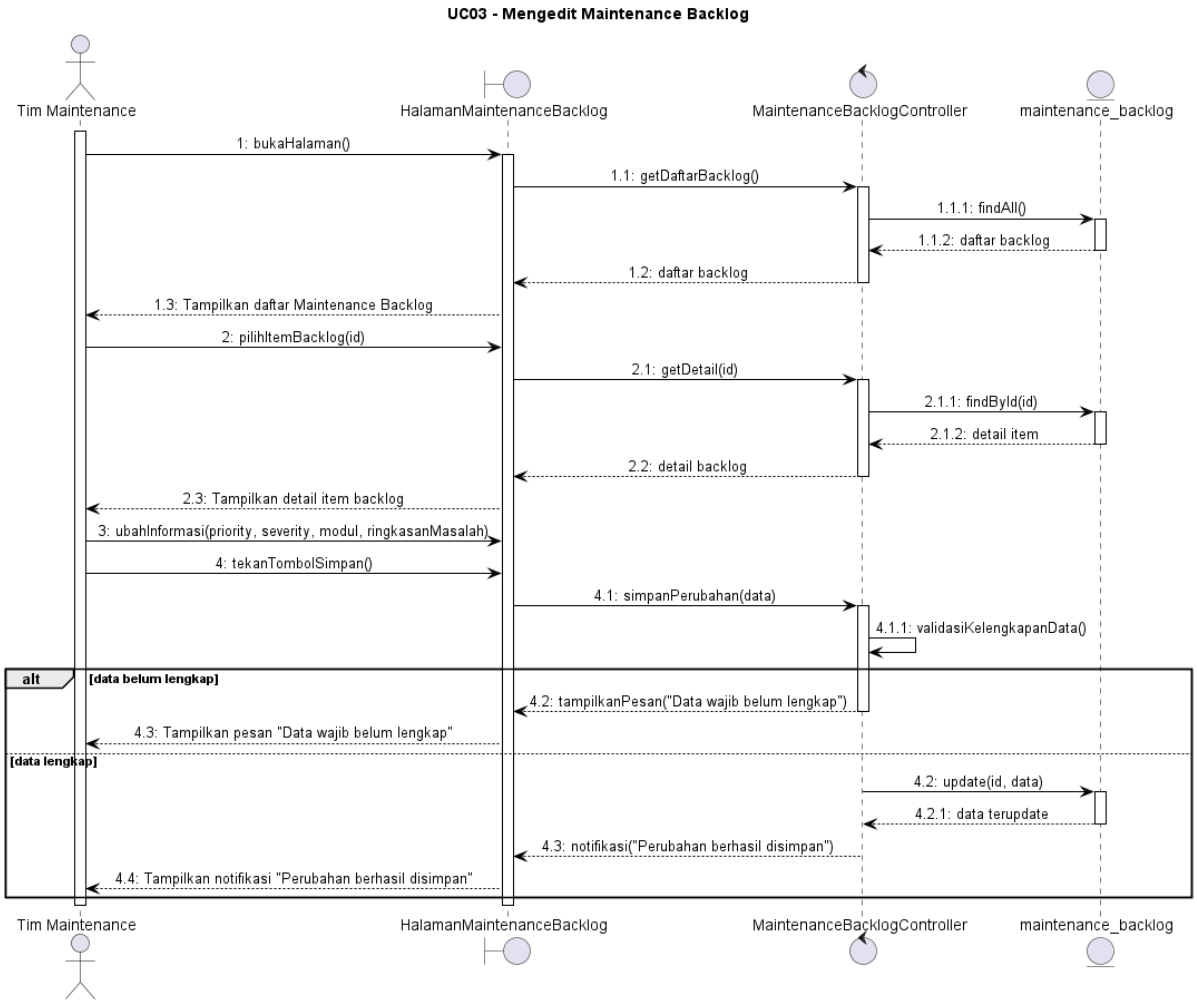
Gambar 6.15 *Sequence Diagram* Menganalisis Feedback

3) Memvalidasi Hasil Feedback



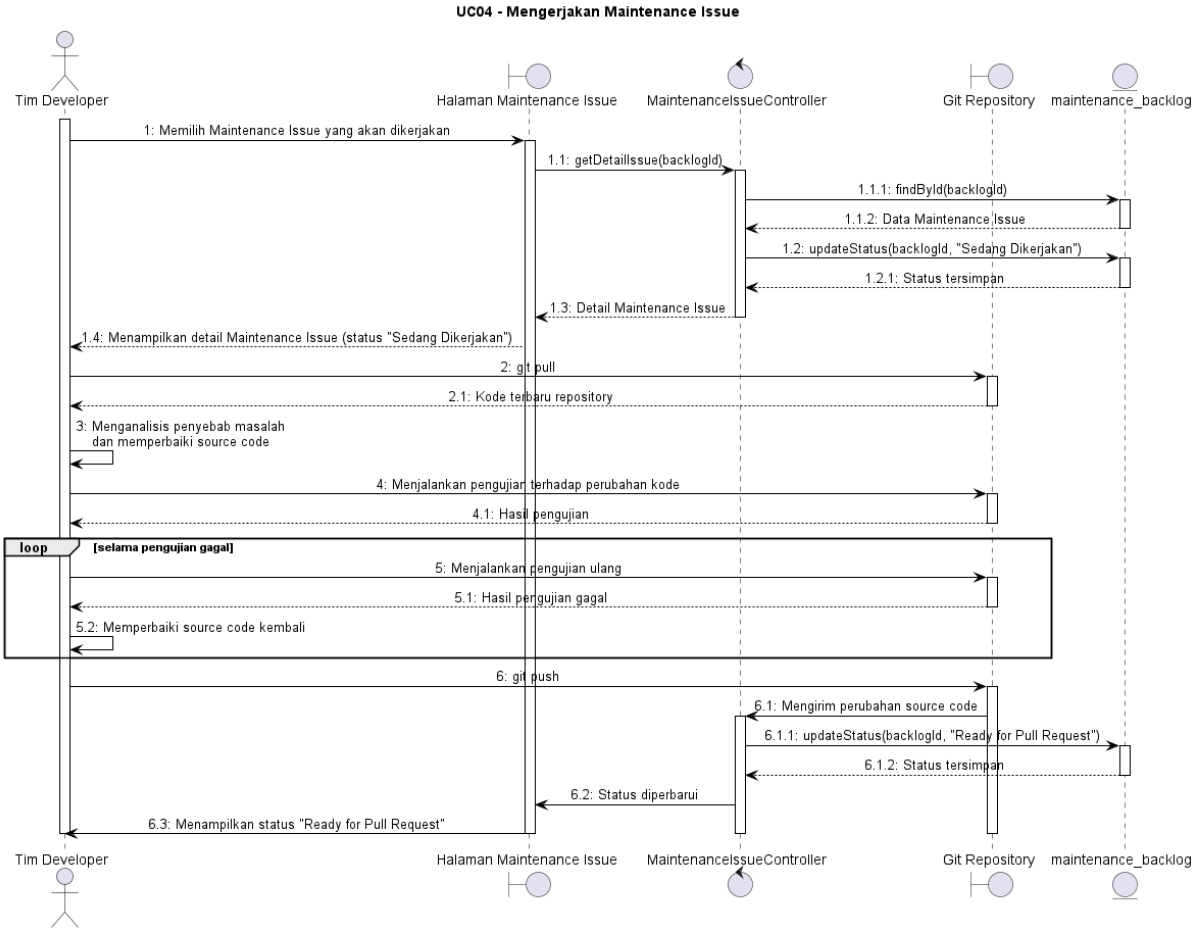
Gambar 6.16 *Sequence Diagram* Memvalidasi Hasil Feedback

4) Mengedit Maintenance Backlog



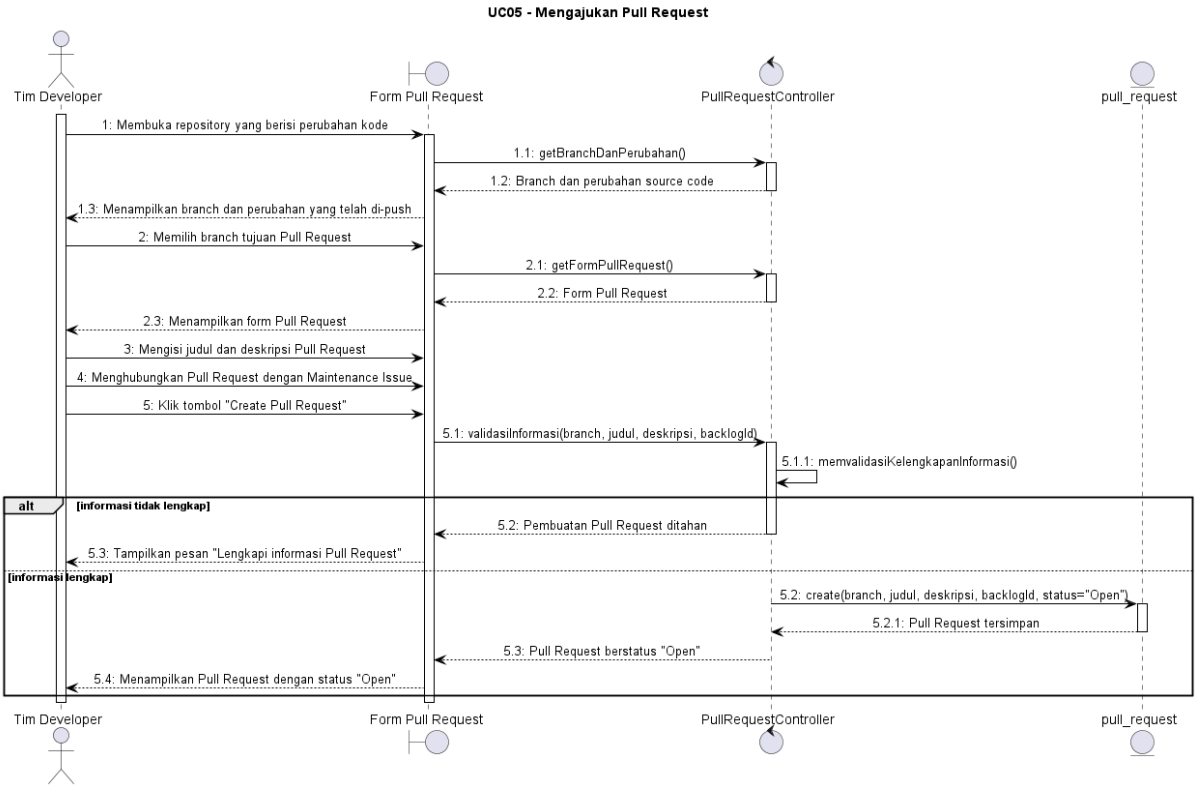
Gambar 6.17 *Sequence Diagram* Mengedit Maintenance Backlog

5) Mengerjakan Maintenance Issue



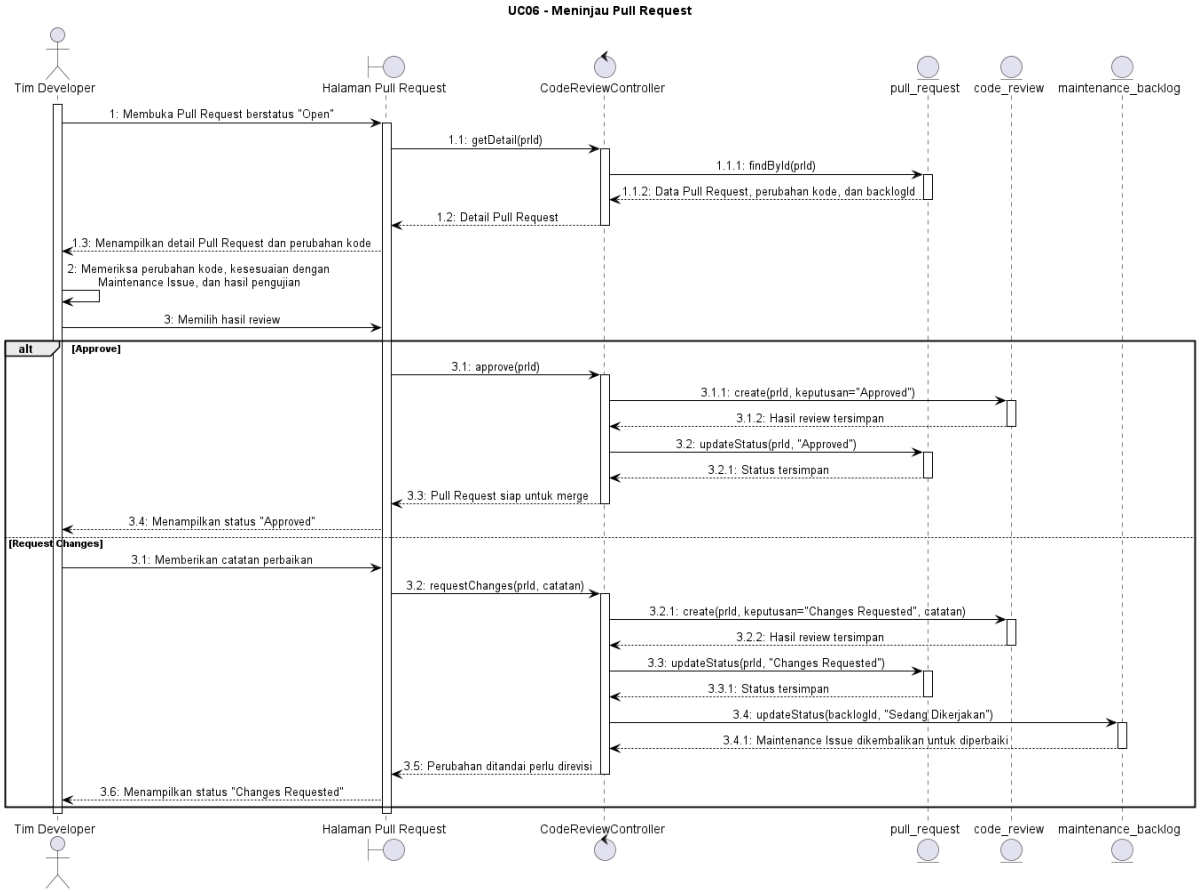
Gambar 6.18 *Sequence Diagram* Mengedit Maintenance Backlog

6) Mengajukan Pull Request



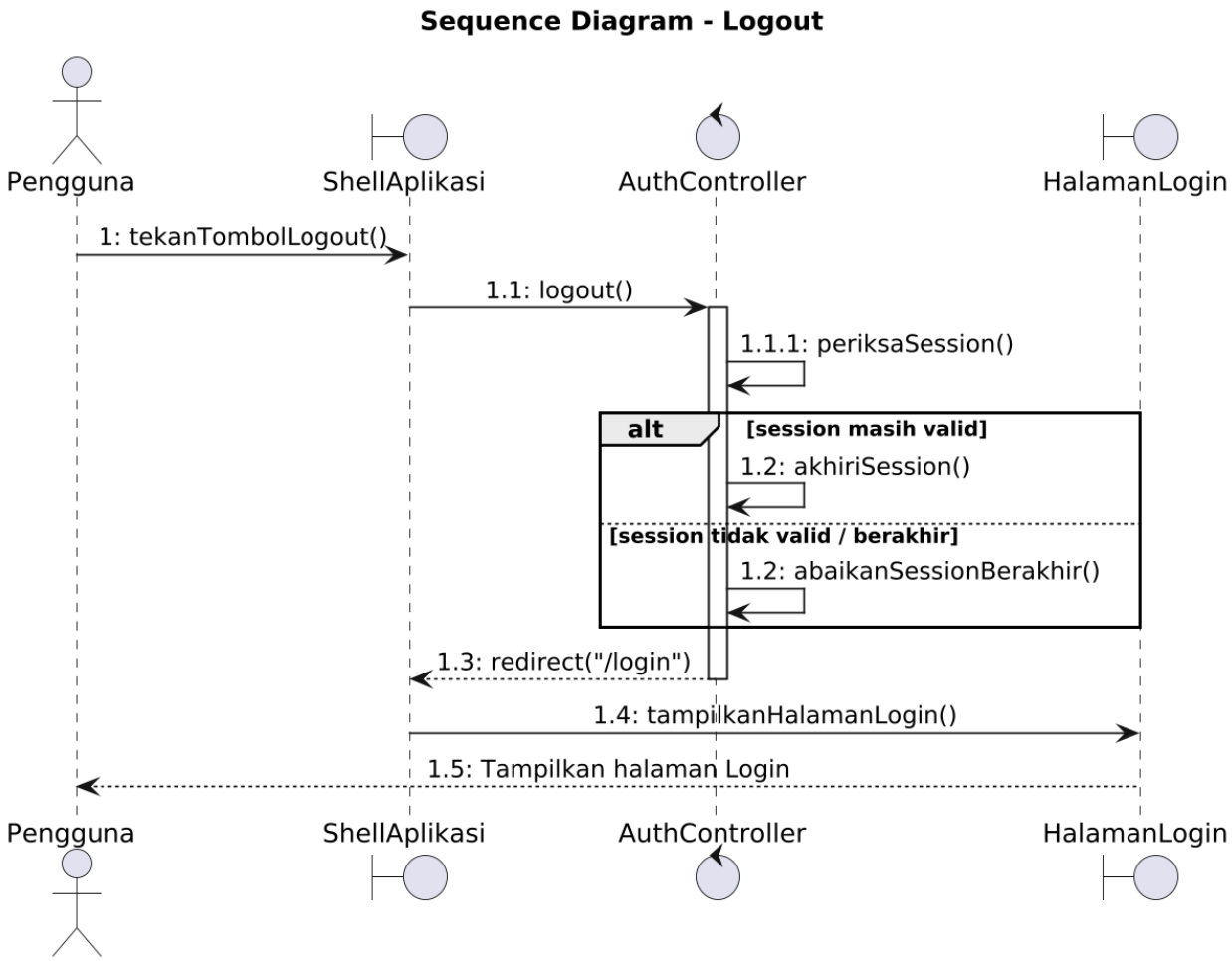
Gambar 6.19 *Sequence Diagram* Mengajukan Pull Request

7) Meninjau Pull Request



Gambar 6.20 *Sequence Diagram* Meninjau Pull Request

8) Logout



Gambar 6.21 *Sequence Diagram* Logout

Appendix C: To Be Determined List

Bagian To Be Determined (TBD) mencatat aspek MaintainGov yang akan diselesaikan pada tahap implementasi setelah perancangan SRS. Fokusnya adalah keputusan teknis, integrasi antarlayanan, pengujian, dan mekanisme fallback ketika suatu komponen mengalami kegagalan. Daftar ini menjadi acuan pengembangan berikutnya, bukan menunjukkan fitur yang sudah berhasil diimplementasikan.

Tabel 6.9 To Be Determined List MaintainGov

No.	Reference Section	TBD Item	Implementation Plan & Fallback	Responsible Party	Target Resolution
1	2.4 Operating Environment	Konfigurasi lingkungan aplikasi	Konfigurasi Next.js, Laravel, FastAPI, dan MySQL. Fallback: menjalankan layanan secara lokal apabila server produksi belum tersedia.	TBD	Initial Development
2	3.1 User Interfaces	Implementasi Login, Register, dan verifikasi akun	Mengembangkan autentikasi dan aktivasi akun internal. Fallback: penyediaan akun secara internal apabila registrasi belum siap.	TBD	Authentication Development
3	3.3 Software Interfaces	Implementasi model Feedback Analysis	Mengevaluasi model klasifikasi, sentimen, dan ekstraksi masalah. Fallback: pendekatan berbasis aturan yang hasilnya tetap memerlukan validasi Tim Maintenance.	TBD	Analysis Development

4	3.3 Software Interfaces	Pengelompokan issue dan perhitungan prioritas	Menguji metode kemiripan dan formula prioritas. Fallback: pengelompokan berbasis aturan dan prioritas yang ditinjau secara manual.	TBD	Analysis Development
5	3.4 Communications Interfaces	Integrasi backend dengan Analysis Service	Mengembangkan komunikasi API dan pemrosesan Analysis Batch. Fallback: menyimpan status gagal atau menunggu, kemudian menjalankan ulang proses.	TBD	Integration Testing
6	3.3–3.4 Software and Communications Interfaces	Integrasi GitHub API	Mengimplementasikan pembuatan PR, pengambilan hasil review, dan sinkronisasi status. Fallback: aktivitas dilakukan melalui GitHub, kemudian metadata disinkronkan setelah koneksi pulih.	TBD	GitHub Integration
7	4.4–4.8 System Features	Konsistensi perubahan status workflow	Mengimplementasikan validasi status dan transaksi database. Fallback: membatalkan perubahan yang gagal dan mempertahankan data terakhir yang valid.	TBD	Core Feature Development
8	5.1 Performance Requirements	Optimasi performa aplikasi dan analisis	Menguji waktu respons dan kapasitas pemrosesan CSV. Fallback: membatasi ukuran batch atau menggunakan pemrosesan bertahap.	TBD	Performance Testing

9	5.2 Safety Requirements	Mekanisme backup dan pemulihan data	Menyiapkan pencadangan berkala dan prosedur pemulihan MySQL. Fallback: melakukan pemulihan dari cadangan valid terakhir.	TBD	Pre-Deployment
10	5.3 Security Requirements	Pengamanan sesi dan kredensial integrasi	Menerapkan kontrol akses, enkripsi komunikasi, serta penyimpanan kredensial yang aman. Fallback: menonaktifkan sementara integrasi atau akses yang tidak dapat diverifikasi.	TBD	Security Testing
11	5.4 Software Quality Attributes	Pengujian sistem dan penerimaan pengguna	Melaksanakan unit testing, integration testing, dan UAT untuk UC01–UC06. Fallback: memperbaiki fitur yang gagal dan melakukan pengujian ulang sebelum rilis.	TBD	Pre-Release

Appendix D: Kontribusi Anggota Kelompok

Tabel 6.10 Kontribusi Anggota Kelompok

No	Foto Diri	Nama / NIM	Kontribusi
1		Gerard Viriyandra / 187241047	- Memimpin dan mengawasi project - BPMN To-Be - Melakukan setup GitHub - System description - System architecture - Use Case Diagram - Use Case Specification (UC 4-6) - Activity Diagram (UC 4-6) - Sequence Diagram (UC 4-6) - User Story Scenario (UC 4-6) - Menyusun SRS - Membuat Data Mockup
2		Demas Axel Marfiansyah / 187241043	- BPMN As-Is - CDM - PDM - Class Diagram - Use Case Specification (UC 1-3) - Activity Diagram (UC 1-3) - Sequence Diagram (UC 1-3) - User Story Scenario (UC 1-3) - Menyusun SRS

3		Valisha Vine Ravelina / 187241012	- Lo-fi - Hi-fi - Membuat logo MaintainGov - User Story - Sequence Diagram (UC 1-3) - Menyusun SRS
4		Muhammad Hanif / 187241062	- Lo-fi - Use Case Specification Login & Logout - Activity Diagram Login & Logout - Sequence Diagram Login & Logout - User Story Scenario Login & Logout - Menyusun SRS